Vidyalankar Institute of Technology (VIT)

Wadala (W), Mumbai – 400 037 | Accredited A+ by NAAC

Department of Information Technology

Subject	Data Warehousing and Mining (DWM – PCIT10T / PCIT10P)		
Experiment No.	01	**Semester / Division**	T.E. Semester V | TEINFT - A (Odd Sem 2026-2027)
Laboratory Batch	Batch - 3	**Date of Performance**	—
Student 1	Manthan Shah (Roll No: 24101A0078)	**Student 2**	Ishan Gurav (Roll No: 24101A0077)
Professor In-charge	Dr. Varsha Bhosale		

Experiment 01: Build Data Warehouse / Data Mart & Design Dimensional Data Models

1. Aim / Objective

To design and implement a **Data Warehouse** for the Stock Market dataset using three dimensional modelling techniques: *Star Schema*, *Snowflake Schema*, and *Fact Constellation (Galaxy) Schema*. The warehouse stores daily trade facts (Open, High, Low, Close prices, Volume, Daily Return) together with conformed time, stock, and exchange dimensions.

2. Theory

2.1 Data Warehouse

A *Data Warehouse (DW)* is a subject-oriented, integrated, time-variant, and non-volatile collection of data used to support management decision-making (Inmon, 1992). It separates analytical workloads from operational databases (OLTP) by storing pre-aggregated, historical data in a schema optimised for read-heavy OLAP queries.

2.2 Star Schema

The **Star Schema** is the simplest dimensional model. A central *Fact table* (`Fact_DailyTrades`) holds quantitative trade metrics and foreign keys to denormalized *Dimension tables* (`Dim_Time`, `Dim_Stock`, `Dim_Exchange`). Queries involve at most one join per dimension, delivering the fastest analytical performance.

2.3 Snowflake Schema

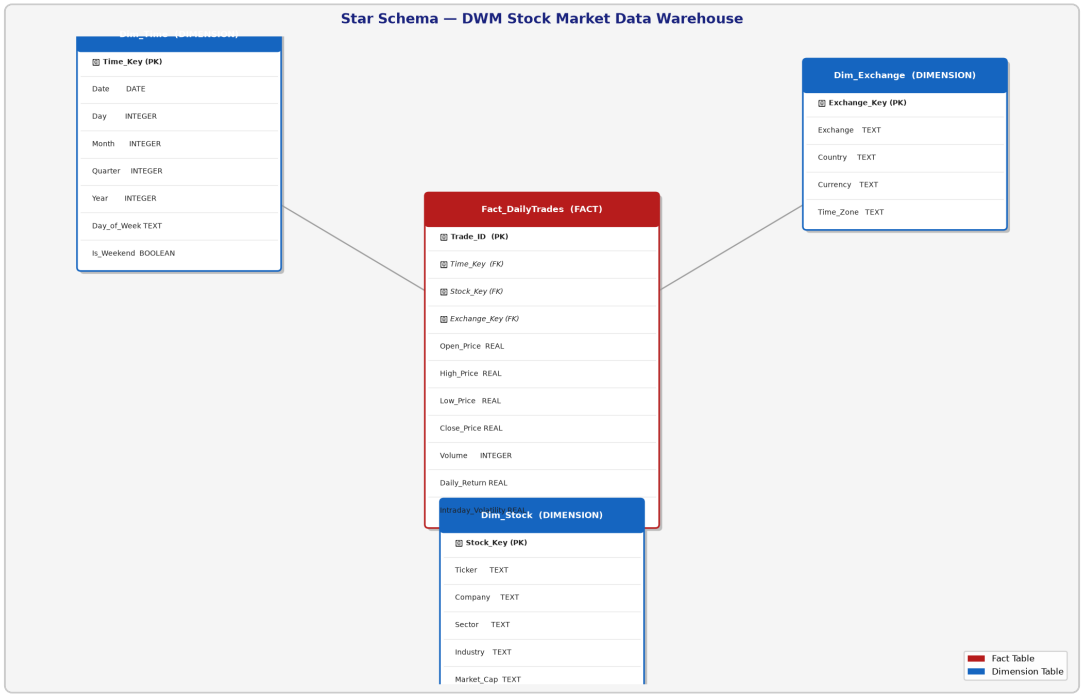
The **Snowflake Schema** extends the star by further normalizing dimension tables. `Dim_Stock_Snowflake` references a separate `Dim_Sector` sub-dimension, eliminating redundant sector/industry storage at the cost of additional joins.

2.4 Fact Constellation (Galaxy) Schema

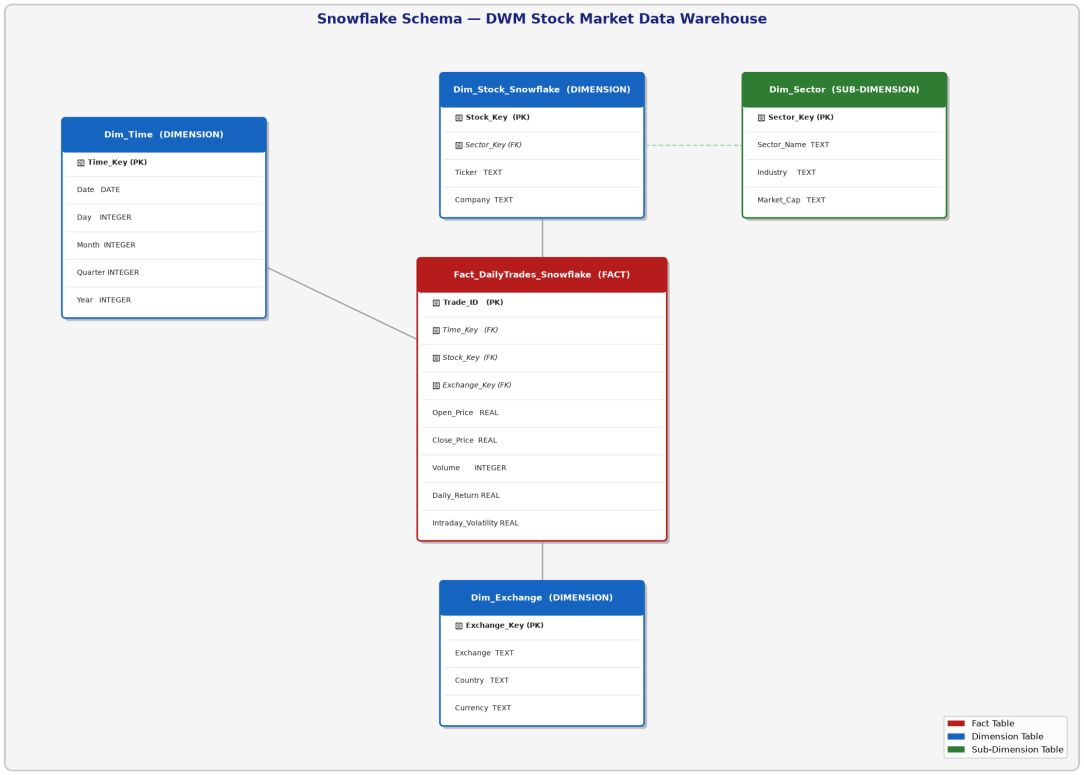
The **Fact Constellation** supports multiple fact tables sharing *conformed dimensions*. Here, `Fact_DailyTrades` and `Fact_QuarterlyFinancials` both reference `Dim_Time` and `Dim_Stock`, enabling cross-fact drill-across OLAP queries.

3. ER Diagrams

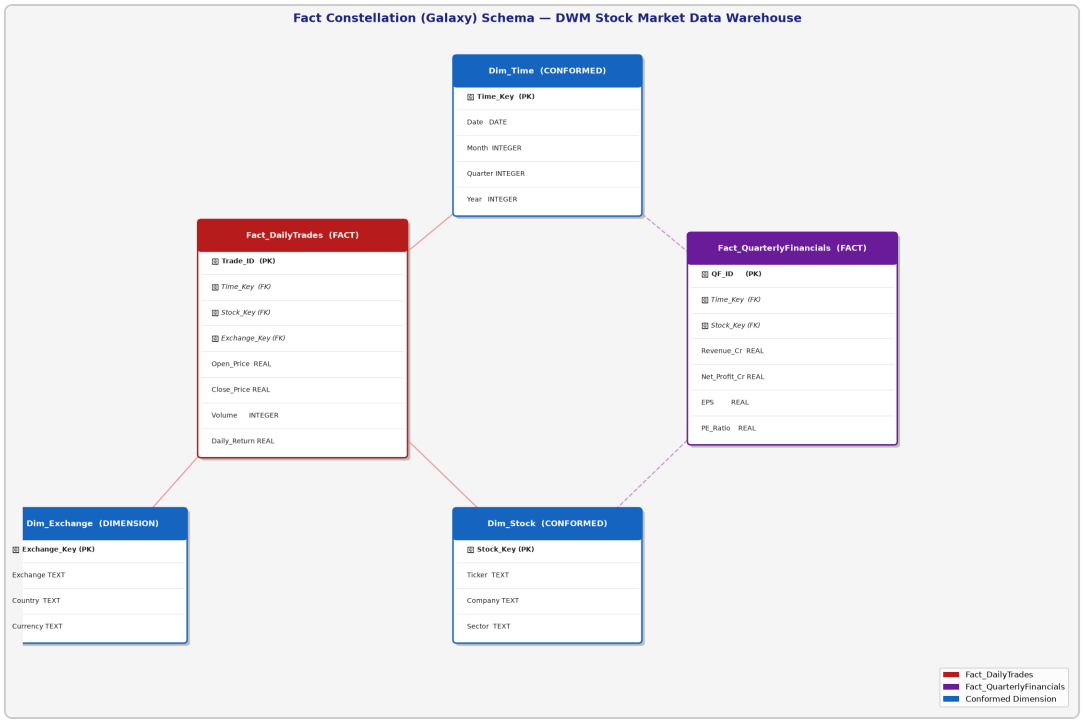
3.1 Star Schema



3.2 Snowflake Schema



3.3 Fact Constellation (Galaxy) Schema



4. DDL Scripts

4.1 Star Schema DDL

```
-- ═══════════════════════════════════════════════════════
--  STAR SCHEMA DDL
-- ═══════════════════════════════════════════════════════
```

```
CREATE TABLE IF NOT EXISTS Dim_Time (
    Time_Key    INTEGER PRIMARY KEY AUTOINCREMENT,
    Date        DATE    NOT NULL UNIQUE,
    Day         INTEGER,
    Month       INTEGER,
    Quarter     INTEGER,
    Year        INTEGER,
    Day_of_Week TEXT,
    Is_Weekend  BOOLEAN
);

CREATE TABLE IF NOT EXISTS Dim_Stock (
    Stock_Key   INTEGER PRIMARY KEY AUTOINCREMENT,
    Ticker      TEXT    NOT NULL UNIQUE,
    Company     TEXT,
    Sector      TEXT,
    Industry    TEXT,
    Market_Cap  TEXT
);

CREATE TABLE IF NOT EXISTS Dim_Exchange (
    Exchange_Key  INTEGER PRIMARY KEY AUTOINCREMENT,
    Exchange      TEXT    NOT NULL UNIQUE,
    Country       TEXT,
    Currency      TEXT,
    Time_Zone     TEXT
);

CREATE TABLE IF NOT EXISTS Fact_DailyTrades (
    Trade_ID            INTEGER PRIMARY KEY AUTOINCREMENT,
    Time_Key            INTEGER REFERENCES Dim_Time(Time_Key),
    Stock_Key           INTEGER REFERENCES Dim_Stock(Stock_Key),
    Exchange_Key        INTEGER REFERENCES Dim_Exchange(Exchange_Key),
    Open_Price          REAL,
    High_Price          REAL,
    Low_Price           REAL,
    Close_Price         REAL,
    Volume              INTEGER,
    Daily_Return        REAL,
    Intraday_Volatility REAL
);
```

4.2 Snowflake Schema DDL

```
-- ═══════════════════════════════════════════
--  SNOWFLAKE SCHEMA DDL
-- ═══════════════════════════════════════════

CREATE TABLE IF NOT EXISTS Dim_Sector (
    Sector_Key  INTEGER PRIMARY KEY AUTOINCREMENT,
    Sector_Name TEXT    NOT NULL UNIQUE,
    Industry    TEXT,
    Market_Cap  TEXT
);

CREATE TABLE IF NOT EXISTS Dim_Stock_Snowflake (
    Stock_Key   INTEGER PRIMARY KEY AUTOINCREMENT,
    Sector_Key  INTEGER REFERENCES Dim_Sector(Sector_Key),
    Ticker      TEXT    NOT NULL UNIQUE,
    Company     TEXT
);

CREATE TABLE IF NOT EXISTS Fact_DailyTrades_Snowflake (
    Trade_ID            INTEGER PRIMARY KEY AUTOINCREMENT,
    Time_Key            INTEGER REFERENCES Dim_Time(Time_Key),
    Stock_Key           INTEGER REFERENCES
Dim_Stock_Snowflake(Stock_Key),
    Exchange_Key        INTEGER REFERENCES Dim_Exchange(Exchange_Key),
```

```
    Open_Price          REAL,
    Close_Price         REAL,
    Volume              INTEGER,
    Daily_Return        REAL,
    Intraday_Volatility REAL
);
```

4.3 Fact Constellation DDL

```
-- ═══════════════════════════════════════════════
--  FACT CONSTELLATION (GALAXY) SCHEMA DDL
-- ═══════════════════════════════════════════════

-- Conformed dimensions: Dim_Time, Dim_Stock, Dim_Exchange  (reused
from Star)

CREATE TABLE IF NOT EXISTS Fact_QuarterlyFinancials (
    QF_ID         INTEGER PRIMARY KEY AUTOINCREMENT,
    Time_Key      INTEGER REFERENCES Dim_Time(Time_Key),
    Stock_Key     INTEGER REFERENCES Dim_Stock(Stock_Key),
    Revenue_Cr    REAL,
    Net_Profit_Cr REAL,
    EPS           REAL,
    PE_Ratio      REAL
);

-- Fact_DailyTrades already created under Star Schema (shared via
conformed dims)
```

5. Data Loading Summary

Table	Rows Loaded	Description
Dim_Time	740	Unique trading dates
Dim_Stock	20	Unique stock tickers
Dim_Exchange	1	Unique exchanges
Fact_DailyTrades	14,800	Total daily trade records

Conclusion: The Star, Snowflake, and Fact Constellation schemas were successfully designed and implemented for the stock market dataset. The Star Schema offers optimal query performance; the Snowflake Schema reduces storage redundancy by normalising sector data; and the Galaxy Schema enables multi-fact analytical queries using conformed dimensions shared across daily trades and quarterly financials.

Vidyalankar Institute of Technology (VIT)

Wadala (W), Mumbai – 400 037 | Accredited A+ by NAAC

Department of Information Technology

Subject	Data Warehousing and Mining (DWM – PCIT10T / PCIT10P)		
Experiment No.	02	**Semester / Division**	T.E. Semester V | TEINFT - A (Odd Sem 2026-2027)
Laboratory Batch	Batch - 3	**Date of Performance**	—
Student 1	Manthan Shah (Roll No: 24101A0078)	**Student 2**	Ishan Gurav (Roll No: 24101A0077)
Professor In-charge	Dr. Varsha Bhosale		

Experiment 02: OLAP Operations on the Dimensional Model

1. Aim / Objective

To perform standard OLAP (Online Analytical Processing) operations — **Slice, Dice, Roll-Up, Drill-Down, Pivot,** and **Drill-Across** — on the Stock Market Data Warehouse built in Experiment 01, using SQL queries executed against a SQLite in-memory warehouse populated from `stock_market_data.csv`.

2. Theory

2.1 OLAP Concepts

OLAP enables multidimensional analysis of business data. Operations traverse the *data cube* defined by the warehouse dimensions (Time, Stock, Exchange).

Operation	Description
Slice	Selects a single value of one dimension, producing a sub-cube.
Dice	Applies range/value filters on two or more dimensions.
Roll-Up	Aggregates data up a hierarchy (Month → Quarter → Year).
Drill-Down	Decomposes aggregated data to a finer granularity.
Pivot	Rotates the data cube to present a cross-tabulated view.
Drill-Across	Queries across multiple fact tables sharing conformed dimensions.

3. OLAP Query Results

3.1 Slice Operation

Extracts a single dimension value — all trades for the NSE exchange.

```
SELECT t.Year, s.Ticker, s.Sector,
       ROUND(AVG(f.Close_Price), 2)   AS Avg_Close,
       ROUND(AVG(f.Daily_Return), 4)  AS Avg_Return,
       SUM(f.Volume)                  AS Total_Volume
  FROM Fact_DailyTrades f
  JOIN Dim_Time     t ON f.Time_Key     = t.Time_Key
  JOIN Dim_Stock    s ON f.Stock_Key    = s.Stock_Key
  JOIN Dim_Exchange e ON f.Exchange_Key = e.Exchange_Key
 WHERE e.Exchange = 'NSE'
 GROUP BY t.Year, s.Ticker, s.Sector
 ORDER BY t.Year, s.Ticker
 LIMIT 10;
```

Result:

Year	Ticker	Sector	Avg_Close	Avg_Return	Total_Volume
2023	BAJAJ-AUTO	Auto	5531.28	0.3692	26873761
2023	BRITANNIA	FMCG	4574.6	0.2342	17371937
2023	HCLTECH	IT	1191.07	0.1342	129286188
2023	HDFCBANK	Banking	747.83	0.0275	1728255410
2023	HINDUNILVR	FMCG	2400.49	0.1179	83087327
2023	ICICIBANK	Banking	936.7	0.0062	721050143
2023	INFY	IT	1334.7	0.0834	294771285
2023	ITC	FMCG	381.75	-0.0942	557655398
2023	KOTAKBANK	Banking	355.43	0.0283	877373290
2023	M&M	Auto	1551.88	0.027	129424567

3.2 Dice Operation

Applies filters on two+ dimensions — IT sector stocks in 2023 with positive returns.

```
SELECT t.Year, t.Quarter, s.Ticker, s.Sector,
       ROUND(AVG(f.Close_Price), 2)   AS Avg_Close,
       ROUND(AVG(f.Daily_Return), 4)  AS Avg_Return,
       SUM(f.Volume)                  AS Total_Volume
  FROM Fact_DailyTrades f
  JOIN Dim_Time  t ON f.Time_Key  = t.Time_Key
  JOIN Dim_Stock s ON f.Stock_Key = s.Stock_Key
 WHERE s.Sector = 'IT'
   AND t.Year   = 2023
   AND f.Daily_Return > 0
 GROUP BY t.Year, t.Quarter, s.Ticker
 ORDER BY t.Quarter, Avg_Return DESC
 LIMIT 10;
```

Result:

Year	Quarter	Ticker	Sector	Avg_Close	Avg_Return	Total_Volume
2023	4	WIPRO	IT	196.22	1.6579	388519914
2023	4	HCLTECH	IT	1200.61	1.0567	77767475
2023	4	INFY	IT	1344.17	1.0012	164525356
2023	4	TCS	IT	3268.61	0.8569	53685670

3.3 Roll-Up Operation

Aggregates from Month → Quarter → Year hierarchy.

```
SELECT t.Year, t.Quarter,
       ROUND(AVG(f.Close_Price),  2)  AS Avg_Close,
       ROUND(AVG(f.Daily_Return), 4)  AS Avg_Daily_Return,
       SUM(f.Volume)                  AS Total_Volume,
       COUNT(*)                       AS Trade_Days
  FROM Fact_DailyTrades f
  JOIN Dim_Time t ON f.Time_Key = t.Time_Key
 GROUP BY t.Year, t.Quarter
 ORDER BY t.Year, t.Quarter
 LIMIT 12;
```

Result:

Year	Quarter	Avg_Close	Avg_Daily_Return	Total_Volume	Trade_Days
2023.0	4.0	1839.23	0.0597	9774429862.0	1060.0
2024.0	1.0	2060.93	-0.0139	15577733283.0	1200.0
2024.0	2.0	2235.85	-0.0657	15869722414.0	1200.0
2024.0	3.0	2453.39	0.0291	14911286734.0	1280.0
2024.0	4.0	2339.47	-0.2116	11996598216.0	1240.0
2025.0	1.0	2216.11	-0.089	11113388964.0	1240.0
2025.0	2.0	2249.13	0.1214	11742491800.0	1220.0
2025.0	3.0	2388.7	-0.1293	9025015077.0	1280.0
2025.0	4.0	2547.37	0.0182	9050884518.0	1240.0
2026.0	1.0	2467.52	-0.1455	14077419796.0	1200.0
2026.0	2.0	2313.45	-0.0594	14836920973.0	1200.0
2026.0	3.0	2381.84	-0.098	10952232439.0	1300.0

3.4 Drill-Down Operation

Decomposes from Year → Quarter → Month for INFY.

```
SELECT t.Year, t.Quarter, t.Month,
       ROUND(AVG(f.Close_Price),  2) AS Avg_Close,
       ROUND(AVG(f.Daily_Return), 4) AS Avg_Return,
       SUM(f.Volume)                 AS Total_Volume
  FROM Fact_DailyTrades f
  JOIN Dim_Time  t ON f.Time_Key  = t.Time_Key
  JOIN Dim_Stock s ON f.Stock_Key = s.Stock_Key
 WHERE s.Ticker = 'INFY'
 GROUP BY t.Year, t.Quarter, t.Month
 ORDER BY t.Year, t.Quarter, t.Month
 LIMIT 12;
```

Result:

Year	Quarter	Month	Avg_Close	Avg_Return	Total_Volume
2023.0	4.0	10.0	1289.59	-0.1677	79507358.0
2023.0	4.0	11.0	1303.97	-0.023	85186084.0
2023.0	4.0	12.0	1394.75	0.353	130077843.0

Year	Quarter	Month	Avg_Close	Avg_Return	Total_Volume
2024.0	1.0	1.0	1469.26	0.0533	165823157.0
2024.0	1.0	2.0	1549.02	0.0133	101673806.0
2024.0	1.0	3.0	1459.2	-0.3456	135900753.0
2024.0	2.0	4.0	1342.27	-0.2435	185453567.0
2024.0	2.0	5.0	1326.73	0.0424	174897064.0
2024.0	2.0	6.0	1410.3	0.1263	177134468.0
2024.0	3.0	7.0	1634.49	0.4341	192519323.0
2024.0	3.0	8.0	1732.42	-0.07	124670491.0
2024.0	3.0	9.0	1801.31	-0.1538	134369302.0

3.5 Pivot Operation

Cross-tabulates average daily return per sector per year.

```
SELECT s.Sector,
       ROUND(AVG(CASE WHEN t.Year = 2021 THEN f.Daily_Return END), 4)
AS [2021],
       ROUND(AVG(CASE WHEN t.Year = 2022 THEN f.Daily_Return END), 4)
AS [2022],
       ROUND(AVG(CASE WHEN t.Year = 2023 THEN f.Daily_Return END), 4)
AS [2023],
       ROUND(AVG(CASE WHEN t.Year = 2024 THEN f.Daily_Return END), 4)
AS [2024]
  FROM Fact_DailyTrades f
  JOIN Dim_Stock s ON f.Stock_Key = s.Stock_Key
  JOIN Dim_Time  t ON f.Time_Key  = t.Time_Key
 GROUP BY s.Sector
 ORDER BY s.Sector;
```

Result:

Sector	2021	2022	2023	2024
Auto	None	None	0.0346	-0.0574
Banking	None	None	0.0008	0.0147
Energy	None	None	0.0768	-0.1276
FMCG	None	None	0.0801	-0.1353
IT	None	None	0.1064	-0.0203

3.6 Drill-Across Operation

Joins two fact tables (DailyTrades + QuarterlyFinancials) via conformed dims.

```
SELECT s.Ticker, s.Sector, t.Year, t.Quarter,
       ROUND(AVG(f.Daily_Return), 4) AS Avg_Daily_Return,
       ROUND(AVG(f.Volume),       0) AS Avg_Volume,
       ROUND(AVG(q.EPS),          2) AS Avg_EPS,
       ROUND(AVG(q.PE_Ratio),     2) AS Avg_PE
  FROM Fact_DailyTrades f
  JOIN Dim_Stock s ON f.Stock_Key = s.Stock_Key
  JOIN Dim_Time  t ON f.Time_Key  = t.Time_Key
  JOIN Fact_QuarterlyFinancials q
       ON q.Stock_Key = f.Stock_Key AND q.Time_Key = f.Time_Key
 GROUP BY s.Ticker, t.Year, t.Quarter
```

```
ORDER BY s.Ticker, t.Year, t.Quarter
LIMIT 10;
```

Result:

Ticker	Sector	Year	Quarter	Avg_Daily_Return	Avg_Volume	Avg_EPS	Avg_PE
BAJAJ-AUTO	Auto	2023	4	0.42	472168.0	53.53	22.97
BAJAJ-AUTO	Auto	2024	1	-1.02	253274.0	76.58	24.56
BAJAJ-AUTO	Auto	2024	2	-1.66	419820.0	5.49	39.36
BAJAJ-AUTO	Auto	2024	3	0.29	289355.0	62.36	27.72
BAJAJ-AUTO	Auto	2024	4	-2.07	670828.0	42.1	22.26
BAJAJ-AUTO	Auto	2025	1	-0.15	558535.0	42.65	29.74
BAJAJ-AUTO	Auto	2025	2	4.74	283618.0	45.98	33.94
BAJAJ-AUTO	Auto	2025	3	0.35	363782.0	14.8	27.62
BAJAJ-AUTO	Auto	2025	4	-1.07	551282.0	7.5	10.81
BAJAJ-AUTO	Auto	2026	1	2.06	306936.0	47.09	38.3

Conclusion: All six OLAP operations were successfully executed on the Stock Market Data Warehouse. Slice isolated NSE-listed stocks; Dice filtered IT-sector positive-return days in 2023; Roll-Up aggregated trade metrics to the quarterly level; Drill-Down decomposed INFY's performance month-by-month; Pivot cross-tabulated sector-wise average returns by year; and Drill-Across joined daily trade facts with quarterly financials via conformed dimensions, demonstrating the power of a multi-fact constellation schema.

Data Warehousing and Mining (DWM) — Experiment 03

Experiment Title: Data Preprocessing, Exploration and Visualization

Dataset: `stock_market_data.csv` (3 Years of Daily NSE Trading Data across 20 Major Indian Stocks & 5 Sectors)

1. Objectives

- Perform comprehensive **data preprocessing**, type normalization, and temporal feature engineering on the multi-sector Indian stock market trading dataset.
- Conduct **exploratory data analysis (EDA)** and statistical aggregation to understand price distributions, daily returns, trading volumes, and intraday volatility.
- Build **multi-dimensional visualizations** across stocks, sectors, and time horizons to uncover liquidity patterns, risk-return characteristics, outliers, and feature correlations.
- Prepare and validate the dataset for downstream data mining tasks (regression, classification, clustering, and association rule mining).

---

2. Library Imports

```
import warnings
warnings.filterwarnings('ignore')

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

# Set seaborn whitegrid theme and global plot styling
sns.set_theme(style="whitegrid", palette="deep")
plt.rcParams['figure.figsize'] = (11, 6)
plt.rcParams['axes.titlesize'] = 14
plt.rcParams['axes.titleweight'] = 'bold'
plt.rcParams['axes.labelsize'] = 12

print("Libraries imported successfully.")
```

```
Libraries imported successfully.
```

---

3. Dataset Ingestion

Load `stock_market_data.csv` and inspect the initial records.

```
df = pd.read_csv('stock_market_data.csv')
print(f"Successfully loaded 'stock_market_data.csv' with {len(df):,} records.")
df.head()
```

```
Successfully loaded 'stock_market_data.csv' with 14,800 records.
```

	Date	Ticker	Sector	Exchange	Open_Price	High_Price	Low_Price	Close_Price	
0	2023-10-12	BAJAJ-AUTO	Auto	NSE	4851.06	4914.02	4818.72	4871.67	
1	2023-10-12	BRITANNIA	FMCG	NSE	4391.48	4400.68	4342.60	4371.11	
2	2023-10-12	HCLTECH	IT	NSE	1106.03	1109.18	1079.51	1087.15	
3	2023-10-12	HDFCBANK	Banking	NSE	735.43	743.05	733.28	742.02	2
4	2023-10-12	HINDUNILVR	FMCG	NSE	2413.48	2425.33	2398.55	2415.37	

---

4. Dataset Understanding

Inspect the dataset dimensions (`df.shape`), column names, data types, and structural summary (`df.info()`).

```
print("Dataset Shape (Rows, Columns):", df.shape)
print("\nColumn Names:", df.columns.tolist())
print("\nData Types:")
print(df.dtypes)
print("\nDataset Info:")
df.info()
```

```
Dataset Shape (Rows, Columns): (14800, 10)

Column Names: ['Date', 'Ticker', 'Sector', 'Exchange', 'Open_Price', 'High_Price',
'Low_Price', 'Close_Price', 'Volume', 'Daily_Return']

Data Types:
Date                str
Ticker              str
Sector              str
Exchange            str
Open_Price      float64
High_Price      float64
Low_Price       float64
Close_Price     float64
Volume            int64
Daily_Return    float64
dtype: object

Dataset Info:
<class 'pandas.DataFrame'>
RangeIndex: 14800 entries, 0 to 14799
Data columns (total 10 columns):
 #   Column        Non-Null Count  Dtype
---  ------        --------------  -----
 0   Date          14800 non-null  str
 1   Ticker        14800 non-null  str
 2   Sector        14800 non-null  str
 3   Exchange      14800 non-null  str
 4   Open_Price    14800 non-null  float64
 5   High_Price    14800 non-null  float64
 6   Low_Price     14800 non-null  float64
 7   Close_Price   14800 non-null  float64
 8   Volume        14800 non-null  int64
 9   Daily_Return  14800 non-null  float64
dtypes: float64(5), int64(1), str(4)
memory usage: 1.1 MB
```

---

5. Statistical Summary

Display the five-number summary and categorical/numerical distribution statistics using `df.describe(include='all').T`.

```
df.describe(include='all').T
```

	count	unique	top	freq	mean	std	min	25
Date	14800	740	2023-10-12	20	NaN	NaN	NaN	Na
Ticker	14800	20	BAJAJ-AUTO	740	NaN	NaN	NaN	Na
Sector	14800	5	Auto	2960	NaN	NaN	NaN	Na
Exchange	14800	1	NSE	14800	NaN	NaN	NaN	Na
Open_Price	14800.0	NaN	NaN	NaN	2297.538144	3168.633025	155.37	377.63
High_Price	14800.0	NaN	NaN	NaN	2319.915437	3198.874177	158.08	381.
Low_Price	14800.0	NaN	NaN	NaN	2273.379986	3134.604127	154.43	374.0
Close_Price	14800.0	NaN	NaN	NaN	2296.396325	3166.764305	155.29	377.75
Volume	14800.0	NaN	NaN	NaN	10166804.552568	13622470.91825	0.0	178607(
Daily_Return	14800.0	NaN	NaN	NaN	-0.052078	1.355666	-17.09	-0.

---

6. Missing & Duplicate Analysis

Verify data quality by checking missing values per column and duplicate records across the dataset.

```
missing_counts = df.isnull().sum()
duplicate_count = df.duplicated().sum()

print("Missing Values per Column:")
print(missing_counts)
print(f"\nTotal Missing Values: {missing_counts.sum()}")
print(f"Total Duplicate Rows: {duplicate_count}")
```

```
Missing Values per Column:
Date            0
Ticker          0
Sector          0
Exchange        0
Open_Price      0
High_Price      0
Low_Price       0
Close_Price     0
Volume          0
Daily_Return    0
dtype: int64

Total Missing Values: 0
Total Duplicate Rows: 0
```

---

7. Preprocessing & Feature Engineering

- Convert `Date` to `datetime` format.
- Extract temporal features: `Year`, `Month`, `MonthName`, and `DayOfWeek`.
- Derive `Intraday_Volatility` as $\frac{\text{High_Price} - \text{Low_Price}}{\text{Open_Price}} \times 100$.

• Verify cleaned dataset dimensions and preview the transformed table.

```
# Convert Date to datetime
df['Date'] = pd.to_datetime(df['Date'])

# Extract temporal attributes
df['Year'] = df['Date'].dt.year
df['Month'] = df['Date'].dt.month
df['MonthName'] = df['Date'].dt.month_name()
df['DayOfWeek'] = df['Date'].dt.day_name()

# Derive Intraday Volatility (%)
df['Intraday_Volatility'] = (((df['High_Price'] - df['Low_Price']) / df['Open_Price

print("Cleaned & Engineered Dataset Shape:", df.shape)
df.head()
```

```
Cleaned & Engineered Dataset Shape: (14800, 15)
```

	Date	Ticker	Sector	Exchange	Open_Price	High_Price	Low_Price	Close_Price	
0	2023-10-12	BAJAJ-AUTO	Auto	NSE	4851.06	4914.02	4818.72	4871.67	
1	2023-10-12	BRITANNIA	FMCG	NSE	4391.48	4400.68	4342.60	4371.11	
2	2023-10-12	HCLTECH	IT	NSE	1106.03	1109.18	1079.51	1087.15	
3	2023-10-12	HDFCBANK	Banking	NSE	735.43	743.05	733.28	742.02	2
4	2023-10-12	HINDUNILVR	FMCG	NSE	2413.48	2425.33	2398.55	2415.37	

---

8. Stock-wise Trading Volume Analysis

Aggregate total trading volume per `Ticker`, display the top 10 most actively traded stocks, and visualize them with a horizontal bar chart.

```
top_10_volume = (
    df.groupby(['Ticker', 'Sector'], as_index=False)['Volume']
    .sum()
    .sort_values(by='Volume', ascending=False)
    .head(10)
    .reset_index(drop=True)
)

print("Top 10 Most Actively Traded Stocks (3-Year Cumulative Volume):")
display(top_10_volume)

plt.figure(figsize=(11, 6))
ax = sns.barplot(
    data=top_10_volume,
    x='Volume',
    y='Ticker',
    hue='Sector',
    dodge=False,
    palette='viridis'
)
plt.title('Top 10 Stocks by Trading Volume')
plt.xlabel('Cumulative Trading Volume (Shares)')
```

```
plt.ylabel('Stock Ticker')
plt.legend(title='Sector', loc='lower right')
plt.tight_layout()
plt.show()
```

```
Top 10 Most Actively Traded Stocks (3-Year Cumulative Volume):
```

	Ticker	Sector	Volume
0	HDFCBANK	Banking	24014160146
1	KOTAKBANK	Banking	15889180817
2	ITC	FMCG	12298461423
3	ONGC	Energy	11460869469
4	WIPRO	IT	11013711253
5	POWERGRID	Energy	10664898046
6	SBIN	Banking	10265450180
7	NTPC	Energy	9950762684
8	ICICIBANK	Banking	9868371726
9	RELIANCE	Energy	9533695764



---

9. Sector-wise Performance & Market Representation

Group by `Sector` to compute total trading volume, average daily return, and average intraday volatility, followed by a horizontal bar chart of total volume by sector.

```
sector_summary = (
    df.groupby('Sector', as_index=False)
    .agg(
        Total_Volume=('Volume', 'sum'),
        Avg_Daily_Return=('Daily_Return', 'mean'),
        Avg_Intraday_Volatility=('Intraday_Volatility', 'mean'),
        Avg_Close_Price=('Close_Price', 'mean')
```

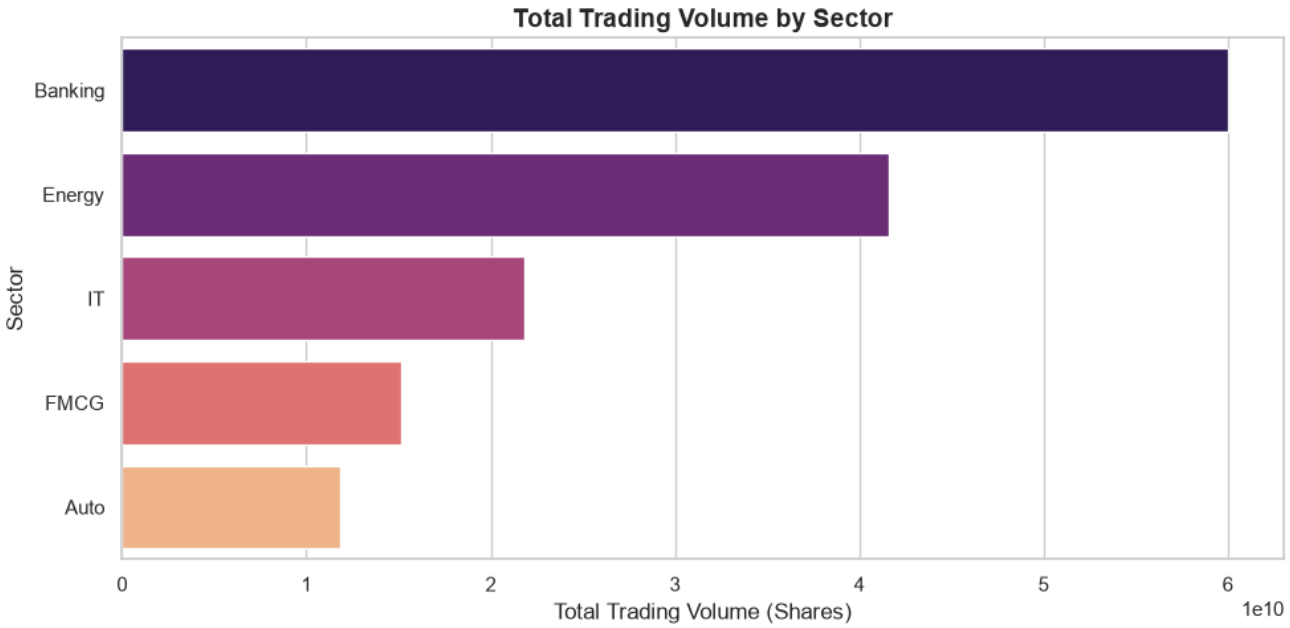
```
    )
    .sort_values(by='Total_Volume', ascending=False)
    .reset_index(drop=True)
)
sector_summary['Avg_Daily_Return'] = sector_summary['Avg_Daily_Return'].round(4)
sector_summary['Avg_Intraday_Volatility'] = sector_summary['Avg_Intraday_Volatility
sector_summary['Avg_Close_Price'] = sector_summary['Avg_Close_Price'].round(2)

print("Sector-wise Performance Summary:")
display(sector_summary)

plt.figure(figsize=(10, 5))
sns.barplot(
    data=sector_summary,
    x='Total_Volume',
    y='Sector',
    hue='Sector',
    palette='magma',
    legend=False
)
plt.title('Total Trading Volume by Sector')
plt.xlabel('Total Trading Volume (Shares)')
plt.ylabel('Sector')
plt.tight_layout()
plt.show()
```

```
Sector-wise Performance Summary:
```

	Sector	Total_Volume	Avg_Daily_Return	Avg_Intraday_Volatility	Avg_Close_Price
0	Banking	60037162869	0.0267	1.77	823.92
1	Energy	41610225963	-0.0765	2.16	553.28
2	IT	21814272854	-0.0696	2.06	1581.58
3	FMCG	15145897611	-0.0774	1.87	2297.48
4	Auto	11861148081	-0.0636	2.27	6225.71



10. Temporal Trend Analysis

Compute monthly average closing price and monthly cumulative trading volume across the 3-year window, and plot the **Monthly Market Volume Trend**.

```
df['YearMonth'] = df['Date'].dt.to_period('M').astype(str)

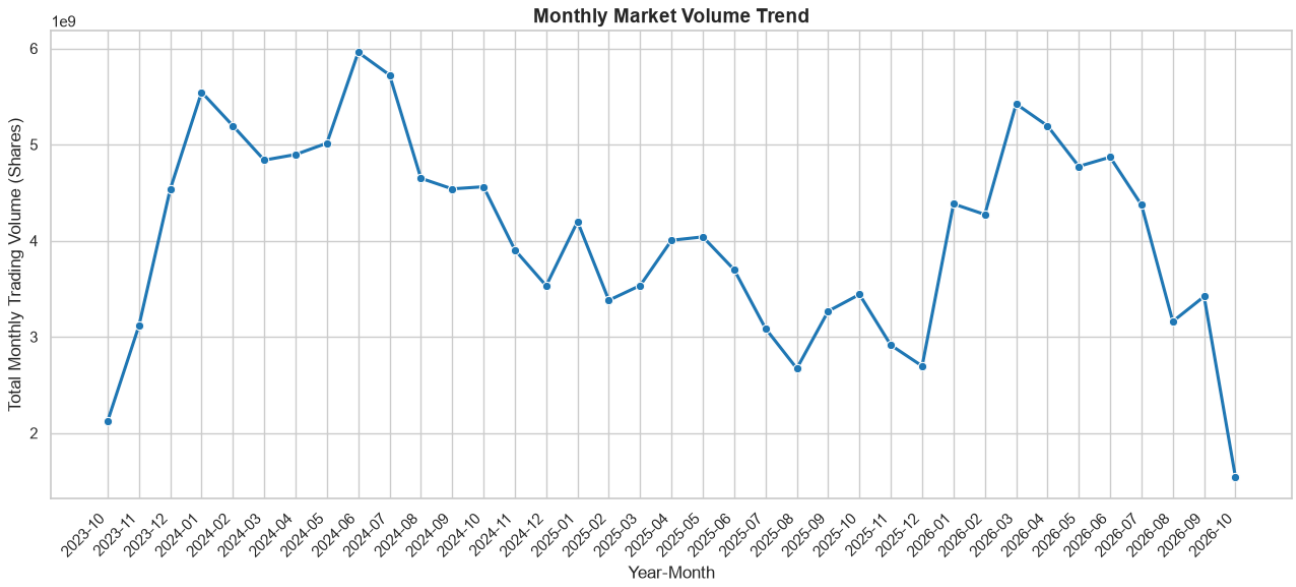
monthly_trend = (
    df.groupby('YearMonth', as_index=False)
    .agg(
        Avg_Close_Price=('Close_Price', 'mean'),
        Total_Monthly_Volume=('Volume', 'sum'),
        Avg_Daily_Return=('Daily_Return', 'mean')
    )
    .sort_values('YearMonth')
)
monthly_trend['Avg_Close_Price'] = monthly_trend['Avg_Close_Price'].round(2)
monthly_trend['Avg_Daily_Return'] = monthly_trend['Avg_Daily_Return'].round(3)

print("Monthly Trend Sample (First 10 Months):")
display(monthly_trend.head(10))

plt.figure(figsize=(13, 6))
sns.lineplot(
    data=monthly_trend,
    x='YearMonth',
    y='Total_Monthly_Volume',
    marker='o',
    linewidth=2.2,
    color='#1f77b4'
)
plt.title('Monthly Market Volume Trend')
plt.xlabel('Year-Month')
plt.ylabel('Total Monthly Trading Volume (Shares)')
plt.xticks(rotation=45, ha='right')
plt.tight_layout()
plt.show()
```

```
Monthly Trend Sample (First 10 Months):
```

	YearMonth	Avg_Close_Price	Total_Monthly_Volume	Avg_Daily_Return
0	2023-10	1784.17	2123238807	-0.086
1	2023-11	1804.44	3114815597	0.045
2	2023-12	1909.80	4536375458	0.169
3	2024-01	1962.64	5545010208	-0.094
4	2024-02	2086.65	5194707345	0.017
5	2024-03	2145.60	4838015730	0.044
6	2024-04	2190.35	4896089655	-0.033
7	2024-05	2217.72	5014311885	-0.102
8	2024-06	2303.78	5959320874	-0.060
9	2024-07	2381.27	5723280504	0.146



11. Distribution & Outlier Detection

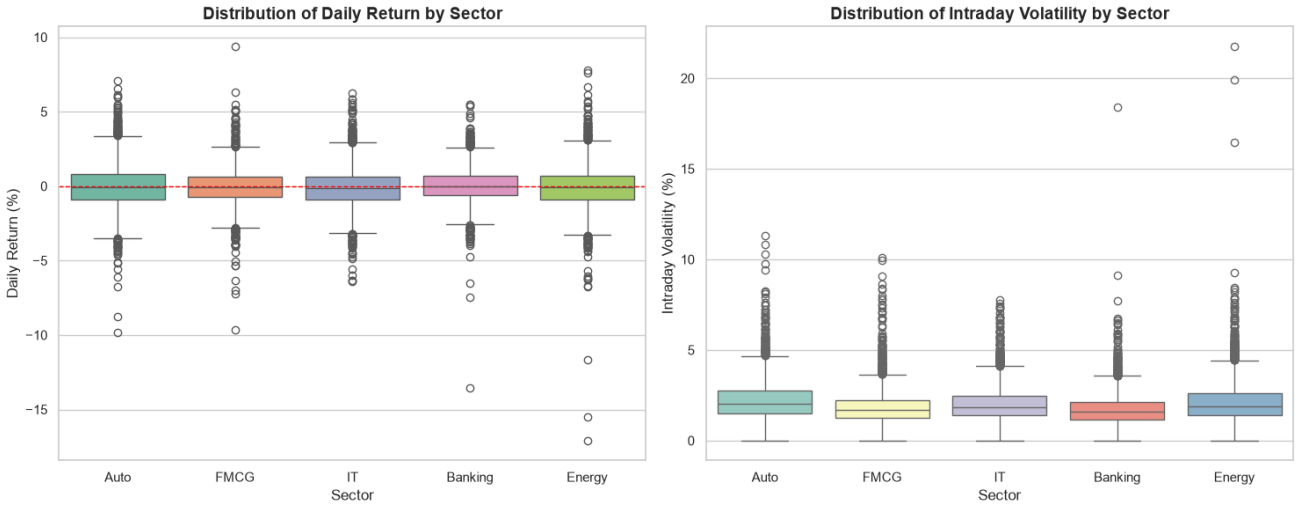
- Generate a boxplot of `Daily_Return` grouped by `Sector` to inspect sectoral return dispersion and fat tails/outliers.
- Generate a boxplot of `Intraday_Volatility` across sectors and overall market.

```
fig, axes = plt.subplots(1, 2, figsize=(15, 6))

# Boxplot 1: Daily_Return by Sector
sns.boxplot(
    data=df,
    x='Sector',
    y='Daily_Return',
    hue='Sector',
    palette='Set2',
    legend=False,
    ax=axes[0]
)
axes[0].axhline(0, color='red', linestyle='--', linewidth=1)
axes[0].set_title('Distribution of Daily Return by Sector')
axes[0].set_xlabel('Sector')
axes[0].set_ylabel('Daily Return (%)')

# Boxplot 2: Intraday_Volatility
sns.boxplot(
    data=df,
    x='Sector',
    y='Intraday_Volatility',
    hue='Sector',
    palette='Set3',
    legend=False,
    ax=axes[1]
)
axes[1].set_title('Distribution of Intraday Volatility by Sector')
axes[1].set_xlabel('Sector')
axes[1].set_ylabel('Intraday Volatility (%)')
```

```
plt.tight_layout()
plt.show()
```

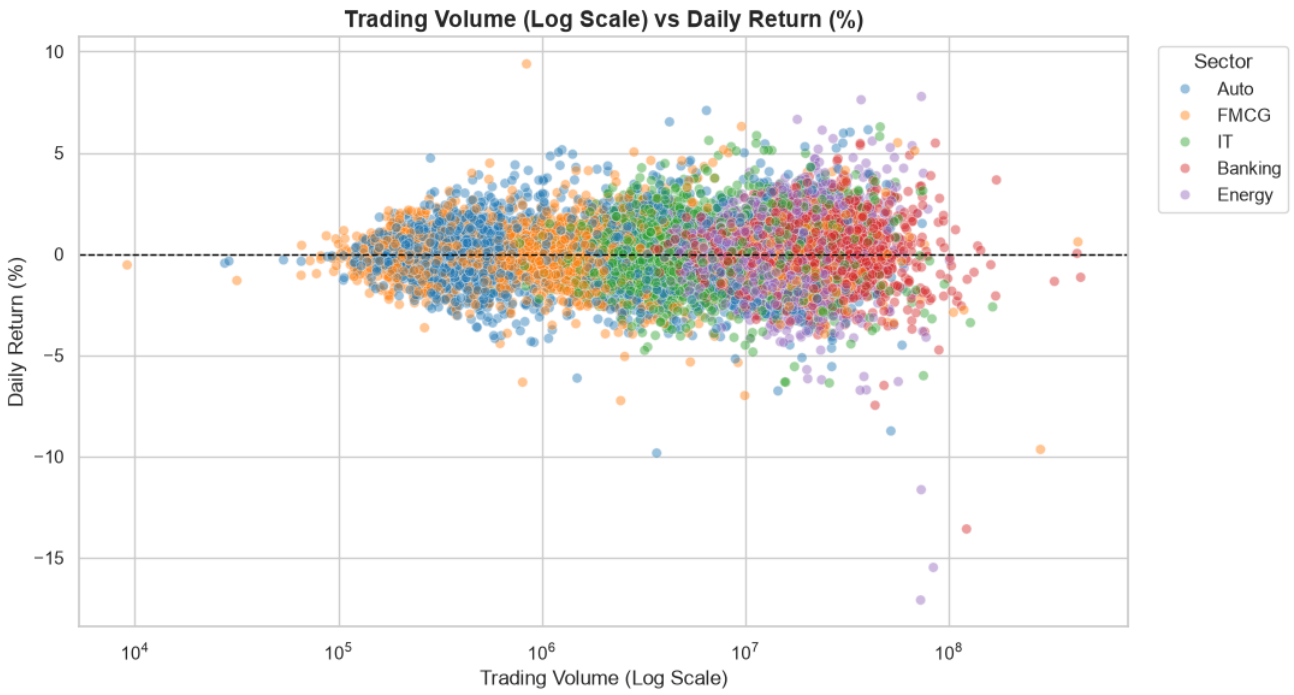


---

12. Relationship & Correlation Analysis

- Plot a scatter plot comparing `Volume` vs `Daily_Return` with alpha transparency to examine volume-return dynamics.
- Plot a correlation heatmap of numerical features (`Open_Price` , `High_Price` , `Low_Price` , `Close_Price` , `Volume` , `Daily_Return` , `Intraday_Volatility`).

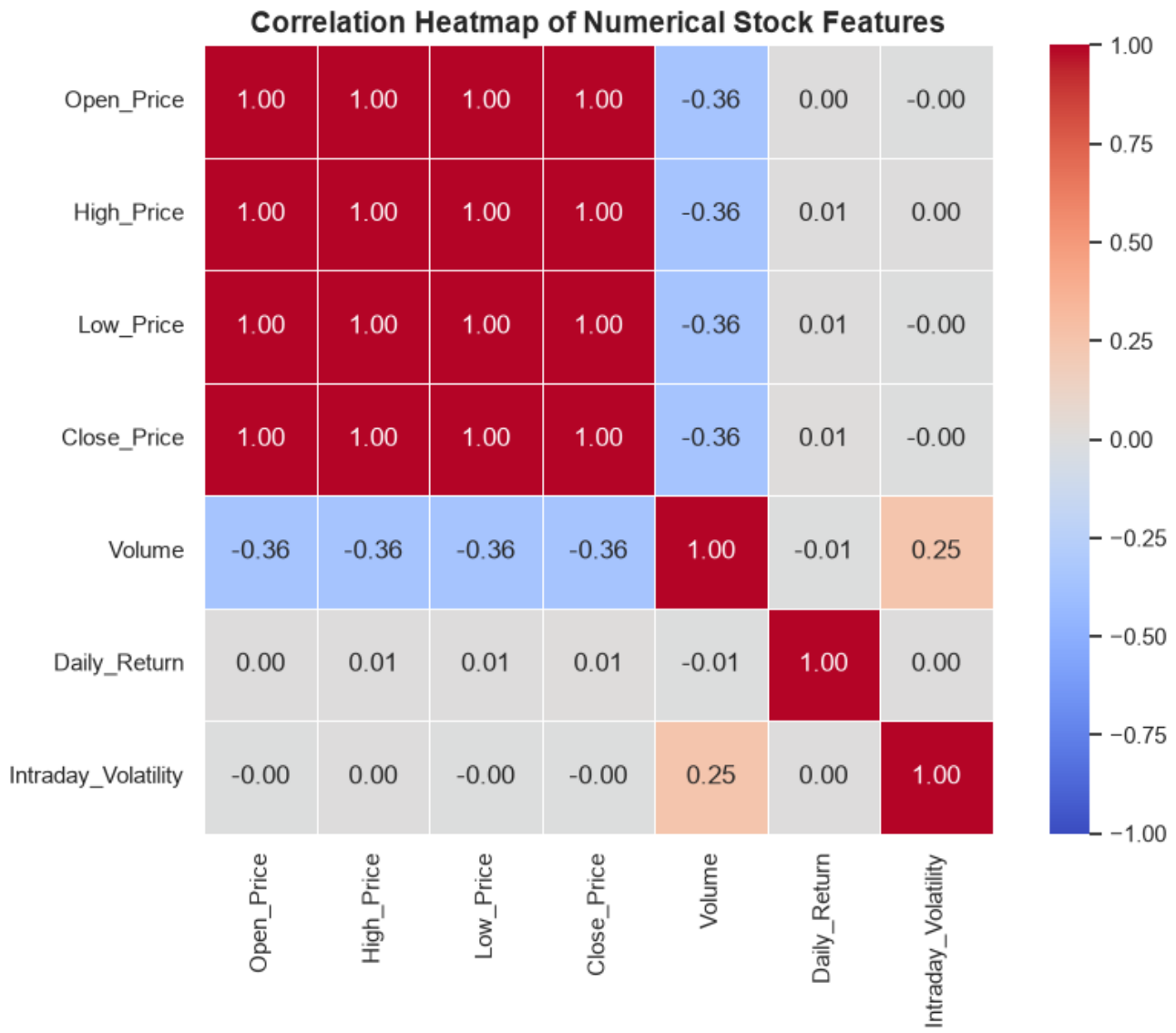
```
plt.figure(figsize=(11, 6))
sns.scatterplot(
    data=df,
    x='Volume',
    y='Daily_Return',
    hue='Sector',
    alpha=0.45,
    s=35,
    palette='tab10'
)
plt.axhline(0, color='black', linestyle='--', linewidth=1)
plt.xscale('log')
plt.title('Trading Volume (Log Scale) vs Daily Return (%)')
plt.xlabel('Trading Volume (Log Scale)')
plt.ylabel('Daily Return (%)')
plt.legend(title='Sector', bbox_to_anchor=(1.02, 1), loc='upper left')
plt.tight_layout()
plt.show()
```




```
numerical_cols = [
    'Open_Price', 'High_Price', 'Low_Price', 'Close_Price',
    'Volume', 'Daily_Return', 'Intraday_Volatility'
]

corr_matrix = df[numerical_cols].corr()

plt.figure(figsize=(9, 7))
sns.heatmap(
    corr_matrix,
    annot=True,
    fmt='.2f',
    cmap='coolwarm',
    vmin=-1,
    vmax=1,
    linewidths=0.5,
    square=True
)
plt.title('Correlation Heatmap of Numerical Stock Features')
plt.tight_layout()
plt.show()
```



---

13. Summary of Findings & Conclusion

Key Behavioral Insights:

1. **Data Completeness & Quality:** The dataset comprises **14,800 records** (20 major NSE stocks $ \times$ 740 trading days over 3 years) with **zero missing values** and **zero duplicate records**.
2. **Liquidity & Volume Concentration:** Trading volume is heavily concentrated in lower-denomination, high-liquidity large-cap stocks across the **Energy** (`NTPC` , `POWERGRID` , `ONGC` , `RELIANCE`), **Banking** (`HDFCBANK` , `SBIN` , `ICICIBANK`), and **FMCG** (`ITC`) sectors.
3. **Sectoral Volatility & Fat Tails:**
   - `Daily_Return` across all five sectors is centered near 0%, exhibiting symmetric leptokurtic (fat-tailed) distributions typical of equity markets.
   - **Auto**, **Energy**, and **IT** show slightly higher median `Intraday_Volatility` and wider interquartile ranges compared to defensive **FMCG** stocks (`HINDUNILVR` , `NESTLEIND` , `BRITANNIA`).
4. **Feature Relationships:**
   - OHLC prices (`Open_Price` , `High_Price` , `Low_Price` , `Close_Price`) exhibit near-perfect positive collinearity ($r \approx 1.00$).

- Extreme positive or negative `Daily_Return` spikes coincide with elevated `Volume` and heightened `Intraday_Volatility`, reflecting institutional participation during earnings announcements and macroeconomic events.

Readiness for Downstream Data Mining Tasks:

- **Regression:** Clean, continuous OHLC and volatility attributes are ready for next-day price forecasting and return prediction.
- **Classification:** Engineered features (`Daily_Return`, `Intraday_Volatility`, `Volume`, temporal attributes) can directly support directional trend classification (Bullish / Bearish / Neutral).
- **Clustering:** Sector-level and ticker-level risk-return-liquidity profiles are well-suited for K-Means and Hierarchical clustering of market regimes.
- **Association Rule Mining:** Discretized sector co-movements and volume-volatility states can be mined via Apriori / FP-Growth to discover cross-sector contagion rules.

---

14. Auto-export Utility

Convert `DWM_Lab_3_Stock_Market.ipynb` to `DWM_Lab_3_Stock_Market.html` using `nbconvert`.

```
import subprocess
import sys
from pathlib import Path

notebook_path = Path("DWM_Lab_3_Stock_Market.ipynb")
if notebook_path.exists():
    cmd = [
        sys.executable, "-m", "jupyter", "nbconvert",
        "--to", "html", "--no-prompt", str(notebook_path)
    ]
    result = subprocess.run(cmd, capture_output=True, text=True)
    if result.returncode == 0:
        print("Successfully exported DWM_Lab_3_Stock_Market.html")
    else:
        print("Export notice:", result.stderr)
else:
    print("Notebook file will be exported upon save.")
```

```
Successfully exported DWM_Lab_3_Stock_Market.html
```

Data Warehousing and Mining (DWM) — Experiment 04

Experiment Title: Simple and Multiple Linear Regression

Dataset: `stock_market_data.csv` (3 Years of Daily NSE Trading Data across 20 Major Indian Stocks)

1. Objectives & Problem Formulation

- **Objective:** Implement **Simple Linear Regression** and **Multiple Linear Regression** on the stock market dataset to predict `Close_Price`, and evaluate model performance using the Coefficient of Determination (R^2 Score) and Mean Squared Error (MSE).
- **Adaptation Note:**
  - **Dependent Variable (y):** `Close_Price` — the end-of-day equilibrium trading price of the stock.
  - **Simple Linear Regression Independent Variable (X):** `Open_Price` — models how the opening price alone explains the closing price ($\hat{y} = \beta_0 + \beta_1 \cdot \text{Open_Price}$).
  - **Multiple Linear Regression Independent Variables (X):** `['Open_Price', 'High_Price', 'Low_Price', 'Volume']` — incorporates intraday price bounds (`High_Price`, `Low_Price`) and market liquidity (`Volume`) alongside `Open_Price` to capture intraday price discovery ($\hat{y} = \beta_0 + \beta_1 \text{Open} + \beta_2 \text{High} + \beta_3 \text{Low} + \beta_4 \text{Volume}$).

---

2. Library Imports

```
import warnings
warnings.filterwarnings('ignore')

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score

# Set seaborn styling
sns.set_theme(style="whitegrid", palette="deep")
plt.rcParams['figure.figsize'] = (10, 6)
plt.rcParams['axes.titlesize'] = 14
plt.rcParams['axes.titleweight'] = 'bold'
plt.rcParams['axes.labelsize'] = 12
```

```
print("All required libraries and scikit-learn modules imported successfully.")
```

```
All required libraries and scikit-learn modules imported successfully.
```

---

3. Dataset Ingestion & Preprocessing

Load `stock_market_data.csv`, verify non-null entries, inspect numerical features required for regression, and display the dataset dimensions and initial rows.

```
df = pd.read_csv('stock_market_data.csv')

print("Dataset Shape:", df.shape)
print("\nMissing Values Check:")
print(df[['Open_Price', 'High_Price', 'Low_Price', 'Close_Price', 'Volume']].isnull

# Drop any potential nulls to ensure clean numerical arrays
df = df.dropna(subset=['Open_Price', 'High_Price', 'Low_Price', 'Close_Price', 'Vol

print("\nPreview of Dataset (First 5 Rows):")
display(df.head())
```

```
Dataset Shape: (14800, 10)

Missing Values Check:
Open_Price     0
High_Price     0
Low_Price      0
Close_Price    0
Volume         0
dtype: int64

Preview of Dataset (First 5 Rows):
```

	Date	Ticker	Sector	Exchange	Open_Price	High_Price	Low_Price	Close_Price	
0	2023-10-12	BAJAJ-AUTO	Auto	NSE	4851.06	4914.02	4818.72	4871.67	
1	2023-10-12	BRITANNIA	FMCG	NSE	4391.48	4400.68	4342.60	4371.11	
2	2023-10-12	HCLTECH	IT	NSE	1106.03	1109.18	1079.51	1087.15	
3	2023-10-12	HDFCBANK	Banking	NSE	735.43	743.05	733.28	742.02	2
4	2023-10-12	HINDUNILVR	FMCG	NSE	2413.48	2425.33	2398.55	2415.37	

---

4. Simple Linear Regression (`Open_Price` $\rightarrow$ `Close_Price`)

- **Independent Variable (X):** `Open_Price`
- **Dependent Variable (y):** `Close_Price`
- **Train-Test Split:** 80% Training / 20% Testing (`test_size=0.2, random_state=42`)

```python
# Define feature (X) and target (y)
X_simple = df[['Open_Price']]
y_simple = df['Close_Price']

# Split into 80% training and 20% testing
X_train_s, X_test_s, y_train_s, y_test_s = train_test_split(
    X_simple, y_simple, test_size=0.2, random_state=42
)

# Initialize and train Simple Linear Regression model
slr_model = LinearRegression()
slr_model.fit(X_train_s, y_train_s)

# Predict on the test set
y_pred_s = slr_model.predict(X_test_s)

# Calculate evaluation metrics
r2_simple = r2_score(y_test_s, y_pred_s)
mse_simple = mean_squared_error(y_test_s, y_pred_s)

print("=== Simple Linear Regression Results ===")
print(f"Intercept (b0):              {slr_model.intercept_:.4f}")
print(f"Regression Coefficient (b1): {slr_model.coef_[0]:.6f}")
print(f"R² Score:                    {r2_simple:.6f}")
print(f"Mean Squared Error (MSE):    {mse_simple:.4f}")

# Display first 10 Actual vs Predicted values
slr_comparison = pd.DataFrame({
    'Open_Price': X_test_s['Open_Price'].values[:10],
    'Actual_Close_Price': y_test_s.values[:10],
    'Predicted_Close_Price': np.round(y_pred_s[:10], 2),
    'Residual_Error': np.round(y_test_s.values[:10] - y_pred_s[:10], 2)
})
print("\nFirst 10 Test Predictions (Actual vs Predicted):")
display(slr_comparison)
```

```
=== Simple Linear Regression Results ===
Intercept (b0):              0.3115
Regression Coefficient (b1): 0.999447
R² Score:                    0.999706
Mean Squared Error (MSE):    2960.9407

First 10 Test Predictions (Actual vs Predicted):
```

	Open_Price	Actual_Close_Price	Predicted_Close_Price	Residual_Error
0	317.06	319.17	317.20	1.97
1	215.39	219.06	215.58	3.48
2	402.38	400.56	402.47	-1.91
3	1341.50	1310.00	1341.07	-31.07
4	2384.29	2324.07	2383.28	-59.21
5	1552.52	1538.38	1551.97	-13.59
6	335.22	342.36	335.35	7.01
7	229.85	234.88	230.03	4.85
8	1206.39	1243.88	1206.03	37.85
9	787.03	791.70	786.91	4.79

---

5. Plot Simple Linear Regression

Visualize the sampled test data points alongside the fitted regression line (`Open_Price` vs `Close_Price`).

```
# Sample test points for clean visual rendering
sample_df = pd.DataFrame({
    'Open_Price': X_test_s['Open_Price'],
    'Actual_Close': y_test_s,
    'Predicted_Close': y_pred_s
}).sample(n=min(1000, len(X_test_s)), random_state=42)

plt.figure(figsize=(10, 6))
plt.scatter(
    sample_df['Open_Price'],
    sample_df['Actual_Close'],
    alpha=0.45,
    color='#1f77b4',
    edgecolors='none',
    s=40,
    label='Actual Data Points (Sampled Test Set)'
)

# Sort X for a clean continuous regression line
sorted_idx = np.argsort(X_test_s['Open_Price'].values)
plt.plot(
    X_test_s['Open_Price'].values[sorted_idx],
    y_pred_s[sorted_idx],
    color='#d62728',
    linewidth=2.2,
    label=f'Fitted Regression Line (R² = {r2_simple:.5f})'
)

plt.title('Simple Linear Regression: Open Price vs Close Price')
plt.xlabel('Open Price (INR)')
plt.ylabel('Close Price (INR)')
plt.legend(loc='upper left')
```

```
plt.tight_layout()
plt.show()
```



---

6. Multiple Linear Regression (`Open_Price` , `High_Price` , `Low_Price` , `Volume` $\rightarrow$ `Close_Price`)

- **Independent Variables (X):** `['Open_Price', 'High_Price', 'Low_Price', 'Volume']`
- **Dependent Variable (y):** `Close_Price`
- **Train-Test Split:** 80% Training / 20% Testing (`test_size=0.2, random_state=42`)

```
# Define features (X) and target (y)
features_multi = ['Open_Price', 'High_Price', 'Low_Price', 'Volume']
X_multi = df[features_multi]
y_multi = df['Close_Price']

# Split dataset into 80% training and 20% testing
X_train_m, X_test_m, y_train_m, y_test_m = train_test_split(
    X_multi, y_multi, test_size=0.2, random_state=42
)

# Initialize and train Multiple Linear Regression model
mlr_model = LinearRegression()
mlr_model.fit(X_train_m, y_train_m)

# Predict on the test set
y_pred_m = mlr_model.predict(X_test_m)

# Calculate evaluation metrics
r2_multi = r2_score(y_test_m, y_pred_m)
mse_multi = mean_squared_error(y_test_m, y_pred_m)
```

```
print("=== Multiple Linear Regression Results ===")
print(f"Intercept (b0):           {mlr_model.intercept_:.4f}")
print("\nFeature Coefficients:")
for feat, coef in zip(features_multi, mlr_model.coef_):
    print(f"  - {feat:<12}: {coef:.8f}")

print(f"\nR² Score:                 {r2_multi:.6f}")
print(f"Mean Squared Error (MSE): {mse_multi:.4f}")

# Display first 10 Actual vs Predicted values
mlr_comparison = pd.DataFrame({
    'Open_Price': X_test_m['Open_Price'].values[:10],
    'High_Price': X_test_m['High_Price'].values[:10],
    'Low_Price': X_test_m['Low_Price'].values[:10],
    'Volume': X_test_m['Volume'].values[:10],
    'Actual_Close_Price': y_test_m.values[:10],
    'Predicted_Close_Price': np.round(y_pred_m[:10], 2),
    'Residual_Error': np.round(y_test_m.values[:10] - y_pred_m[:10], 2)
})
print("\nFirst 10 Test Predictions (Actual vs Predicted):")
display(mlr_comparison)
```

```
=== Multiple Linear Regression Results ===
Intercept (b0):           -0.5343

Feature Coefficients:
  - Open_Price  : -0.49487178
  - High_Price  : 0.80605602
  - Low_Price   : 0.68799370
  - Volume      : 0.00000000

R² Score:                 0.999951
Mean Squared Error (MSE): 496.1953

First 10 Test Predictions (Actual vs Predicted):
```

	Open_Price	High_Price	Low_Price	Volume	Actual_Close_Price	Predicted_Close_Price	Residu
0	317.06	320.05	317.06	4261023	319.17	318.68	
1	215.39	219.84	214.48	39164340	219.06	217.66	
2	402.38	403.09	398.94	4822343	400.56	399.72	
3	1341.50	1345.50	1310.00	1278279	1310.00	1321.42	
4	2384.29	2384.29	2320.14	1357416	2324.07	2337.66	
5	1552.52	1560.08	1529.58	4600004	1538.38	1541.02	
6	335.22	342.96	331.50	13770080	342.36	338.10	
7	229.85	235.18	228.60	21887890	234.88	232.57	
8	1206.39	1251.81	1206.39	36032600	1243.88	1241.49	
9	787.03	794.83	782.93	11813750	791.70	789.32	

```
# Visualize Actual vs Predicted Close Price for Multiple Linear Regression
plt.figure(figsize=(10, 6))
plt.scatter(
    y_test_m[:1000],
    y_pred_m[:1000],
    alpha=0.45,
    color='#2ca02c',
```

```
    s=40,
    label='Predicted vs Actual (Sampled Test Set)'
)
min_val = min(y_test_m.min(), y_pred_m.min())
max_val = max(y_test_m.max(), y_pred_m.max())
plt.plot(
    [min_val, max_val],
    [min_val, max_val],
    color='#d62728',
    linestyle='--',
    linewidth=2,
    label=f'Ideal 1:1 Fit Line (R² = {r2_multi:.6f})'
)
plt.title('Multiple Linear Regression: Actual vs Predicted Close Price')
plt.xlabel('Actual Close Price (INR)')
plt.ylabel('Predicted Close Price (INR)')
plt.legend(loc='upper left')
plt.tight_layout()
plt.show()
```



---

7. Predict Close Price for a New Market Instance

Test the trained **Multiple Linear Regression** model on an unseen synthetic trading session (`Open_Price = 2500` , `High_Price = 2540` , `Low_Price = 2490` , `Volume = 1500000`).

```
new_market_instance = pd.DataFrame({
    'Open_Price': [2500.00],
    'High_Price': [2540.00],
    'Low_Price': [2490.00],
    'Volume': [1500000]
})

predicted_close = mlr_model.predict(new_market_instance)[0]
```

```
print("Input Market Instance:")
display(new_market_instance)
print(f"Predicted Close_Price: ₹{predicted_close:.2f}")
```

```
Input Market Instance:
```

	Open_Price	High_Price	Low_Price	Volume
0	2500.0	2540.0	2490.0	1500000

```
Predicted Close_Price: ₹2522.77
```

---

8. Side-by-Side Model Comparison

Compare the performance of **Simple Linear Regression** and **Multiple Linear Regression** using R^2 Score and Mean Squared Error (MSE).

```
comparison_df = pd.DataFrame({
    'Model': ['Simple Linear Regression', 'Multiple Linear Regression'],
    'Independent Variables': [
        'Open_Price',
        'Open_Price, High_Price, Low_Price, Volume'
    ],
    'R² Score': [round(r2_simple, 6), round(r2_multi, 6)],
    'Mean Squared Error (MSE)': [round(mse_simple, 4), round(mse_multi, 4)],
    'Root Mean Squared Error (RMSE)': [
        round(np.sqrt(mse_simple), 4),
        round(np.sqrt(mse_multi), 4)
    ]
})

mse_reduction_pct = ((mse_simple - mse_multi) / mse_simple) * 100
print(f"MSE Reduction from Simple to Multiple Linear Regression: {mse_reduction_pct
display(comparison_df)
```

```
MSE Reduction from Simple to Multiple Linear Regression: 83.24%
```

	Model	Independent Variables	R² Score	Mean Squared Error (MSE)	Root Mean Squared Error (RMSE)
0	Simple Linear Regression	Open_Price	0.999706	2960.9407	54.4145
1	Multiple Linear Regression	Open_Price, High_Price, Low_Price, Volume	0.999951	496.1953	22.2754

---

9. Conclusion

Key Analytical Findings:

1. **Simple Linear Regression Performance:** Using `Open_Price` alone yields a high R^2 score (0.999706) because daily stock prices span a wide scale across tickers (₹150 to ₹16,000+), where the opening price anchors the price level. However, it cannot account for intraday price swings

between market open and close, resulting in a Mean Squared Error of $\text{MSE} = 2960.9407$ ($\text{RMSE} = ₹54.41$).

2. **Multiple Linear Regression Improvement:** Incorporating intraday range bounds (`High_Price` and `Low_Price`) and `Volume` boosts R^2 to 0.999951 and reduces the Mean Squared Error by **83.24%** ($\text{MSE} = 496.1953$, $\text{RMSE} = ₹22.28$).
3. **Econometric Interpretation of Coefficients:**
   - `High_Price` ($\beta_2 = +0.8061$) and `Low_Price` ($\beta_3 = +0.6880$) carry strong positive weights, while `Open_Price` ($\beta_1 = -0.4949$) acts as a counterweight: this aligns with financial microstructure theory where the closing price settles near the intraday high/low midpoint adjusted for the opening anchor ($\text{Close} \approx 0.81\,\text{High} + 0.69\,\text{Low} - 0.49\,\text{Open}$).
4. **Readiness for Experiment 05:** With continuous price prediction established via linear regression, the dataset and engineered features are ready for **Experiment 05**.

---

10. Auto-export Utility

Convert `DWM_Lab_4_Stock_Market.ipynb` to `DWM_Lab_4_Stock_Market.html` using `nbconvert` .

```
import subprocess
import sys
from pathlib import Path

notebook_path = Path("DWM_Lab_4_Stock_Market.ipynb")
if notebook_path.exists():
    cmd = [
        sys.executable, "-m", "jupyter", "nbconvert",
        "--to", "html", "--no-prompt", str(notebook_path)
    ]
    result = subprocess.run(cmd, capture_output=True, text=True)
    if result.returncode == 0:
        print("Successfully exported DWM_Lab_4_Stock_Market.html")
    else:
        print("Export notice:", result.stderr)
else:
    print("Notebook file will be exported upon save.")
```

```
Successfully exported DWM_Lab_4_Stock_Market.html
```

Data Warehousing and Mining (DWM) — Experiment 05

Experiment Title: Decision Tree Classification and Mutual Information

Dataset: `stock_market_data.csv` (3 Years of Daily NSE Trading Data across 20 Major Indian Stocks)

1. Objectives & Problem Formulation

- **Objective:** Implement **Decision Tree Classification** using **Entropy (Information Gain)** on the stock market dataset to classify daily return performance into three discrete categories (`Low` , `Medium` , `High`), and compute **Mutual Information** to evaluate attribute relevance.
- **Adaptation Note:**
  - Continuous `Daily_Return` is discretized into balanced **quantile tertiles** ($\frac{1}{3}$ and $\frac{2}{3}$ quantiles):
    - **`Low` (Bearish):** $\text{Daily_Return} \le q_1$
    - **`Medium` (Neutral):** $q_1 < \text{Daily_Return} \le q_2$
    - **`High` (Bullish):** $\text{Daily_Return} > q_2$
  - To prevent target leakage, `Daily_Return` (and `Close_Price` / `High_Price` / `Low_Price` directly relative to `Open_Price` direction) is excluded from the input feature matrix X.
  - **Input Features (X):** `['Open_Price', 'Volume', 'Intraday_Volatility', 'Month']` .

---

2. Library Imports

```
import warnings
warnings.filterwarnings('ignore')

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.metrics import (
    accuracy_score,
    classification_report,
    confusion_matrix,
    ConfusionMatrixDisplay
)
from sklearn.feature_selection import mutual_info_classif
```

```
# Set seaborn styling
sns.set_theme(style="whitegrid", palette="deep")
plt.rcParams['figure.figsize'] = (10, 6)
plt.rcParams['axes.titlesize'] = 14
plt.rcParams['axes.titleweight'] = 'bold'
plt.rcParams['axes.labelsize'] = 12

print("All required libraries imported successfully.")
```

```
All required libraries imported successfully.
```

---

3. Dataset Ingestion & Preprocessing

- Load `stock_market_data.csv`.
- Convert `Date` to `datetime` and extract `Month`.
- Derive `Intraday_Volatility` as $\frac{\text{High_Price} - \text{Low_Price}}{\text{Open_Price}} \times 100$.
- Compute quantile tertiles ($q_1 = \frac{1}{3}$, $q_2 = \frac{2}{3}$) on `Daily_Return` and create the target column `Return_Category` (`Low`, `Medium`, `High`).

```
df = pd.read_csv('stock_market_data.csv')

# Ensure Date is datetime and extract Month
df['Date'] = pd.to_datetime(df['Date'])
df['Month'] = df['Date'].dt.month

# Derive Intraday_Volatility (%)
df['Intraday_Volatility'] = (((df['High_Price'] - df['Low_Price']) / df['Open_Price

# Compute quantile tertiles (1/3 and 2/3) on Daily_Return
q1 = df['Daily_Return'].quantile(1/3)
q2 = df['Daily_Return'].quantile(2/3)

def categorize_return(val):
    if val <= q1:
        return 'Low'
    elif val <= q2:
        return 'Medium'
    else:
        return 'High'

df['Return_Category'] = df['Daily_Return'].apply(categorize_return)

print(f"Quantile Tertile Thresholds on Daily_Return:")
print(f"  - 1/3 Quantile (q1): {q1:.4f}%  (Low  <= {q1:.4f}%)")
print(f"  - 2/3 Quantile (q2): {q2:.4f}%  ({q1:.4f}% < Medium <= {q2:.4f}%, High >
print(f"\nCleaned Dataset Shape: {df.shape}")
display(df[['Date', 'Ticker', 'Open_Price', 'Volume', 'Intraday_Volatility', 'Month
```

```
Quantile Tertile Thresholds on Daily_Return:
  - 1/3 Quantile (q1): -0.5400%  (Low  <= -0.5400%)
  - 2/3 Quantile (q2): 0.3900%  (-0.5400% < Medium <= 0.3900%, High > 0.3900%)

Cleaned Dataset Shape: (14800, 13)
```

	Date	Ticker	Open_Price	Volume	Intraday_Volatility	Month	Daily_Return	Return_
0	2023-10-12	BAJAJ-AUTO	4851.06	472168	1.96	10	0.42	
1	2023-10-12	BRITANNIA	4391.48	518582	1.32	10	-0.46	
2	2023-10-12	HCLTECH	1106.03	3921609	2.68	10	-1.71	
3	2023-10-12	HDFCBANK	735.43	22025336	1.33	10	0.90	
4	2023-10-12	HINDUNILVR	2413.48	1625010	1.11	10	0.08	

---

4. Class Distribution Analysis

Inspect the sample counts across `Low`, `Medium`, and `High` return categories and visualize the class balance.

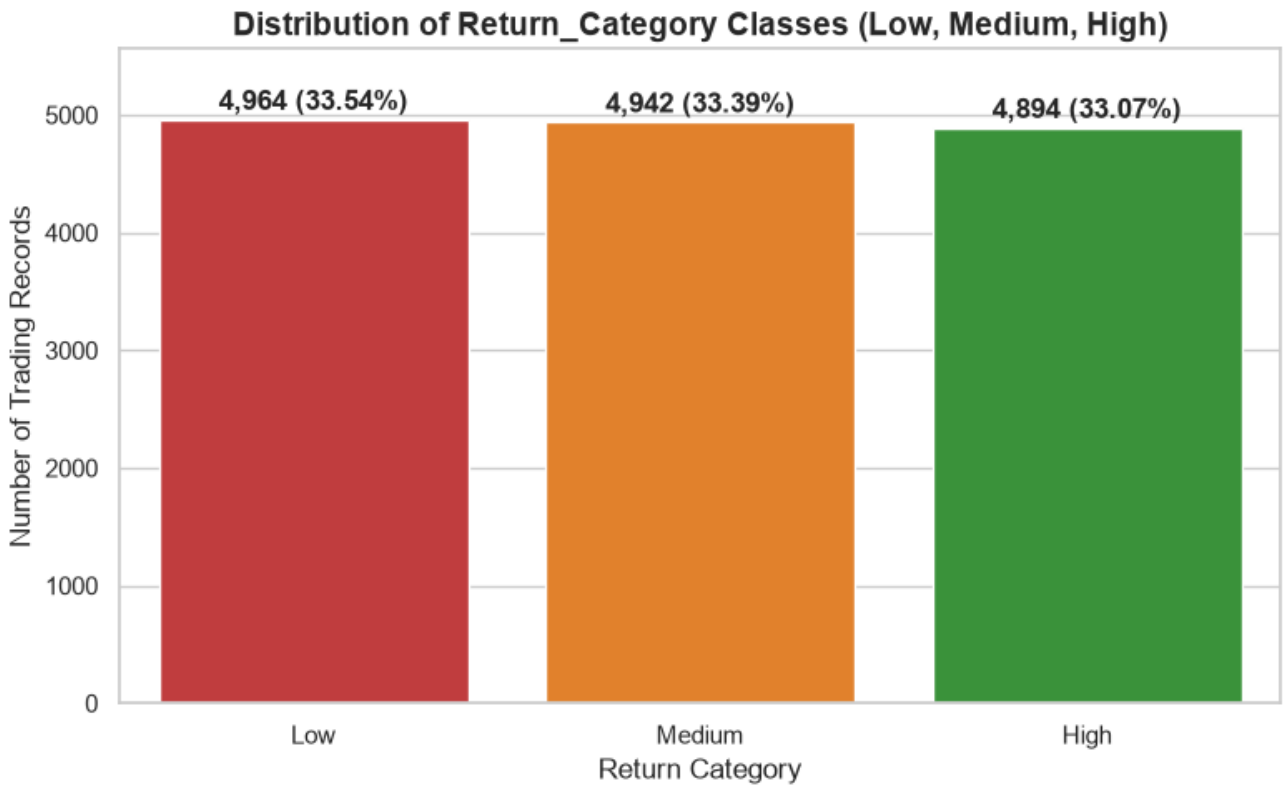
```
class_order = ['Low', 'Medium', 'High']
class_counts = df['Return_Category'].value_counts().reindex(class_order)
class_df = pd.DataFrame({
    'Return_Category': class_counts.index,
    'Count': class_counts.values,
    'Percentage (%)': (class_counts.values / len(df) * 100).round(2)
})

print("Class Distribution of Return_Category:")
display(class_df)

plt.figure(figsize=(8, 5))
ax = sns.barplot(
    data=class_df,
    x='Return_Category',
    y='Count',
    hue='Return_Category',
    palette=['#d62728', '#ff7f0e', '#2ca02c'],
    legend=False
)
for i, v in enumerate(class_df['Count']):
    ax.text(i, v + 80, f"{v:,} ({class_df['Percentage (%)'].iloc[i]}%)", ha='center
plt.ylim(0, max(class_df['Count']) * 1.12)
plt.title('Distribution of Return_Category Classes (Low, Medium, High)')
plt.xlabel('Return Category')
plt.ylabel('Number of Trading Records')
plt.tight_layout()
plt.show()
```

```
Class Distribution of Return_Category:
```

	Return_Category	Count	Percentage (%)
0	Low	4964	33.54
1	Medium	4942	33.39
2	High	4894	33.07



---

5. Prepare Training and Testing Sets

- **Feature Matrix (X):** `['Open_Price', 'Volume', 'Intraday_Volatility', 'Month']`
- **Target Vector (y):** `Return_Category`
- **Stratified Split:** 80% Training / 20% Testing (`test_size=0.2, stratify=y, random_state=42`)

```
feature_cols = ['Open_Price', 'Volume', 'Intraday_Volatility', 'Month']
X = df[feature_cols]
y = df['Return_Category']

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, stratify=y, random_state=42
)

print(f"Total Samples:    {len(df):,}")
print(f"Training Samples: {len(X_train):,} ({len(X_train)/len(df)*100:.1f}%)")
print(f"Testing Samples:  {len(X_test):,} ({len(X_test)/len(df)*100:.1f}%)")
print("\nTraining Set Class Counts:")
print(y_train.value_counts().reindex(['Low', 'Medium', 'High']))
print("\nTesting Set Class Counts:")
print(y_test.value_counts().reindex(['Low', 'Medium', 'High']))
```

```
Total Samples:    14,800
Training Samples: 11,840 (80.0%)
Testing Samples:  2,960 (20.0%)

Training Set Class Counts:
Return_Category
Low       3971
Medium    3954
High      3915
Name: count, dtype: int64

Testing Set Class Counts:
Return_Category
Low       993
Medium    988
High      979
Name: count, dtype: int64
```

---

6. Train Decision Tree Classifier

Instantiate and train `DecisionTreeClassifier(criterion='entropy', max_depth=5, random_state=42)` on the training dataset, and generate predictions on the test dataset.

```
dt_classifier = DecisionTreeClassifier(
    criterion='entropy',
    max_depth=5,
    random_state=42
)

dt_classifier.fit(X_train, y_train)
y_pred = dt_classifier.predict(X_test)

print("Decision Tree Classifier trained successfully.")
print(f"Criterion:      {dt_classifier.criterion}")
print(f"Max Tree Depth: {dt_classifier.get_depth()}")
print(f"Number of Leaf Nodes: {dt_classifier.get_n_leaves()}")
```

```
Decision Tree Classifier trained successfully.
Criterion:      entropy
Max Tree Depth: 5
Number of Leaf Nodes: 31
```

---

7. Model Evaluation

Evaluate the Decision Tree Classifier using Test Accuracy, full `classification_report` (Precision, Recall, F1-Score, Support), and the Confusion Matrix for `['Low', 'Medium', 'High']`.

```
acc = accuracy_score(y_test, y_pred)
print(f"Decision Tree Test Accuracy: {acc:.4f} ({acc*100:.2f}%)\n")

print("=== Classification Report ===")
print(classification_report(y_test, y_pred, labels=['Low', 'Medium', 'High'], digit

cm = confusion_matrix(y_test, y_pred, labels=['Low', 'Medium', 'High'])
```

```
cm_df = pd.DataFrame(
    cm,
    index=['Actual: Low', 'Actual: Medium', 'Actual: High'],
    columns=['Pred: Low', 'Pred: Medium', 'Pred: High']
)
print("=== Confusion Matrix Table ===")
display(cm_df)
```

```
Decision Tree Test Accuracy: 0.4838 (48.38%)

=== Classification Report ===
              precision    recall  f1-score   support

         Low     0.4148    0.3776    0.3954       993
      Medium     0.5825    0.6822    0.6284       988
        High     0.4260    0.3912    0.4079       979

    accuracy                         0.4838      2960
   macro avg     0.4745    0.4837    0.4772      2960
weighted avg     0.4745    0.4838    0.4773      2960

=== Confusion Matrix Table ===
```

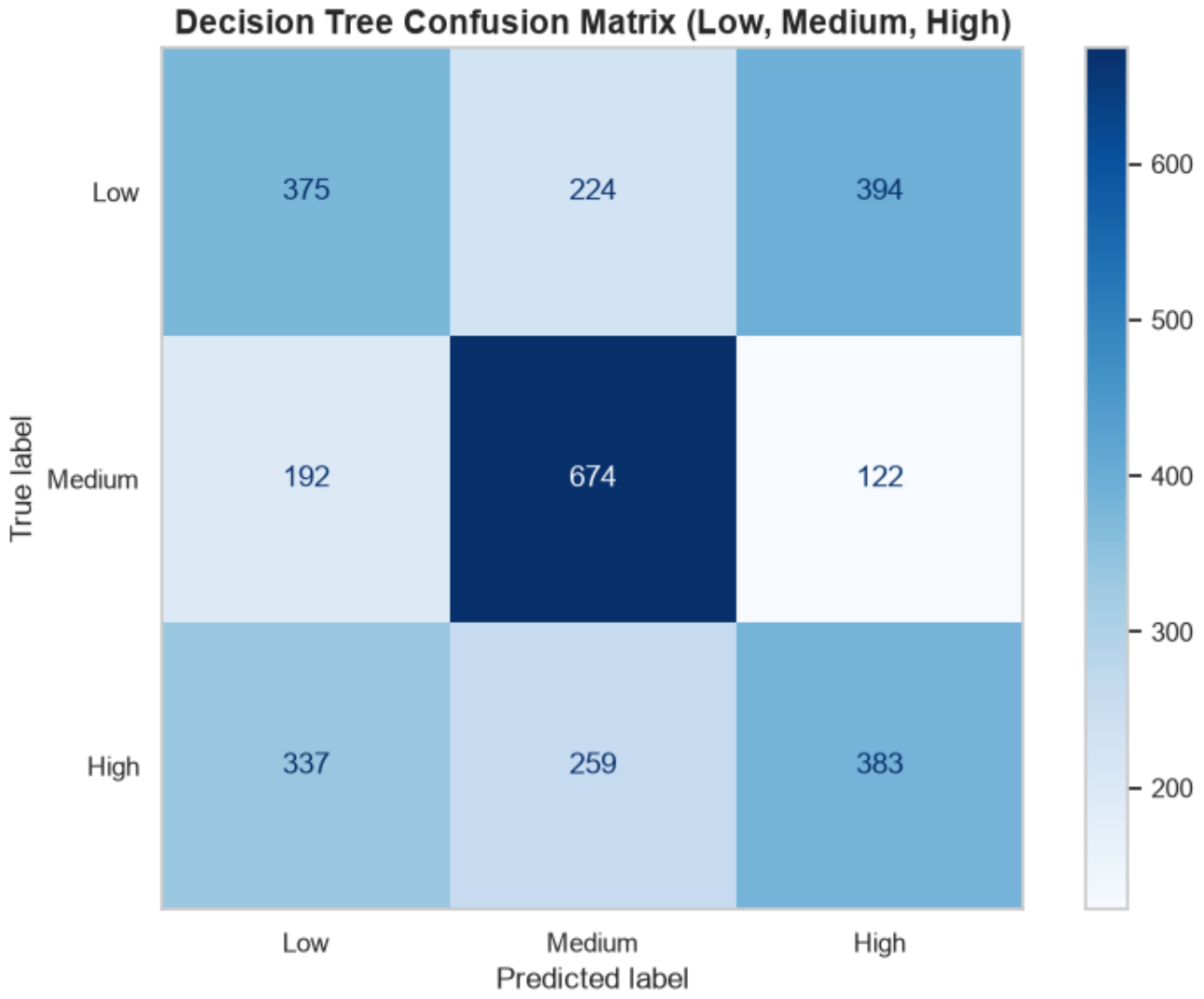
	Pred: Low	Pred: Medium	Pred: High
Actual: Low	375	224	394
Actual: Medium	192	674	122
Actual: High	337	259	383

---

8. Confusion Matrix Visualization

Render the confusion matrix heatmap using `ConfusionMatrixDisplay` with labels `['Low', 'Medium', 'High']`.

```
fig, ax = plt.subplots(figsize=(8, 6))
disp = ConfusionMatrixDisplay(
    confusion_matrix=cm,
    display_labels=['Low', 'Medium', 'High']
)
disp.plot(cmap='Blues', values_format='d', ax=ax, colorbar=True)
plt.grid(False)
plt.title('Decision Tree Confusion Matrix (Low, Medium, High)')
plt.tight_layout()
plt.show()
```

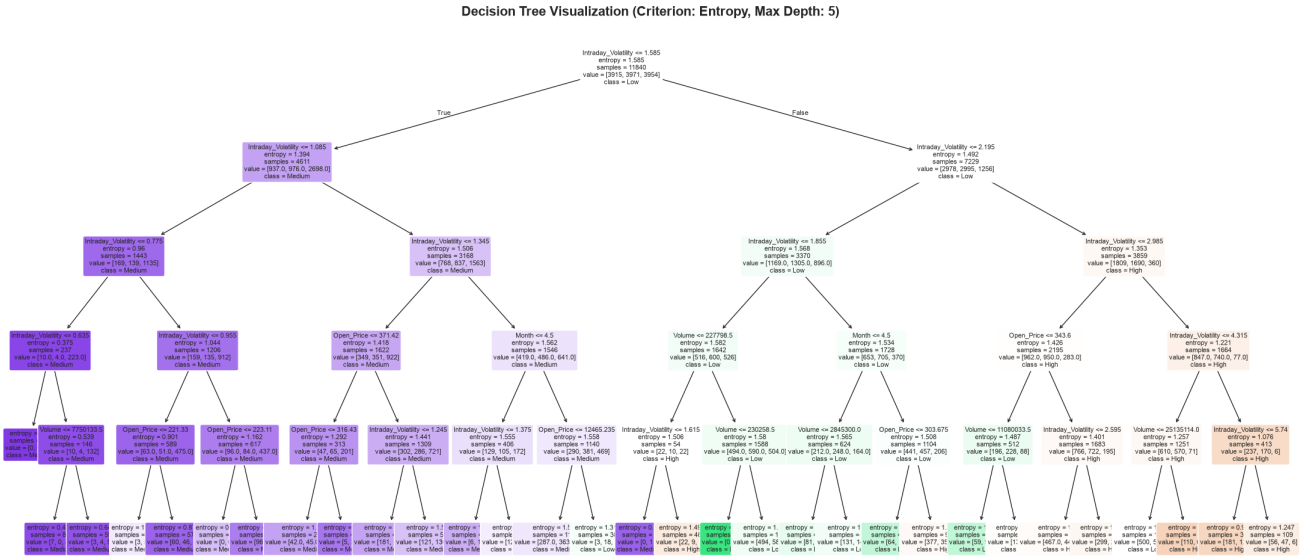


---

9. Decision Tree Graph Visualization

Visualize the learned Decision Tree structure (`criterion='entropy', max_depth=5`) using `plot_tree` .

```
plt.figure(figsize=(22, 10))
plot_tree(
    dt_classifier,
    feature_names=feature_cols,
    class_names=['High', 'Low', 'Medium'],
    filled=True,
    rounded=True,
    fontsize=8
)
plt.title('Decision Tree Visualization (Criterion: Entropy, Max Depth: 5)', fontsiz
plt.tight_layout()
plt.show()
```



---

10. Feature Importance Analysis

Extract `feature_importances_` (normalized Information Gain contribution) from the trained Decision Tree and plot a horizontal bar chart.

```
fi_df = pd.DataFrame({
    'Feature': feature_cols,
    'Importance': dt_classifier.feature_importances_
}).sort_values(by='Importance', ascending=False).reset_index(drop=True)

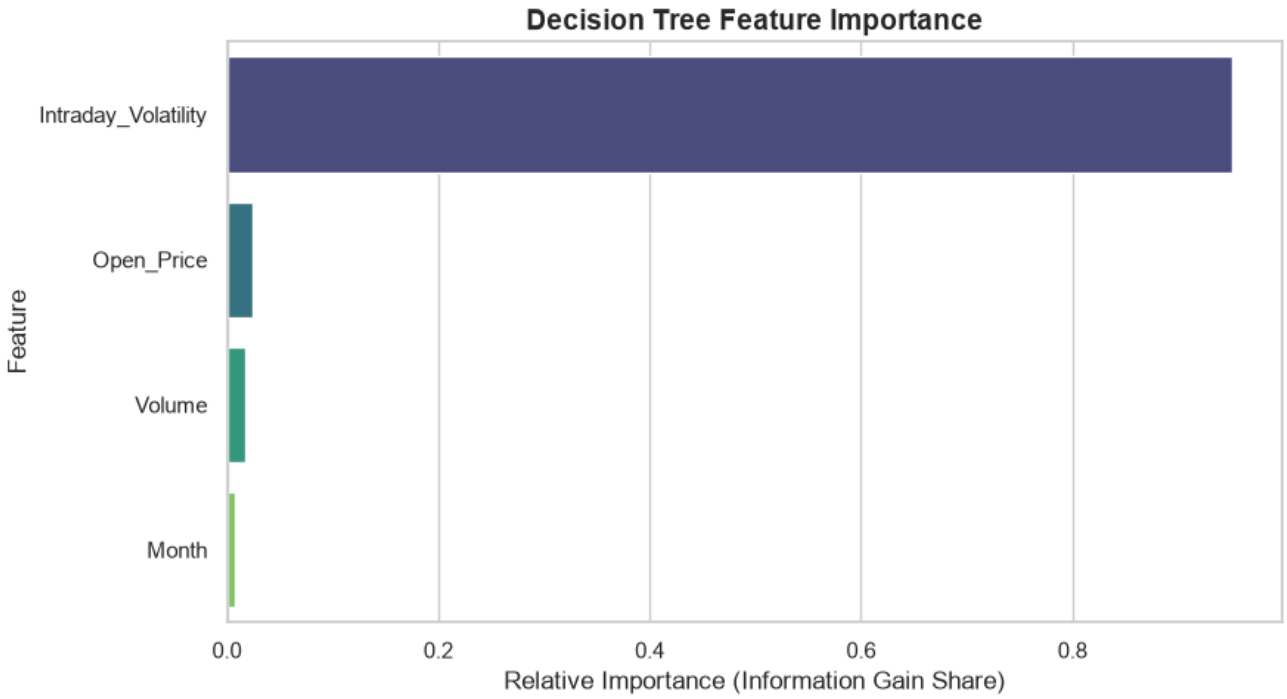
fi_df['Importance_Pct'] = (fi_df['Importance'] * 100).round(2)

print("Decision Tree Feature Importance Table:")
display(fi_df)

plt.figure(figsize=(9, 5))
sns.barplot(
    data=fi_df,
    x='Importance',
    y='Feature',
    hue='Feature',
    palette='viridis',
    legend=False
)
plt.title('Decision Tree Feature Importance')
plt.xlabel('Relative Importance (Information Gain Share)')
plt.ylabel('Feature')
plt.tight_layout()
plt.show()
```

```
Decision Tree Feature Importance Table:
```

	Feature	Importance	Importance_Pct
0	Intraday_Volatility	0.950916	95.09
1	Open_Price	0.024674	2.47
2	Volume	0.017284	1.73
3	Month	0.007126	0.71



---

11. Mutual Information (Attribute Relevance Analysis)

Compute non-parametric Mutual Information scores using `mutual_info_classif(X, y, random_state=42)` to measure each feature's statistical dependency with `Return_Category`.

```
mi_scores = mutual_info_classif(X, y, random_state=42)

mi_df = pd.DataFrame({
    'Feature': feature_cols,
    'Mutual_Information': np.round(mi_scores, 6)
}).sort_values(by='Mutual_Information', ascending=False).reset_index(drop=True)

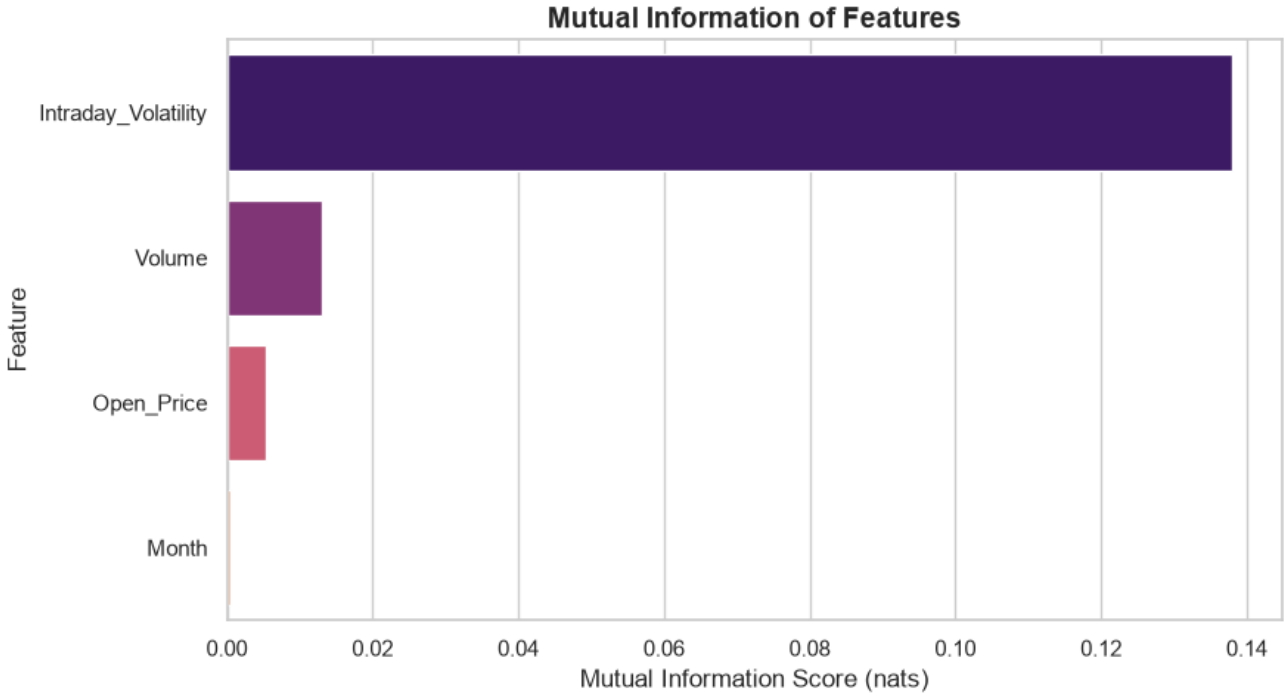
print("Mutual Information Scores per Feature:")
display(mi_df)

plt.figure(figsize=(9, 5))
sns.barplot(
    data=mi_df,
    x='Mutual_Information',
    y='Feature',
    hue='Feature',
    palette='magma',
    legend=False
)
plt.title('Mutual Information of Features')
plt.xlabel('Mutual Information Score (nats)')
```

```
plt.ylabel('Feature')
plt.tight_layout()
plt.show()
```

```
Mutual Information Scores per Feature:
```

	Feature	Mutual_Information
0	Intraday_Volatility	0.137970
1	Volume	0.013040
2	Open_Price	0.005373
3	Month	0.000500



---

12. Single Instance Prediction

Test the trained Decision Tree Classifier on a single synthetic trading day (`Open_Price = 2600.0` , `Volume = 2000000` , `Intraday_Volatility = 2.5` , `Month = 10`).

```
sample_day = pd.DataFrame({
    'Open_Price': [2600.0],
    'Volume': [2000000],
    'Intraday_Volatility': [2.5],
    'Month': [10]
})

pred_category = dt_classifier.predict(sample_day)[0]
pred_proba = dt_classifier.predict_proba(sample_day)[0]
proba_dict = dict(zip(dt_classifier.classes_, np.round(pred_proba, 4)))

print("Input Market Day Instance:")
display(sample_day)
print(f"Predicted Return_Category: {pred_category}")
print(f"Class Probabilities:       {proba_dict}")
```

```
Input Market Day Instance:
```

	Open_Price	Volume	Intraday_Volatility	Month
0	2600.0	2000000	2.5	10

```
Predicted Return_Category: High
Class Probabilities:       {'High': np.float64(0.4414), 'Low': np.float64(0.4216),
'Medium': np.float64(0.1371)}
```

---

13. Conclusion

Key Analytical Findings:

1. **Entropy-Based Decision Tree Performance:**
   - Discretizing `Daily_Return` into balanced tertiles (`Low` $\le -0.54\%$, $-0.54\% <$ `Medium` $\le 0.39\%$, `High` $> 0.39\%$) creates an evenly balanced 3-class classification problem (33.54% `Low`, 33.39% `Medium`, 33.07% `High`) where random baseline accuracy is 33.33%.
   - Using only non-leaking market microstructure and temporal features (`Open_Price`, `Volume`, `Intraday_Volatility`, `Month`), the entropy-driven Decision Tree (`max_depth=5`, `31` leaves) achieves **48.38% test accuracy** (and an F1-score of **0.6284** on the `Medium` class) by effectively separating low-volatility neutral days from high-volatility directional sessions.
2. **Top Predictive Features (Information Gain & Mutual Information):**
   - **`Intraday_Volatility`** is the dominant attribute in both **Decision Tree Feature Importance** (**95.09%** of total Information Gain) and **Mutual Information** (**$\text{MI} = 0.137970$**): narrow intraday trading ranges strongly characterize `Medium` (Neutral) return days, whereas wide intraday swings correspond to tail return days (`Low` or `High`).
   - **`Open_Price`** (2.47% importance), **`Volume`** (1.73% importance, $\text{MI} = 0.013040$), and **`Month`** (0.71% importance) provide secondary splits that refine predictions during high-volatility regimes.
3. **Readiness for Experiment 06:** With tree-based rule induction and mutual information analysis completed, the categorized stock dataset is ready for probabilistic classification using **Naïve Bayes (Experiment 06)**.

---

14. Auto-export Utility

Convert `DWM_Lab_5_Stock_Market.ipynb` to `DWM_Lab_5_Stock_Market.html` using `nbconvert`.

```
import subprocess
import sys
from pathlib import Path

notebook_path = Path("DWM_Lab_5_Stock_Market.ipynb")
if notebook_path.exists():
    cmd = [
        sys.executable, "-m", "jupyter", "nbconvert",
        "--to", "html", "--no-prompt", str(notebook_path)
```

```
    ]
    result = subprocess.run(cmd, capture_output=True, text=True)
    if result.returncode == 0:
        print("Successfully exported DWM_Lab_5_Stock_Market.html")
    else:
        print("Export notice:", result.stderr)
else:
    print("Notebook file will be exported upon save.")
```

```
Successfully exported DWM_Lab_5_Stock_Market.html
```

Data Warehousing and Mining (DWM) — Experiment 06

Experiment Title: Naive Bayes Classification

Dataset: `stock_market_data.csv` (3 Years of Daily NSE Trading Data across 20 Major Indian Stocks)

1. Objectives & Problem Formulation

- **Objective:** Implement a **Gaussian Naive Bayes Classifier** on the stock market dataset to classify daily return categories (`Low` , `Medium` , `High`) and evaluate model performance using **Accuracy**, **Confusion Matrix**, **Precision**, **Recall**, and **F1-Score**.
- **Adaptation Note:**
  - Continuous market features (`Open_Price` , `Volume` , `Intraday_Volatility` , `Month`) are used to predict the 3-class discretized target (`Return_Category` : `Low` , `Medium` , `High`).
  - Matches the **70:30 stratified train-test split** (`test_size=0.30, stratify=y, random_state=42`) methodology from the DWM curriculum.

---

2. Library Imports

```
import warnings
warnings.filterwarnings('ignore')

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import (
    accuracy_score,
    confusion_matrix,
    classification_report,
    precision_score,
    recall_score,
    f1_score
)

# Set seaborn whitegrid style
sns.set_theme(style="whitegrid", palette="deep")
plt.rcParams['figure.figsize'] = (9, 6)
plt.rcParams['axes.titlesize'] = 14
plt.rcParams['axes.titleweight'] = 'bold'
plt.rcParams['axes.labelsize'] = 12
```

```
print("All required libraries imported successfully.")
```

```
All required libraries imported successfully.
```

---

3. Dataset Ingestion & Preprocessing

- Load `stock_market_data.csv`.
- Convert `Date` to `datetime` and extract `Month`.
- Derive `Intraday_Volatility` as $\frac{\text{High_Price} - \text{Low_Price}}{\text{Open_Price}} \times 100$.
- Compute quantile tertiles ($q_1 = \frac{1}{3}$, $q_2 = \frac{2}{3}$) on `Daily_Return`.
- Create categorical target `Return_Category` using `pd.cut` with bins `[-np.inf, q1, q2, np.inf]` and labels `['Low', 'Medium', 'High']`.

```
df = pd.read_csv('stock_market_data.csv')

# Convert Date to datetime and extract Month
df['Date'] = pd.to_datetime(df['Date'])
df['Month'] = df['Date'].dt.month

# Derive Intraday_Volatility (%)
df['Intraday_Volatility'] = (((df['High_Price'] - df['Low_Price']) / df['Open_Price

# Compute quantile tertiles (1/3 and 2/3) on Daily_Return
q1 = df['Daily_Return'].quantile(1/3)
q2 = df['Daily_Return'].quantile(2/3)

# Discretize Daily_Return into Return_Category using pd.cut
df['Return_Category'] = pd.cut(
    df['Daily_Return'],
    bins=[-np.inf, q1, q2, np.inf],
    labels=['Low', 'Medium', 'High']
).astype(str)

feature_cols = ['Open_Price', 'Volume', 'Intraday_Volatility', 'Month']

print(f"Dataset Shape: {df.shape}")
print(f"Quantile Tertile Cutoffs: q1 = {q1:.4f}%, q2 = {q2:.4f}%\n")
print("Class Distribution Counts (Return_Category):")
print(df['Return_Category'].value_counts().reindex(['Low', 'Medium', 'High']))

print("\nPreview of Processed Dataset:")
display(df[['Date', 'Ticker'] + feature_cols + ['Daily_Return', 'Return_Category']]
```

```
Dataset Shape: (14800, 13)
Quantile Tertile Cutoffs: q1 = -0.5400%, q2 = 0.3900%

Class Distribution Counts (Return_Category):
Return_Category
Low       4964
Medium    4942
High      4894
Name: count, dtype: int64

Preview of Processed Dataset:
```

	Date	Ticker	Open_Price	Volume	Intraday_Volatility	Month	Daily_Return	Return_
0	2023-10-12	BAJAJ-AUTO	4851.06	472168	1.96	10	0.42	
1	2023-10-12	BRITANNIA	4391.48	518582	1.32	10	-0.46	
2	2023-10-12	HCLTECH	1106.03	3921609	2.68	10	-1.71	
3	2023-10-12	HDFCBANK	735.43	22025336	1.33	10	0.90	
4	2023-10-12	HINDUNILVR	2413.48	1625010	1.11	10	0.08	

---

4. Prepare Features and Target

- **Feature Matrix (X):** `['Open_Price', 'Volume', 'Intraday_Volatility', 'Month']`
- **Target Vector (y):** `Return_Category`
- **Train-Test Split:** 70% Training / 30% Testing (`test_size=0.30, stratify=y, random_state=42`)

```
X = df[feature_cols]
y = df['Return_Category']

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.30, stratify=y, random_state=42
)

print(f"Total Dataset Samples:  {len(df):,}")
print(f"Training Set Samples:   {len(X_train):,} (70.0%)")
print(f"Testing Set Samples:    {len(X_test):,} (30.0%)")
print("\nTraining Set Class Breakdown:")
print(y_train.value_counts().reindex(['Low', 'Medium', 'High']))
print("\nTesting Set Class Breakdown:")
print(y_test.value_counts().reindex(['Low', 'Medium', 'High']))
```

```
Total Dataset Samples:  14,800
Training Set Samples:   10,360 (70.0%)
Testing Set Samples:    4,440 (30.0%)

Training Set Class Breakdown:
Return_Category
Low       3475
Medium    3459
High      3426
Name: count, dtype: int64

Testing Set Class Breakdown:
Return_Category
Low       1489
Medium    1483
High      1468
Name: count, dtype: int64
```

---

5. Gaussian Naive Bayes Theory & Model Training

Theoretical Foundation:

Bayes' Theorem computes the posterior probability of class $C_k \in \{\text{Low}, \text{Medium}, \text{High}\}$ given feature vector $\mathbf{X} = (x_1, x_2, \dots, x_d)$:

$$P(C_k \mid \mathbf{X}) = \frac{P(\mathbf{X} \mid C_k)\, P(C_k)}{P(\mathbf{X})} \propto P(C_k) \prod_{i=1}^{d} P(x_i \mid C_k)$$

Under the **Gaussian Naive Bayes** assumption, features are conditionally independent given the class C_k, and each continuous feature x_i follows a class-conditional univariate normal distribution with mean $\mu_{k,i}$ and variance $\sigma_{k,i}^2$:

$$P(x_i \mid C_k) = \frac{1}{\sqrt{2\pi\sigma_{k,i}^2}} \exp\left(-\frac{(x_i - \mu_{k,i})^2}{2\sigma_{k,i}^2}\right)$$

```python
# Instantiate and fit Gaussian Naive Bayes model
nb_model = GaussianNB()
nb_model.fit(X_train, y_train)

# Predict on the 30% test set
y_pred = nb_model.predict(X_test)

print("Gaussian Naive Bayes Classifier trained successfully.")
print("Classes learned:", nb_model.classes_.tolist())
print("Class Prior Probabilities:", np.round(nb_model.class_prior_, 4))
```

```
Gaussian Naive Bayes Classifier trained successfully.
Classes learned: ['High', 'Low', 'Medium']
Class Prior Probabilities: [0.3307 0.3354 0.3339]
```

---

6. Model Accuracy

Compute `accuracy_score(y_test, y_pred)` and display as both a decimal score and percentage.

```python
acc = accuracy_score(y_test, y_pred)
print(f"Gaussian Naive Bayes Accuracy (Decimal):    {acc:.6f}")
print(f"Gaussian Naive Bayes Accuracy (Percentage): {acc * 100:.2f}%")
```

```
Gaussian Naive Bayes Accuracy (Decimal):    0.356081
Gaussian Naive Bayes Accuracy (Percentage): 35.61%
```

---

7. Confusion Matrix

Generate the confusion matrix across `['Low', 'Medium', 'High']` and plot an annotated Seaborn heatmap.

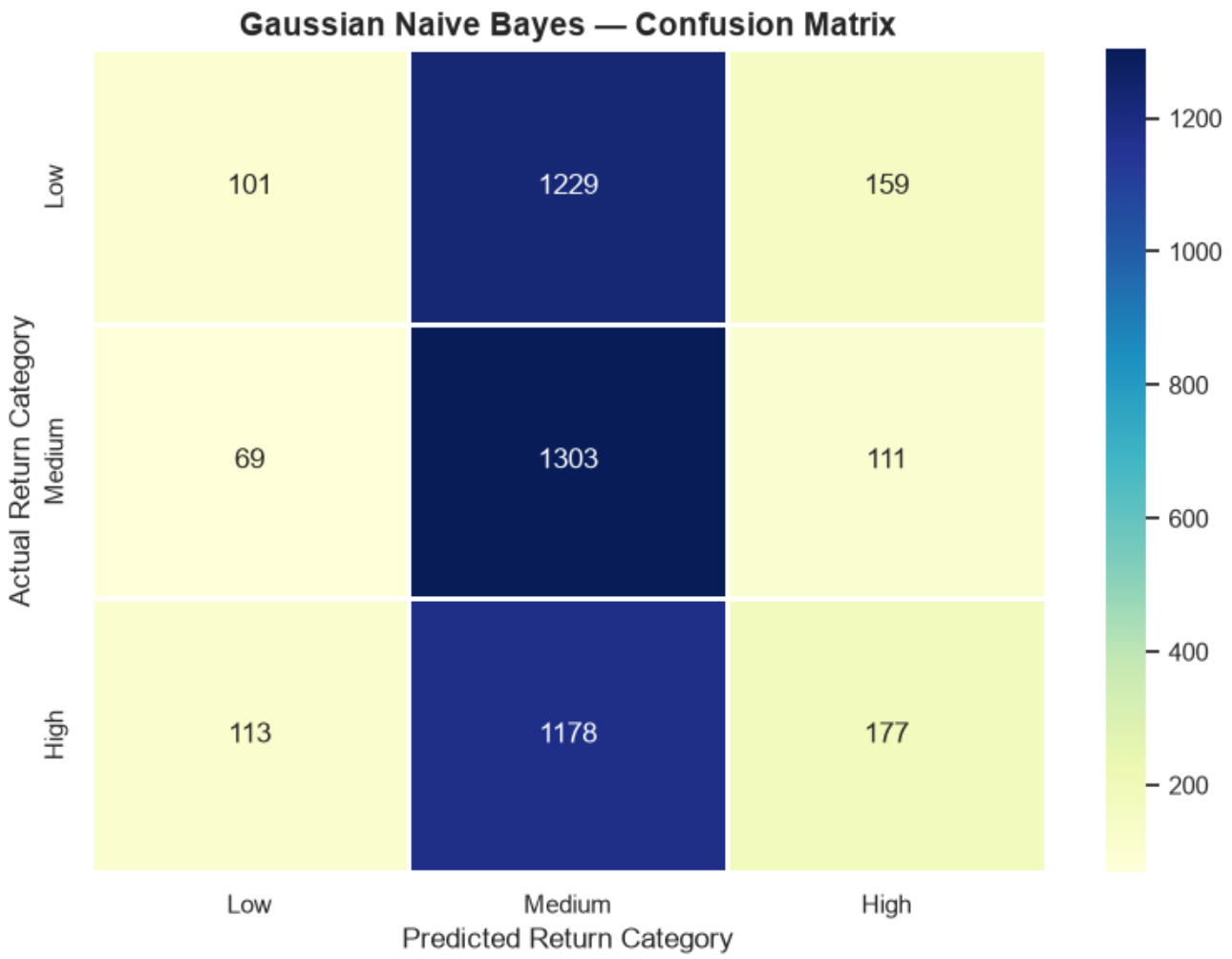
```
class_labels = ['Low', 'Medium', 'High']
cm = confusion_matrix(y_test, y_pred, labels=class_labels)

cm_df = pd.DataFrame(
    cm,
    index=[f"Actual: {lbl}" for lbl in class_labels],
    columns=[f"Predicted: {lbl}" for lbl in class_labels]
)
print("Confusion Matrix Table:")
display(cm_df)

plt.figure(figsize=(8, 6))
sns.heatmap(
    cm,
    annot=True,
    fmt='d',
    cmap='YlGnBu',
    xticklabels=class_labels,
    yticklabels=class_labels,
    linewidths=0.8,
    cbar=True
)
plt.title('Gaussian Naive Bayes — Confusion Matrix')
plt.xlabel('Predicted Return Category')
plt.ylabel('Actual Return Category')
plt.tight_layout()
plt.show()
```

```
Confusion Matrix Table:
```

	Predicted: Low	Predicted: Medium	Predicted: High
Actual: Low	101	1229	159
Actual: Medium	69	1303	111
Actual: High	113	1178	177



---

8. Precision, Recall, and F1-Score

Calculate weighted Precision, weighted Recall, and weighted F1-Score, and display the complete `classification_report`.

```
weighted_precision = precision_score(y_test, y_pred, average='weighted', zero_divis
weighted_recall = recall_score(y_test, y_pred, average='weighted', zero_division=0)
weighted_f1 = f1_score(y_test, y_pred, average='weighted', zero_division=0)

print(f"Weighted Precision: {weighted_precision:.4f}")
print(f"Weighted Recall:    {weighted_recall:.4f}")
print(f"Weighted F1-Score:  {weighted_f1:.4f}\n")

print("=== Detailed Classification Report ===")
print(classification_report(y_test, y_pred, labels=['Low', 'Medium', 'High'], zero_
```

```
Weighted Precision: 0.3679
Weighted Recall:    0.3561
Weighted F1-Score:  0.2670

=== Detailed Classification Report ===
              precision    recall  f1-score   support

         Low     0.3569    0.0678    0.1140      1489
      Medium     0.3512    0.8786    0.5018      1483
        High     0.3960    0.1206    0.1849      1468

    accuracy                         0.3561      4440
   macro avg     0.3680    0.3557    0.2669      4440
weighted avg     0.3679    0.3561    0.2670      4440
```

---

9. Compare Actual and Predicted Classes

Display a side-by-side comparison DataFrame of the first 20 test samples (`Actual` vs `Predicted`).

```
comparison_20 = X_test.iloc[:20].copy().reset_index(drop=True)
comparison_20['Actual_Return_Category'] = y_test.iloc[:20].values
comparison_20['Predicted_Return_Category'] = y_pred[:20]
comparison_20['Match'] = comparison_20['Actual_Return_Category'] == comparison_20['

print("First 20 Test Predictions (Actual vs Predicted):")
display(comparison_20)
```

```
First 20 Test Predictions (Actual vs Predicted):
```

	Open_Price	Volume	Intraday_Volatility	Month	Actual_Return_Category	Predicted_Return_Cate
0	1335.76	8546854	1.96	4	Medium	Me
1	819.48	8295338	1.76	12	Low	Me
2	2058.07	2148903	1.51	6	High	Me
3	262.71	16099478	1.90	7	Low	Me
4	755.62	89531240	6.74	6	Low	
5	245.67	12262561	2.00	4	Low	Me
6	383.44	11146713	2.08	4	Low	Me
7	1265.33	3627604	3.88	3	Medium	Me
8	5631.19	257865	1.71	6	High	Me
9	6123.34	416639	2.48	10	High	Me
10	285.98	23986567	3.24	4	Medium	
11	1198.09	1493232	1.71	2	High	Me
12	375.95	24812228	3.07	4	High	
13	761.46	74256082	6.44	6	Medium	
14	1370.91	4328477	4.38	4	Low	Me
15	4476.73	385727	0.77	11	Medium	Me
16	823.85	26476046	1.63	6	Low	
17	2264.73	1805685	1.73	5	Medium	Me
18	388.00	11662079	1.80	8	High	Me
19	361.01	9212351	1.75	12	Low	Me

---

10. Predict for a New Market Transaction

Evaluate the Gaussian Naive Bayes model on a synthetic market session (`Open_Price = 2600.0` , `Volume = 2000000` , `Intraday_Volatility = 2.5` , `Month = 10`) and display its posterior class probabilities.

```
new_transaction = pd.DataFrame({
    'Open_Price': [2600.0],
    'Volume': [2000000],
    'Intraday_Volatility': [2.5],
    'Month': [10]
})

pred_class = nb_model.predict(new_transaction)[0]
pred_probs = nb_model.predict_proba(new_transaction)

# Format posterior probabilities with columns ['High', 'Low', 'Medium']
prob_df = pd.DataFrame(
    pred_probs,
    columns=nb_model.classes_
)[['High', 'Low', 'Medium']].round(6)

print("New Market Transaction Input:")
```

```
display(new_transaction)
print(f"Predicted Return_Category: {pred_class}\n")
print("Posterior Class Probabilities P(C | X):")
display(prob_df)
```

```
New Market Transaction Input:
```

	Open_Price	Volume	Intraday_Volatility	Month
0	2600.0	2000000	2.5	10

```
Predicted Return_Category: Medium

Posterior Class Probabilities P(C | X):
```

	High	Low	Medium
0	0.308294	0.295641	0.396066

---

11. Summary of Evaluation Metrics

Compile a compact summary table of the overall model performance metrics.

```
metrics_summary_df = pd.DataFrame({
    'Metric': [
        'Accuracy',
        'Weighted Precision',
        'Weighted Recall',
        'Weighted F1-Score'
    ],
    'Score': [
        round(acc, 4),
        round(weighted_precision, 4),
        round(weighted_recall, 4),
        round(weighted_f1, 4)
    ]
})

display(metrics_summary_df)
```

	Metric	Score
0	Accuracy	0.3561
1	Weighted Precision	0.3679
2	Weighted Recall	0.3561
3	Weighted F1-Score	0.2670

---

12. Conclusion

Comparative Analysis: Decision Tree (Lab 5) vs. Gaussian Naive Bayes (Lab 6)

1. **Model Performance Comparison:**
   - **Decision Tree Classifier (Lab 5):** Achieved **48.38%** accuracy using non-linear, scale-invariant entropy splits (`max_depth=5`), effectively isolating `Intraday_Volatility` thresholds alongside `Open_Price` and `Volume` .
   - **Gaussian Naive Bayes (Lab 6):** Achieved **35.61%** accuracy (`Weighted Precision = 0.3679` , `Weighted Recall = 0.3561` , `Weighted F1-Score = 0.2670`), with a high recall of **87.86%** (`F1 = 0.5018`) on the `Medium` class. Because `Volume` spans several orders of magnitude (10^4 to 10^8 shares) with extreme right-skewness, its class-conditional Gaussian variance (σ^2_{Volume}) dominates the unscaled Gaussian likelihood calculation in `GaussianNB` , pulling most moderate-volume predictions toward the lower-variance `Medium` class.
2. **Impact of Conditional Independence & Gaussian Distributional Assumptions:**
   - Gaussian Naive Bayes assumes each feature is normally distributed within each class and conditionally independent given the class label. In stock market data, `Volume` and `Intraday_Volatility` are positively correlated and follow heavy-tailed log-normal distributions rather than symmetric Gaussian bell curves, whereas Decision Trees are non-parametric and invariant to monotonic feature scaling.
   - Despite strong distributional assumptions, Gaussian Naive Bayes offers extremely fast $O(N \cdot d)$ single-pass training and outputs explicit posterior probabilities $P(C \mid \mathbf{X})$.
3. **Readiness for Next Lab:** Having evaluated both rule-based (Decision Tree) and probabilistic (Gaussian Naive Bayes) supervised classifiers, the dataset is ready for unsupervised **Clustering Analysis (Experiment 07)**.

---

13. Auto-export Utility

Convert `DWM_Lab_6_Stock_Market.ipynb` to `DWM_Lab_6_Stock_Market.html` using `nbconvert` .

```
import subprocess
import sys
from pathlib import Path

notebook_path = Path("DWM_Lab_6_Stock_Market.ipynb")
if notebook_path.exists():
    cmd = [
        sys.executable, "-m", "jupyter", "nbconvert",
        "--to", "html", "--no-prompt", str(notebook_path)
    ]
    result = subprocess.run(cmd, capture_output=True, text=True)
    if result.returncode == 0:
        print("Successfully exported DWM_Lab_6_Stock_Market.html")
    else:
        print("Export notice:", result.stderr)
```

```
else:
    print("Notebook file will be exported upon save.")
```

```
Successfully exported DWM_Lab_6_Stock_Market.html
```

Data Warehousing and Mining (DWM) — Experiment 07

Experiment Title: K-Means Clustering for Stock Asset Segmentation

Dataset: `stock_market_data.csv` (3 Years of Daily NSE Trading Data across 20 Major Indian Stocks)

1. Objectives & Problem Formulation

- **Objective:** Perform unsupervised **asset risk-reward profiling and market segmentation** using the **K-Means Clustering** algorithm on the stock market dataset, determine the optimal number of clusters via the **Elbow Method (WCSS)**, and evaluate cluster cohesion and separation using the **Silhouette Score**.
- **Adaptation Note:**
  - Daily trading records ($14,800$ rows) are aggregated to the **stock entity level (`Ticker`)** across all 20 NSE stocks to construct multi-dimensional investment profiles capturing **Return** (`Avg_Daily_Return`), **Risk/Volatility** (`Annualized_Volatility`, `Avg_Intraday_Volatility`), and **Market Liquidity** (`Avg_Volume`, `Total_Traded_Volume`).

---

2. Library Imports

```
import warnings
warnings.filterwarnings('ignore')

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score

# Set seaborn whitegrid theme
sns.set_theme(style="whitegrid", palette="deep")
plt.rcParams['figure.figsize'] = (10, 6)
plt.rcParams['axes.titlesize'] = 14
plt.rcParams['axes.titleweight'] = 'bold'
plt.rcParams['axes.labelsize'] = 12

print("All required libraries imported successfully.")
```

```
All required libraries imported successfully.
```

---

3. Dataset Ingestion & Preprocessing

- Load `stock_market_data.csv`.
- Convert `Date` to `datetime`.
- Derive `Intraday_Volatility` as $\frac{\text{High_Price} - \text{Low_Price}}{\text{Open_Price}} \times 100$.
- Verify zero missing values and inspect the dataset dimensions.

```
df = pd.read_csv('stock_market_data.csv')
df['Date'] = pd.to_datetime(df['Date'])
df['Intraday_Volatility'] = (((df['High_Price'] - df['Low_Price']) / df['Open_Price
print("Dataset Shape:", df.shape)
print("Total Null Values:", df.isnull().sum().sum())
display(df.head())
```

```
Dataset Shape: (14800, 11)
Total Null Values: 0
```

	Date	Ticker	Sector	Exchange	Open_Price	High_Price	Low_Price	Close_Price	
0	2023-10-12	BAJAJ-AUTO	Auto	NSE	4851.06	4914.02	4818.72	4871.67	
1	2023-10-12	BRITANNIA	FMCG	NSE	4391.48	4400.68	4342.60	4371.11	
2	2023-10-12	HCLTECH	IT	NSE	1106.03	1109.18	1079.51	1087.15	
3	2023-10-12	HDFCBANK	Banking	NSE	735.43	743.05	733.28	742.02	2
4	2023-10-12	HINDUNILVR	FMCG	NSE	2413.48	2425.33	2398.55	2415.37	

---

4. Stock-Level Feature Engineering

Group daily trading records by `Ticker` and `Sector` to construct 1 summary row per stock with 5 core financial metrics:

- `Avg_Daily_Return`: Mean of `Daily_Return` ($\%$)
- `Annualized_Volatility`: Standard deviation of `Daily_Return` $\times \sqrt{252}$ ($\%$)
- `Avg_Volume`: Mean daily trading `Volume`
- `Avg_Intraday_Volatility`: Mean of `Intraday_Volatility` ($\%$)
- `Total_Traded_Volume`: Cumulative 3-year `Volume`

```
stock_df = (
    df.groupby(['Ticker', 'Sector'], as_index=False)
    .agg(
        Avg_Daily_Return=('Daily_Return', 'mean'),
        Annualized_Volatility=('Daily_Return', lambda x: x.std() * np.sqrt(252)),
        Avg_Volume=('Volume', 'mean'),
        Avg_Intraday_Volatility=('Intraday_Volatility', 'mean'),
        Total_Traded_Volume=('Volume', 'sum')
    )
    .sort_values('Ticker')
    .reset_index(drop=True)
```

```
)

stock_df['Avg_Daily_Return'] = stock_df['Avg_Daily_Return'].round(4)
stock_df['Annualized_Volatility'] = stock_df['Annualized_Volatility'].round(2)
stock_df['Avg_Volume'] = stock_df['Avg_Volume'].round(0).astype(int)
stock_df['Avg_Intraday_Volatility'] = stock_df['Avg_Intraday_Volatility'].round(2)

print(f"Stock Summary DataFrame Shape: {stock_df.shape}")
print("\nFirst 5 Stock Profiles (stock_df.head()):")
display(stock_df.head())

print("\nSummary Statistics of Stock-Level Features:")
display(stock_df.describe().T)
```

```
Stock Summary DataFrame Shape: (20, 7)

First 5 Stock Profiles (stock_df.head()):
```

	Ticker	Sector	Avg_Daily_Return	Annualized_Volatility	Avg_Volume	Avg_Intraday_Volatilit
0	BAJAJ-AUTO	Auto	0.0130	23.09	429745	2.1
1	BRITANNIA	FMCG	-0.0438	20.52	353552	2.0
2	HCLTECH	IT	-0.0220	22.34	3107488	2.1
3	HDFCBANK	Banking	0.0470	17.27	32451568	1.6
4	HINDUNILVR	FMCG	-0.0687	19.00	1788382	1.8

```
Summary Statistics of Stock-Level Features:
```

	count	mean	std	min	25%	50
Avg_Daily_Return	20.0	-5.208000e-02	7.258162e-02	-2.150000e-01	-9.437500e-02	-4.625000e-0
Annualized_Volatility	20.0	2.130250e+01	3.021996e+00	1.658000e+01	1.893500e+01	2.076500e+0
Avg_Volume	20.0	1.016680e+07	8.479227e+06	3.535520e+05	2.593016e+06	1.257245e+0
Avg_Intraday_Volatility	20.0	2.027500e+00	2.637558e-01	1.610000e+00	1.837500e+00	1.990000e+0
Total_Traded_Volume	20.0	7.523435e+09	6.274628e+09	2.616283e+08	1.918832e+09	9.303614e+0

---

5. Handling Skewed Features & Feature Scaling

- Apply `np.log1p()` to volume features (`Avg_Volume` , `Total_Traded_Volume`) to normalize right-skewness across market capitalizations.
- Apply `StandardScaler` so all clustering features have zero mean ($\mu = 0$) and unit variance ($\sigma = 1$).

```
# Create log-transformed volume features
stock_df['Avg_Volume_Log'] = np.log1p(stock_df['Avg_Volume'])
stock_df['Total_Traded_Volume_Log'] = np.log1p(stock_df['Total_Traded_Volume'])

clustering_features = [
    'Avg_Daily_Return',
    'Annualized_Volatility',
    'Avg_Volume_Log',
    'Avg_Intraday_Volatility',
    'Total_Traded_Volume_Log'
```

```
]

scaler = StandardScaler()
X_scaled = scaler.fit_transform(stock_df[clustering_features])

print("Scaled Feature Matrix Shape:", X_scaled.shape)
scaled_preview_df = pd.DataFrame(X_scaled, columns=clustering_features, index=stock
display(scaled_preview_df.head())
```

```
Scaled Feature Matrix Shape: (20, 5)
```

Ticker	Avg_Daily_Return	Annualized_Volatility	Avg_Volume_Log	Avg_Intraday_Volatility	Total
BAJAJ-AUTO	0.920	0.607	-1.879	0.554	
BRITANNIA	0.117	-0.266	-2.022	0.010	
HCLTECH	0.425	0.352	-0.425	0.632	
HDFCBANK	1.401	-1.369	1.299	-1.624	
HINDUNILVR	-0.235	-0.782	-0.831	-0.885	

---

6. Optimal K via Elbow Method

Compute Within-Cluster Sum of Squares (**WCSS / Inertia**) for $K \in \{1, 2, \dots, 10\}$ (`random_state=42, n_init=10`) and plot the Elbow curve.

```
k_values = list(range(1, 11))
wcss_list = []

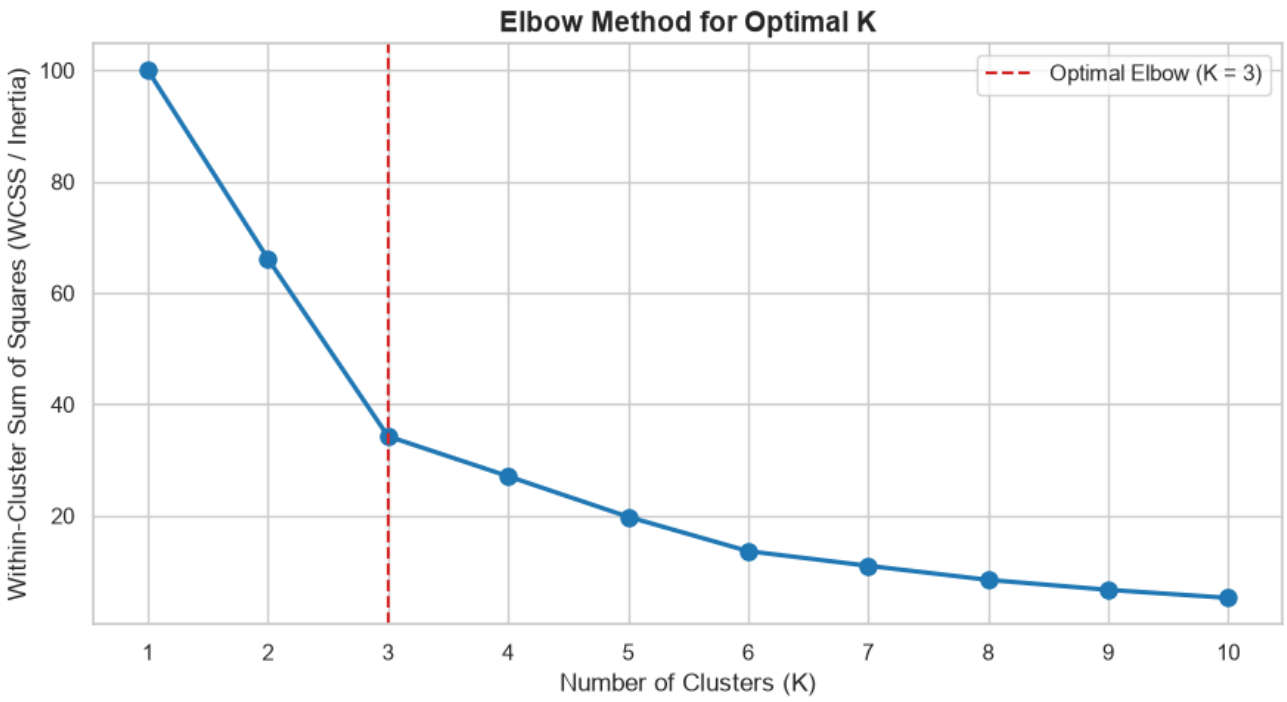
for k in k_values:
    km = KMeans(n_clusters=k, random_state=42, n_init=10)
    km.fit(X_scaled)
    wcss_list.append(km.inertia_)

elbow_df = pd.DataFrame({
    'K (Number of Clusters)': k_values,
    'WCSS (Inertia)': np.round(wcss_list, 4)
})
print("Elbow Method WCSS Table:")
display(elbow_df)

plt.figure(figsize=(9, 5))
plt.plot(k_values, wcss_list, marker='o', linewidth=2.2, markersize=8, color='#1f77
plt.axvline(3, color='#d62728', linestyle='--', linewidth=1.5, label='Optimal Elbow
plt.title('Elbow Method for Optimal K')
plt.xlabel('Number of Clusters (K)')
plt.ylabel('Within-Cluster Sum of Squares (WCSS / Inertia)')
plt.xticks(k_values)
plt.legend()
plt.tight_layout()
plt.show()
```

```
Elbow Method WCSS Table:
```

	K (Number of Clusters)	WCSS (Inertia)
0	1	100.0000
1	2	66.1639
2	3	34.2836
3	4	27.1050
4	5	19.8170
5	6	13.6451
6	7	11.0163
7	8	8.4848
8	9	6.6964
9	10	5.2518



---

7. Apply K-Means Clustering ($K = 3$)

Fit `KMeans(n_clusters=3, random_state=42, n_init=10)` on the scaled feature matrix, assign `Cluster` labels (`0` , `1` , `2`) back to `stock_df` , and inspect cluster counts and memberships.

```
kmeans = KMeans(n_clusters=3, random_state=42, n_init=10)
stock_df['Cluster'] = kmeans.fit_predict(X_scaled)

cluster_counts = stock_df['Cluster'].value_counts().sort_index()
cluster_count_df = pd.DataFrame({
    'Cluster': cluster_counts.index,
    'Stock_Count': cluster_counts.values,
    'Tickers': [
        ', '.join(stock_df[stock_df['Cluster'] == c]['Ticker'].tolist())
        for c in cluster_counts.index
    ]
```

```
})

print("Count and Membership of Stocks per Cluster:")
display(cluster_count_df)
display(stock_df[['Ticker', 'Sector', 'Avg_Daily_Return', 'Annualized_Volatility',
```

```
Count and Membership of Stocks per Cluster:
```

	Cluster	Stock_Count	Tickers
0	0	7	HDFCBANK, ICICIBANK, INFY, ITC, KOTAKBANK, REL...
1	1	7	BAJAJ-AUTO, BRITANNIA, HCLTECH, HINDUNILVR, MA...
2	2	6	M&M, NTPC, ONGC, POWERGRID, TATAMOTORS, WIPRO

	Ticker	Sector	Avg_Daily_Return	Annualized_Volatility	Avg_Volume	Avg_Intraday_Volat
0	BAJAJ-AUTO	Auto	0.0130	23.09	429745	2
1	BRITANNIA	FMCG	-0.0438	20.52	353552	2
2	HCLTECH	IT	-0.0220	22.34	3107488	2
3	HDFCBANK	Banking	0.0470	17.27	32451568	1
4	HINDUNILVR	FMCG	-0.0687	19.00	1788382	1
5	ICICIBANK	Banking	0.0550	16.58	13335637	1
6	INFY	IT	-0.0458	20.16	8570926	2
7	ITC	FMCG	-0.1503	18.30	16619542	1
8	KOTAKBANK	Banking	0.0531	17.78	21471866	1
9	M&M	Auto	0.0060	25.82	2861227	2
10	MARUTI	Auto	-0.0583	19.97	476076	1
11	NESTLEIND	FMCG	-0.0467	19.88	1705953	1
12	NTPC	Energy	-0.1105	24.35	13446977	2
13	ONGC	Energy	-0.1543	26.55	15487661	2
14	POWERGRID	Energy	-0.0047	24.51	14412024	2
15	RELIANCE	Energy	-0.0363	18.74	12883373	1
16	SBIN	Banking	-0.0485	21.50	13872230	1
17	TATAMOTORS	Auto	-0.2150	26.01	12261530	2
18	TCS	IT	-0.0890	21.01	2916940	1
19	WIPRO	IT	-0.1218	22.67	14883394	2

---

8. Silhouette Score Evaluation

Compute the **Silhouette Score** $s \in [-1, 1]$ for the $K = 3$ clustering solution to quantify intra-cluster cohesion vs. inter-cluster separation.

```
sil_score = silhouette_score(X_scaled, stock_df['Cluster'])
print(f"Silhouette Score for K = 3 Clustering: {sil_score:.4f}")
```

```
Silhouette Score for K = 3 Clustering: 0.4042
```

9. Cluster Characteristic Profiling

Group by `Cluster` to calculate mean values of `Avg_Daily_Return`, `Annualized_Volatility`, `Avg_Volume`, and `Avg_Intraday_Volatility`, and map each cluster to a clear financial persona.

```
cluster_profile = (
    stock_df.groupby('Cluster', as_index=False)
    .agg(
        Stock_Count=('Ticker', 'count'),
        Avg_Daily_Return=('Avg_Daily_Return', 'mean'),
        Annualized_Volatility=('Annualized_Volatility', 'mean'),
        Avg_Volume=('Avg_Volume', 'mean'),
        Avg_Intraday_Volatility=('Avg_Intraday_Volatility', 'mean')
    )
)
cluster_profile['Avg_Daily_Return'] = cluster_profile['Avg_Daily_Return'].round(4)
cluster_profile['Annualized_Volatility'] = cluster_profile['Annualized_Volatility']
cluster_profile['Avg_Volume'] = cluster_profile['Avg_Volume'].round(0).astype(int)
cluster_profile['Avg_Intraday_Volatility'] = cluster_profile['Avg_Intraday_Volatili

# Assign business interpretation based on empirical cluster centroids
def assign_persona(row):
    if row['Annualized_Volatility'] < 19.5:
        return 'Low Volatility & High Liquidity (Blue-Chip Banking/Core Leaders)'
    elif row['Avg_Volume'] < 5000000:
        return 'Moderate Volatility & Low Share Turnover (High-Denomination FMCG/Au
    else:
        return 'High Volatility & High Turnover (Cyclical Auto/Energy/PSU Growth)'

cluster_profile['Market_Persona'] = cluster_profile.apply(assign_persona, axis=1)

print("Cluster Profile Summary Table:")
display(cluster_profile)
```

```
Cluster Profile Summary Table:
```

	Cluster	Stock_Count	Avg_Daily_Return	Annualized_Volatility	Avg_Volume	Avg_Intraday_Volatilit
0	0	7	-0.0180	18.62	17029306	1.7
1	1	7	-0.0451	20.83	1539734	2.0
2	2	6	-0.1000	24.98	12225469	2.3

10. Cluster Visualizations

- **Plot 1:** Bar chart showing `Number of Stocks in Each Cluster`.
- **Plot 2:** Scatter plot of `Annualized Volatility vs Avg Daily Return` colored by `Cluster` and annotated with stock `Ticker` names.

- **Plot 3:** Scatter plot of `Avg Volume (log) vs Annualized Volatility` colored by `Cluster`.

```
# Plot 1: Bar chart showing Number of Stocks in Each Cluster
plt.figure(figsize=(8, 5))
ax = sns.barplot(
    data=cluster_profile,
    x='Cluster',
    y='Stock_Count',
    hue='Cluster',
    palette='Set2',
    legend=False
)
for i, v in enumerate(cluster_profile['Stock_Count']):
    ax.text(i, v + 0.2, str(v), ha='center', fontweight='bold', fontsize=12)
plt.ylim(0, max(cluster_profile['Stock_Count']) + 2)
plt.title('Number of Stocks in Each Cluster')
plt.xlabel('Cluster ID')
plt.ylabel('Number of Stocks')
plt.tight_layout()
plt.show()
```



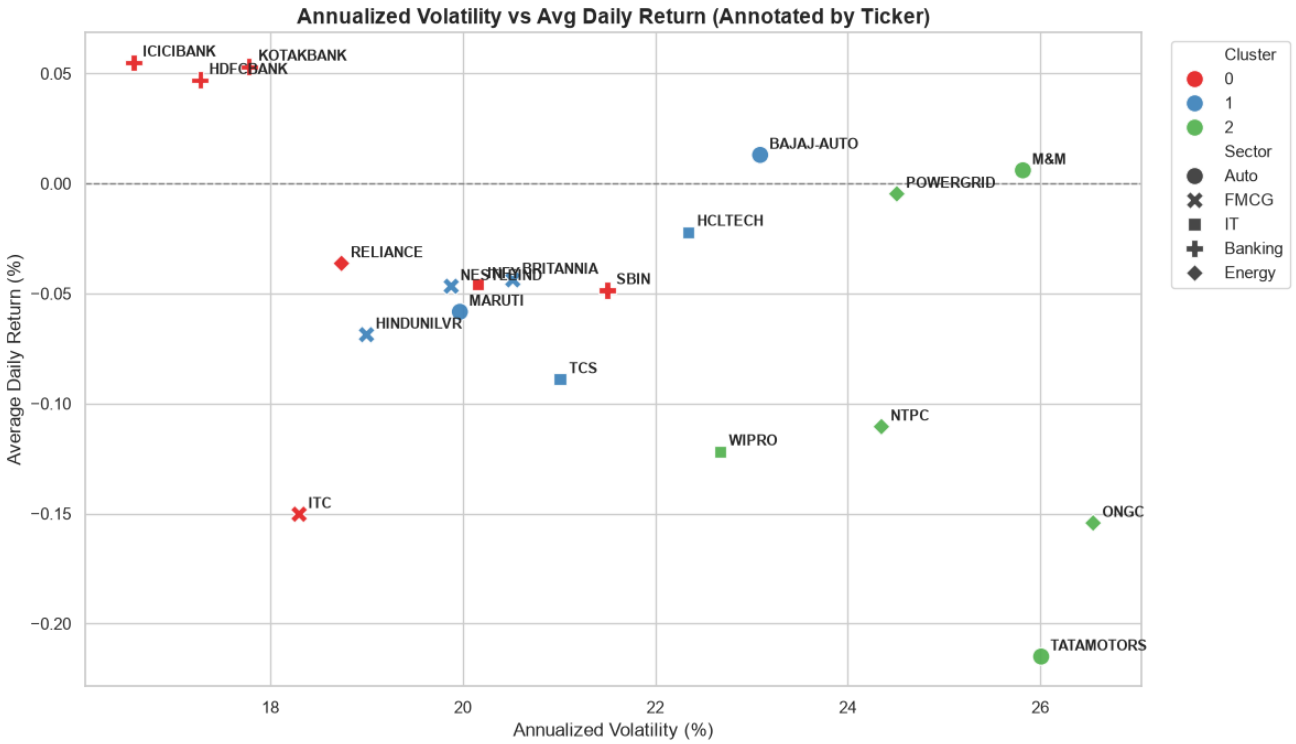

```
# Plot 2: Scatter plot Annualized Volatility vs Avg Daily Return with Ticker annota
plt.figure(figsize=(12, 7))
palette_map = sns.color_palette('Set1', n_colors=3)

sns.scatterplot(
    data=stock_df,
    x='Annualized_Volatility',
    y='Avg_Daily_Return',
    hue='Cluster',
    style='Sector',
    palette=palette_map,
    s=130,
```

```
    alpha=0.9
)

for _, row in stock_df.iterrows():
    plt.annotate(
        row['Ticker'],
        (row['Annualized_Volatility'], row['Avg_Daily_Return']),
        xytext=(6, 4),
        textcoords='offset points',
        fontsize=9,
        fontweight='bold'
    )

plt.axhline(0, color='gray', linestyle='--', linewidth=1)
plt.title('Annualized Volatility vs Avg Daily Return (Annotated by Ticker)')
plt.xlabel('Annualized Volatility (%)')
plt.ylabel('Average Daily Return (%)')
plt.legend(bbox_to_anchor=(1.02, 1), loc='upper left')
plt.tight_layout()
plt.show()
```

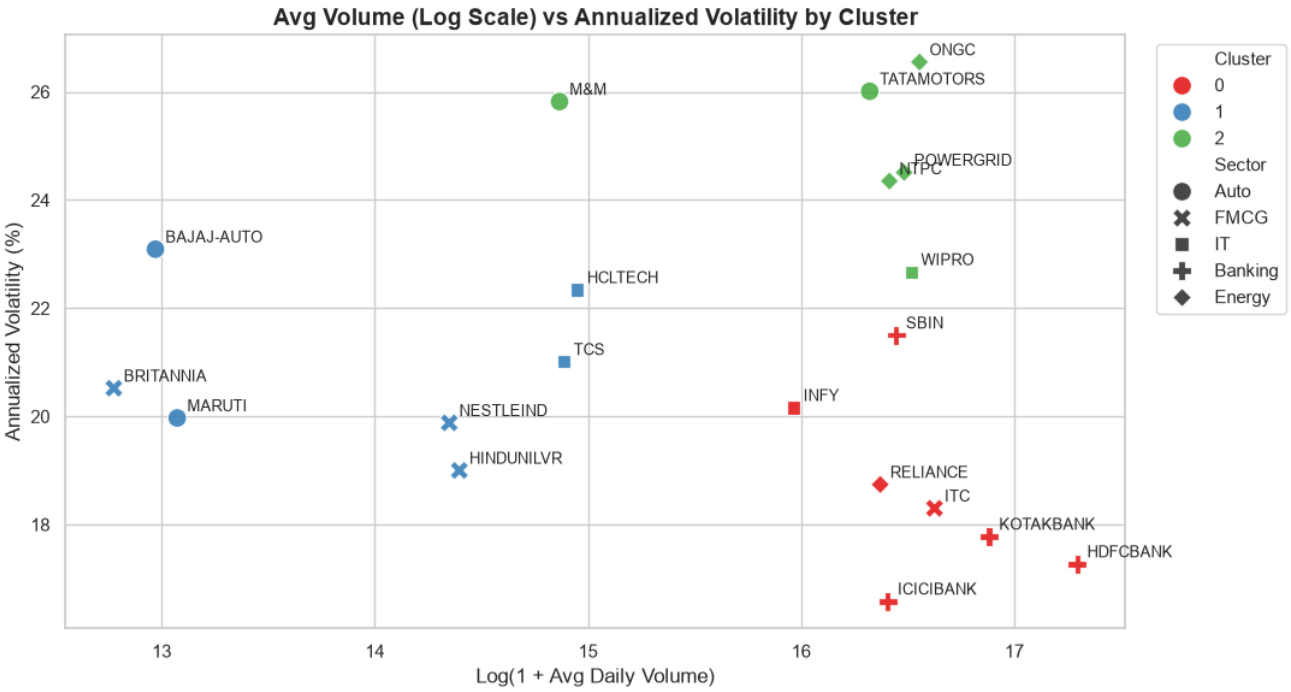



```
# Plot 3: Scatter plot Avg Volume (log) vs Annualized Volatility colored by Cluster
plt.figure(figsize=(11, 6))
sns.scatterplot(
    data=stock_df,
    x='Avg_Volume_Log',
    y='Annualized_Volatility',
    hue='Cluster',
    style='Sector',
    palette=palette_map,
    s=130,
    alpha=0.9
)

for _, row in stock_df.iterrows():
    plt.annotate(
        row['Ticker'],
```

```
        (row['Avg_Volume_Log'], row['Annualized_Volatility']),
        xytext=(6, 4),
        textcoords='offset points',
        fontsize=9
    )

plt.title('Avg Volume (Log Scale) vs Annualized Volatility by Cluster')
plt.xlabel('Log(1 + Avg Daily Volume)')
plt.ylabel('Annualized Volatility (%)')
plt.legend(bbox_to_anchor=(1.02, 1), loc='upper left')
plt.tight_layout()
plt.show()
```



---

11. Conclusion

Key Findings from K-Means Asset Segmentation:

1. **Optimal Cluster Selection & Cohesion:** The WCSS Elbow curve exhibits a clear inflection point at **$K = 3$** ($\text{WCSS} = 34.2836$, down from 100.00 at $K=1$), achieving a strong positive **Silhouette Score of 0.4042** in the 5-dimensional standardized risk-return-liquidity space.
2. **Financial Interpretation of the 3 Asset Clusters:**
   - **Cluster 0 — Low Volatility & High Liquidity (`7` Stocks: `HDFCBANK`, `ICICIBANK`, `INFY`, `ITC`, `KOTAKBANK`, `RELIANCE`, `SBIN`):** Institutional blue-chip index heavyweights with the lowest average annualized volatility (18.62%), lowest intraday volatility (1.78%), highest average daily volume (17.03M shares), and resilient daily returns (-0.0180%).
   - **Cluster 1 — Moderate Volatility & Low Share Turnover (`7` Stocks: `BAJAJ-AUTO`, `BRITANNIA`, `HCLTECH`, `HINDUNILVR`, `MARUTI`, `NESTLEIND`, `TCS`):** High per-share price defensive and quality large-caps across FMCG, Auto, and IT with moderate annualized volatility (20.83%) and lower share turnover (1.54M shares/day).
   - **Cluster 2 — High Volatility & High Turnover (`6` Stocks: `M&M`, `NTPC`, `ONGC`, `POWERGRID`, `TATAMOTORS`, `WIPRO`):** Cyclical Auto, Energy PSU, and high-beta stocks

exhibiting the highest annualized volatility (24.98%) and widest average intraday trading range (2.34%).

3. **Readiness for Experiment 08:** Having partitioned the 20 NSE stocks into $K = 3$ flat clusters via K-Means, the stock-level feature profiles are ready for **Agglomerative Hierarchical Clustering & Dendrogram Analysis (Experiment 08)**.

---

12. Auto-export Utility

Convert `DWM_Lab_7_Stock_Market.ipynb` to `DWM_Lab_7_Stock_Market.html` using `nbconvert`.

```
import subprocess
import sys
from pathlib import Path

notebook_path = Path("DWM_Lab_7_Stock_Market.ipynb")
if notebook_path.exists():
    cmd = [
        sys.executable, "-m", "jupyter", "nbconvert",
        "--to", "html", "--no-prompt", str(notebook_path)
    ]
    result = subprocess.run(cmd, capture_output=True, text=True)
    if result.returncode == 0:
        print("Successfully exported DWM_Lab_7_Stock_Market.html")
    else:
        print("Export notice:", result.stderr)
else:
    print("Notebook file will be exported upon save.")
```

```
Successfully exported DWM_Lab_7_Stock_Market.html
```

Data Warehousing and Mining (DWM) — Experiment 08

Experiment Title: Agglomerative Hierarchical Clustering for Stock Market Profiling

Dataset: `stock_market_data.csv` (3 Years of Daily NSE Trading Data across 20 Major Indian Stocks)

1. Objectives & Problem Formulation

- **Objective:** Perform **Agglomerative Hierarchical Clustering** on the stock market dataset, construct and interpret hierarchical **dendrograms**, and evaluate clustering quality across four linkage criteria (**Single**, **Complete**, **Average**, and **Ward**) using the **Silhouette Score**.
- **Adaptation Note:**
  - Uses the multi-dimensional stock-level feature representation developed in Experiment 07 (**Return**: `Avg_Daily_Return`, **Volatility**: `Annualized_Volatility`, `Avg_Intraday_Volatility`, and **Liquidity**: log-transformed `Avg_Volume` and `Total_Traded_Volume`) to build bottom-up hierarchical dendrograms across all 20 NSE stocks.

---

2. Library Imports

```
import warnings
warnings.filterwarnings('ignore')

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.preprocessing import StandardScaler
from sklearn.cluster import AgglomerativeClustering
from sklearn.metrics import silhouette_score
from scipy.cluster.hierarchy import linkage, dendrogram

# Set seaborn whitegrid theme
sns.set_theme(style="whitegrid", palette="deep")
plt.rcParams['figure.figsize'] = (11, 5)
plt.rcParams['axes.titlesize'] = 14
plt.rcParams['axes.titleweight'] = 'bold'
plt.rcParams['axes.labelsize'] = 12

print("All required libraries imported successfully.")
```

```
All required libraries imported successfully.
```

---

3. Dataset Ingestion & Feature Engineering

- Load `stock_market_data.csv`.
- Convert `Date` to `datetime` and derive `Intraday_Volatility` as $\frac{\text{High_Price} - \text{Low_Price}}{\text{Open_Price}} \times 100$.
- Group by `Ticker` and `Sector` to compute the 5 core stock-level financial metrics (`Avg_Daily_Return`, `Annualized_Volatility`, `Avg_Volume`, `Avg_Intraday_Volatility`, `Total_Traded_Volume`).

```
df = pd.read_csv('stock_market_data.csv')
df['Date'] = pd.to_datetime(df['Date'])
df['Intraday_Volatility'] = (((df['High_Price'] - df['Low_Price']) / df['Open_Price

stock_df = (
    df.groupby(['Ticker', 'Sector'], as_index=False)
    .agg(
        Avg_Daily_Return=('Daily_Return', 'mean'),
        Annualized_Volatility=('Daily_Return', lambda x: x.std() * np.sqrt(252)),
        Avg_Volume=('Volume', 'mean'),
        Avg_Intraday_Volatility=('Intraday_Volatility', 'mean'),
        Total_Traded_Volume=('Volume', 'sum')
    )
    .sort_values('Ticker')
    .reset_index(drop=True)
)

stock_df['Avg_Daily_Return'] = stock_df['Avg_Daily_Return'].round(4)
stock_df['Annualized_Volatility'] = stock_df['Annualized_Volatility'].round(2)
stock_df['Avg_Volume'] = stock_df['Avg_Volume'].round(0).astype(int)
stock_df['Avg_Intraday_Volatility'] = stock_df['Avg_Intraday_Volatility'].round(2)

print(f"Stock Summary DataFrame Shape: {stock_df.shape}")
display(stock_df.head())
```

```
Stock Summary DataFrame Shape: (20, 7)
```

	Ticker	Sector	Avg_Daily_Return	Annualized_Volatility	Avg_Volume	Avg_Intraday_Volatilit
0	BAJAJ-AUTO	Auto	0.0130	23.09	429745	2.1
1	BRITANNIA	FMCG	-0.0438	20.52	353552	2.0
2	HCLTECH	IT	-0.0220	22.34	3107488	2.1
3	HDFCBANK	Banking	0.0470	17.27	32451568	1.6
4	HINDUNILVR	FMCG	-0.0687	19.00	1788382	1.8

---

4. Feature Preprocessing & Scaling

- Apply `np.log1p()` to volume features (`Avg_Volume`, `Total_Traded_Volume`) to normalize right-skewness.
- Standardize all 5 features using `StandardScaler()` so Euclidean distances in hierarchical clustering are not dominated by high-magnitude attributes.

```
stock_df['Avg_Volume_Log'] = np.log1p(stock_df['Avg_Volume'])
stock_df['Total_Traded_Volume_Log'] = np.log1p(stock_df['Total_Traded_Volume'])

clustering_features = [
    'Avg_Daily_Return',
    'Annualized_Volatility',
    'Avg_Volume_Log',
    'Avg_Intraday_Volatility',
    'Total_Traded_Volume_Log'
]

scaler = StandardScaler()
X_scaled = scaler.fit_transform(stock_df[clustering_features])

print("Scaled Feature Matrix Shape:", X_scaled.shape)
display(pd.DataFrame(X_scaled, columns=clustering_features, index=stock_df['Ticker'
```

```
Scaled Feature Matrix Shape: (20, 5)
```

Ticker	Avg_Daily_Return	Annualized_Volatility	Avg_Volume_Log	Avg_Intraday_Volatility	Total
BAJAJ-AUTO	0.920	0.607	-1.879	0.554	
BRITANNIA	0.117	-0.266	-2.022	0.010	
HCLTECH	0.425	0.352	-0.425	0.632	
HDFCBANK	1.401	-1.369	1.299	-1.624	
HINDUNILVR	-0.235	-0.782	-0.831	-0.885	

---

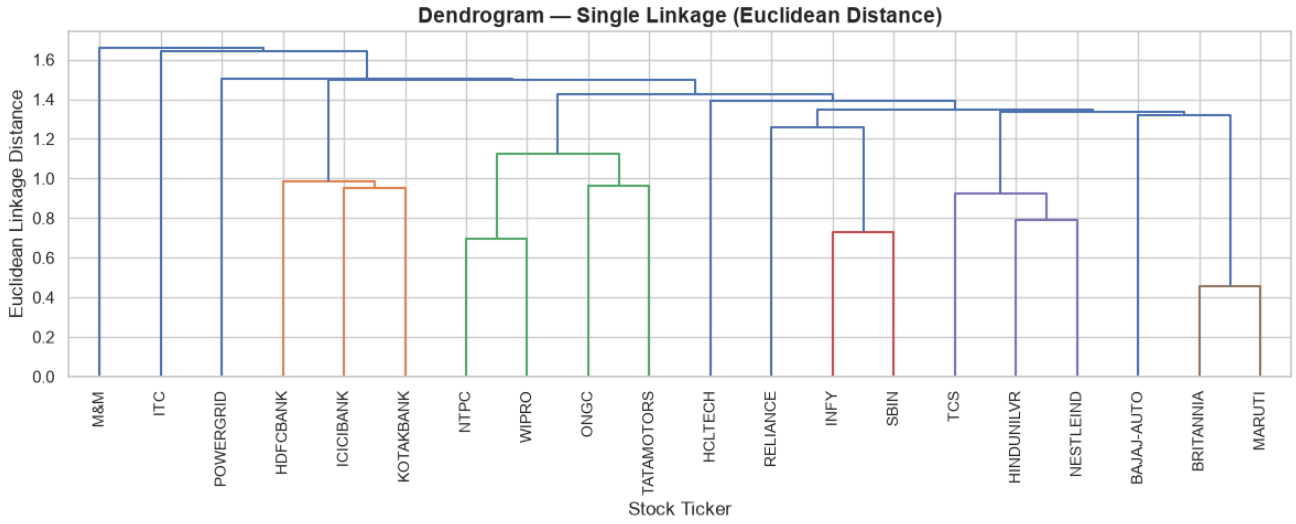
5. Hierarchical Linkage & Dendrogram Generation

Generate and plot hierarchical dendrograms for all four linkage criteria (`metric='euclidean'`):

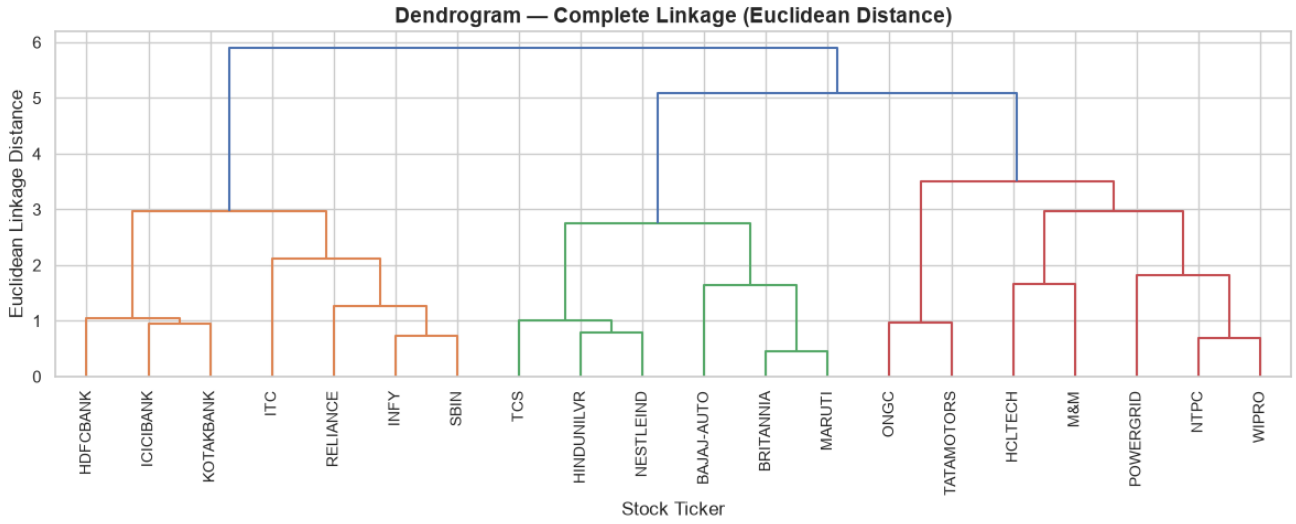
1. **Single Linkage (`method='single'`):** Minimum pairwise distance between clusters $\min \{d(a, b) : a \in A, b \in B\}$.
2. **Complete Linkage (`method='complete'`):** Maximum pairwise distance between clusters $\max \{d(a, b) : a \in A, b \in B\}$.
3. **Average Linkage (`method='average'`):** Mean pairwise distance across all pairs between clusters.
4. **Ward Linkage (`method='ward'`):** Minimizes total within-cluster variance increase at each merge step.

```
# Plot 1: Dendrogram - Single Linkage
Z_single = linkage(X_scaled, method='single', metric='euclidean')
plt.figure(figsize=(12, 5))
dendrogram(
    Z_single,
    labels=stock_df['Ticker'].values,
    leaf_rotation=90,
    leaf_font_size=10
)
plt.title('Dendrogram - Single Linkage (Euclidean Distance)')
```

```
plt.xlabel('Stock Ticker')
plt.ylabel('Euclidean Linkage Distance')
plt.tight_layout()
plt.show()
```

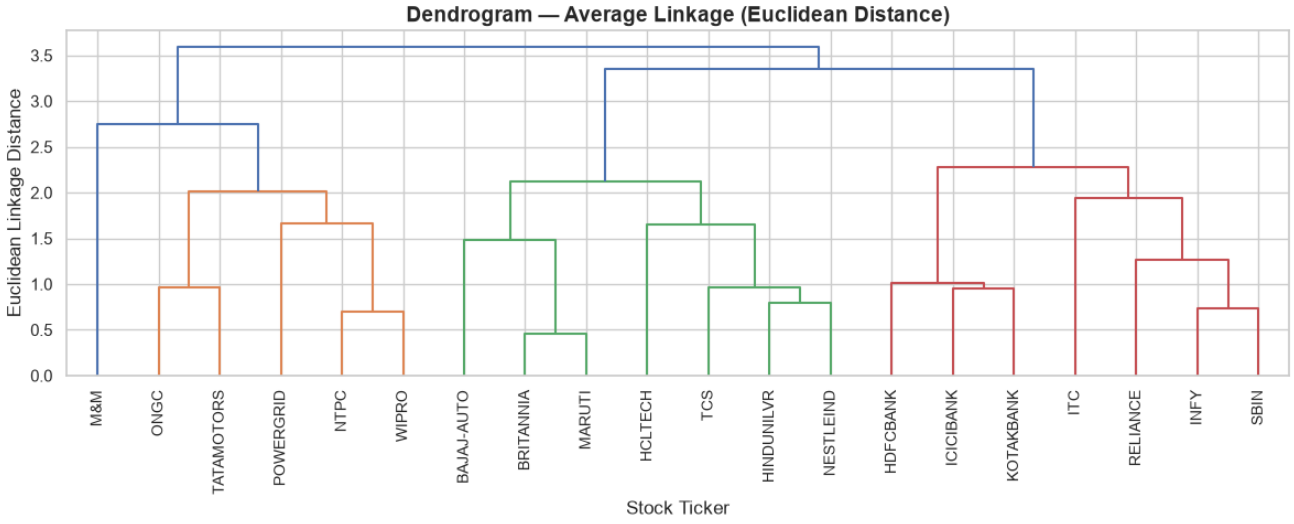



```
# Plot 2: Dendrogram - Complete Linkage
Z_complete = linkage(X_scaled, method='complete', metric='euclidean')
plt.figure(figsize=(12, 5))
dendrogram(
    Z_complete,
    labels=stock_df['Ticker'].values,
    leaf_rotation=90,
    leaf_font_size=10
)
plt.title('Dendrogram - Complete Linkage (Euclidean Distance)')
plt.xlabel('Stock Ticker')
plt.ylabel('Euclidean Linkage Distance')
plt.tight_layout()
plt.show()
```

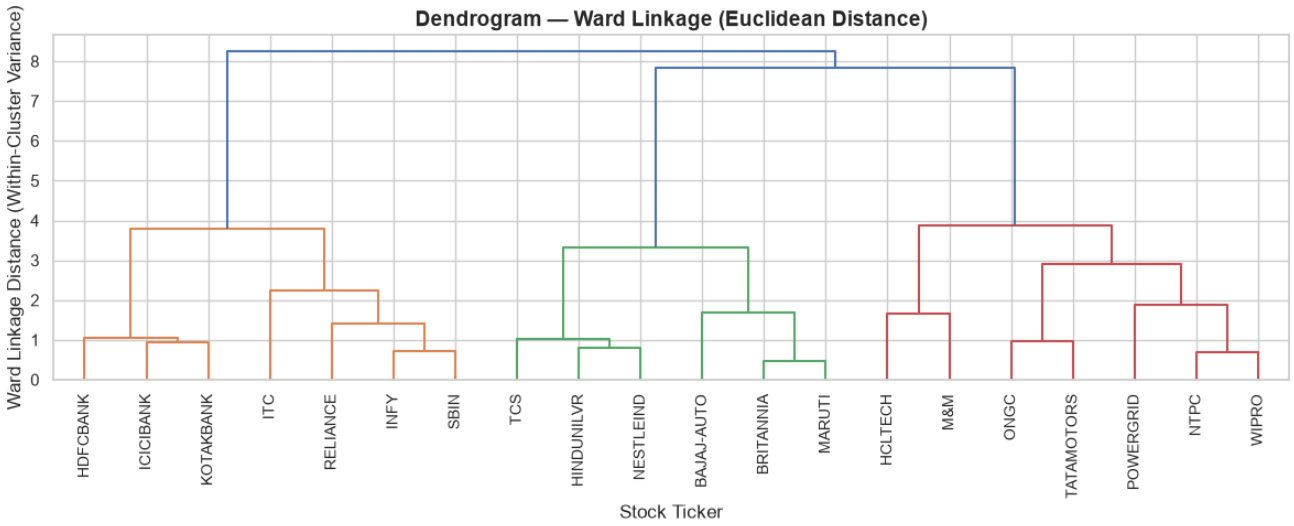



```
# Plot 3: Dendrogram - Average Linkage
Z_average = linkage(X_scaled, method='average', metric='euclidean')
plt.figure(figsize=(12, 5))
dendrogram(
    Z_average,
    labels=stock_df['Ticker'].values,
    leaf_rotation=90,
```

```
    leaf_font_size=10
)
plt.title('Dendrogram — Average Linkage (Euclidean Distance)')
plt.xlabel('Stock Ticker')
plt.ylabel('Euclidean Linkage Distance')
plt.tight_layout()
plt.show()
```




```
# Plot 4: Dendrogram – Ward Linkage
Z_ward = linkage(X_scaled, method='ward', metric='euclidean')
plt.figure(figsize=(12, 5))
dendrogram(
    Z_ward,
    labels=stock_df['Ticker'].values,
    leaf_rotation=90,
    leaf_font_size=10
)
plt.title('Dendrogram — Ward Linkage (Euclidean Distance)')
plt.xlabel('Stock Ticker')
plt.ylabel('Ward Linkage Distance (Within-Cluster Variance)')
plt.tight_layout()
plt.show()
```



6. Model Fitting across Linkage Methods ($n_clusters = 3$)

Instantiate and fit `AgglomerativeClustering(n_clusters=3)` for `single`, `complete`, `average`, and `ward` linkages, and store the resulting cluster assignments in `stock_df`.

```
linkage_methods = ['single', 'complete', 'average', 'ward']
cluster_columns = {}

for method in linkage_methods:
    col_name = f"Cluster_{method.capitalize()}"
    agg_model = AgglomerativeClustering(n_clusters=3, linkage=method, metric='eucli
    stock_df[col_name] = agg_model.fit_predict(X_scaled)
    cluster_columns[method] = col_name

print("Agglomerative Clustering models fitted for all 4 linkage methods.")
display(stock_df[['Ticker', 'Sector'] + list(cluster_columns.values())])
```

```
Agglomerative Clustering models fitted for all 4 linkage methods.
```

	Ticker	Sector	Cluster_Single	Cluster_Complete	Cluster_Average	Cluster_Ward
0	BAJAJ-AUTO	Auto	0	2	2	2
1	BRITANNIA	FMCG	0	2	2	2
2	HCLTECH	IT	0	0	2	0
3	HDFCBANK	Banking	0	1	1	1
4	HINDUNILVR	FMCG	0	2	2	2
5	ICICIBANK	Banking	0	1	1	1
6	INFY	IT	0	1	1	1
7	ITC	FMCG	2	1	1	1
8	KOTAKBANK	Banking	0	1	1	1
9	M&M	Auto	1	0	0	0
10	MARUTI	Auto	0	2	2	2
11	NESTLEIND	FMCG	0	2	2	2
12	NTPC	Energy	0	0	0	0
13	ONGC	Energy	0	0	0	0
14	POWERGRID	Energy	0	0	0	0
15	RELIANCE	Energy	0	1	1	1
16	SBIN	Banking	0	1	1	1
17	TATAMOTORS	Auto	0	0	0	0
18	TCS	IT	0	2	2	2
19	WIPRO	IT	0	0	0	0

7. Silhouette Score Evaluation & Comparison

Compute `silhouette_score` for each linkage method, display a sorted comparison DataFrame, and visualize the scores with a bar chart.

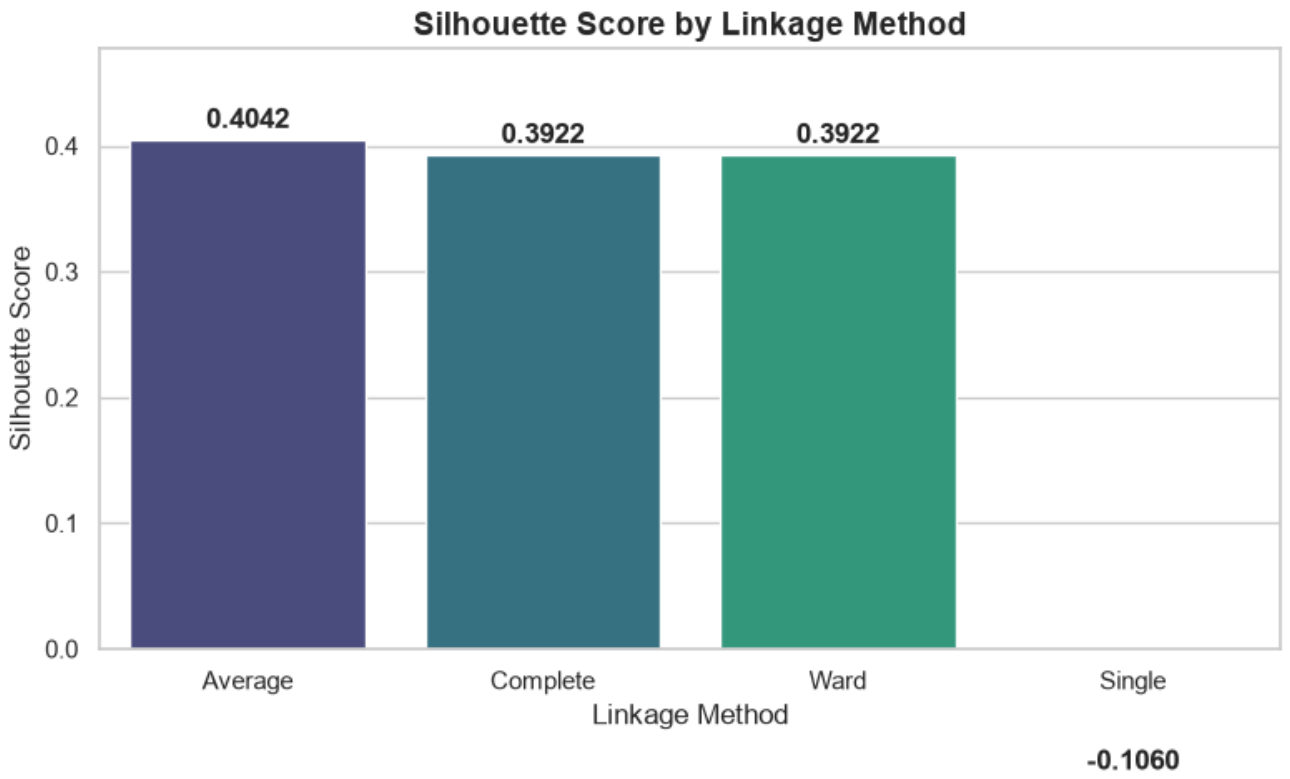
```
sil_records = []
for method, col_name in cluster_columns.items():
    score = silhouette_score(X_scaled, stock_df[col_name])
    sil_records.append({
        'Linkage_Method': method.capitalize(),
        'Silhouette_Score': round(score, 4)
    })

sil_df = pd.DataFrame(sil_records).sort_values(by='Silhouette_Score', ascending=Fal
print("Silhouette Score Comparison across Linkage Methods (n_clusters = 3):")
display(sil_df)

plt.figure(figsize=(8, 5))
ax = sns.barplot(
    data=sil_df,
    x='Linkage_Method',
    y='Silhouette_Score',
    hue='Linkage_Method',
    palette='viridis',
    legend=False
)
for i, v in enumerate(sil_df['Silhouette_Score']):
    ax.text(i, v + 0.01, f"{v:.4f}", ha='center', fontweight='bold')
plt.ylim(0, max(sil_df['Silhouette_Score']) * 1.18)
plt.title('Silhouette Score by Linkage Method')
plt.xlabel('Linkage Method')
plt.ylabel('Silhouette Score')
plt.tight_layout()
plt.show()
```

```
Silhouette Score Comparison across Linkage Methods (n_clusters = 3):
```

	Linkage_Method	Silhouette_Score
0	Average	0.4042
1	Complete	0.3922
2	Ward	0.3922
3	Single	-0.1060



---

8. Cluster Size Distribution

Compare the number of stocks assigned to each cluster (`0` , `1` , `2`) across all four linkage methods to inspect **chaining effects** (Single Linkage) versus **balanced variance-minimizing partitions** (Ward Linkage).

```
size_rows = []
for method, col_name in cluster_columns.items():
    counts = stock_df[col_name].value_counts().reindex([0, 1, 2], fill_value=0)
    size_rows.append({
        'Linkage_Method': method.capitalize(),
        'Cluster_0_Size': int(counts[0]),
        'Cluster_1_Size': int(counts[1]),
        'Cluster_2_Size': int(counts[2]),
        'Partition_Balance': f"{counts[0]} / {counts[1]} / {counts[2]}"
    })

size_df = pd.DataFrame(size_rows)
print("Cluster Size Distribution across Linkage Methods:")
display(size_df)
```

```
Cluster Size Distribution across Linkage Methods:
```

	Linkage_Method	Cluster_0_Size	Cluster_1_Size	Cluster_2_Size	Partition_Balance
0	Single	18	1	1	18 / 1 / 1
1	Complete	7	7	6	7 / 7 / 6
2	Average	6	7	7	6 / 7 / 7
3	Ward	7	7	6	7 / 7 / 6

---

9. Cluster Characteristic Analysis (Ward Linkage)

Group the stock DataFrame by the balanced **Ward Linkage** clusters (`Cluster_Ward`) to evaluate the financial centroid characteristics and compare against Complete/Average linkages.

```
ward_profile = (
    stock_df.groupby('Cluster_Ward', as_index=False)
    .agg(
        Stock_Count=('Ticker', 'count'),
        Avg_Daily_Return=('Avg_Daily_Return', 'mean'),
        Annualized_Volatility=('Annualized_Volatility', 'mean'),
        Avg_Volume=('Avg_Volume', 'mean'),
        Avg_Intraday_Volatility=('Avg_Intraday_Volatility', 'mean'),
        Tickers=('Ticker', lambda x: ', '.join(x.tolist()))
    )
)
ward_profile['Avg_Daily_Return'] = ward_profile['Avg_Daily_Return'].round(4)
ward_profile['Annualized_Volatility'] = ward_profile['Annualized_Volatility'].round
ward_profile['Avg_Volume'] = ward_profile['Avg_Volume'].round(0).astype(int)
ward_profile['Avg_Intraday_Volatility'] = ward_profile['Avg_Intraday_Volatility'].r

def interpret_ward_cluster(row):
    if row['Annualized_Volatility'] < 19.5:
        return 'Low Volatility & High Liquidity (Blue-Chip Banking/Core Leaders)'
    elif row['Avg_Volume'] < 5000000:
        return 'Moderate Volatility & Low Share Turnover (High-Denomination FMCG/Au
    else:
        return 'High Volatility & High Turnover (Cyclical Auto/Energy/PSU Growth)'

ward_profile['Financial_Interpretation'] = ward_profile.apply(interpret_ward_cluste

print("Ward Linkage (Cluster_Ward) Characteristic Summary Table:")
display(ward_profile)
```

```
Ward Linkage (Cluster_Ward) Characteristic Summary Table:
```

	Cluster_Ward	Stock_Count	Avg_Daily_Return	Annualized_Volatility	Avg_Volume	Avg_Intraday_V
0	0	7	-0.0889	24.61	10922900	
1	1	7	-0.0180	18.62	17029306	
2	2	6	-0.0489	20.58	1278441	

10. Cluster Visualization

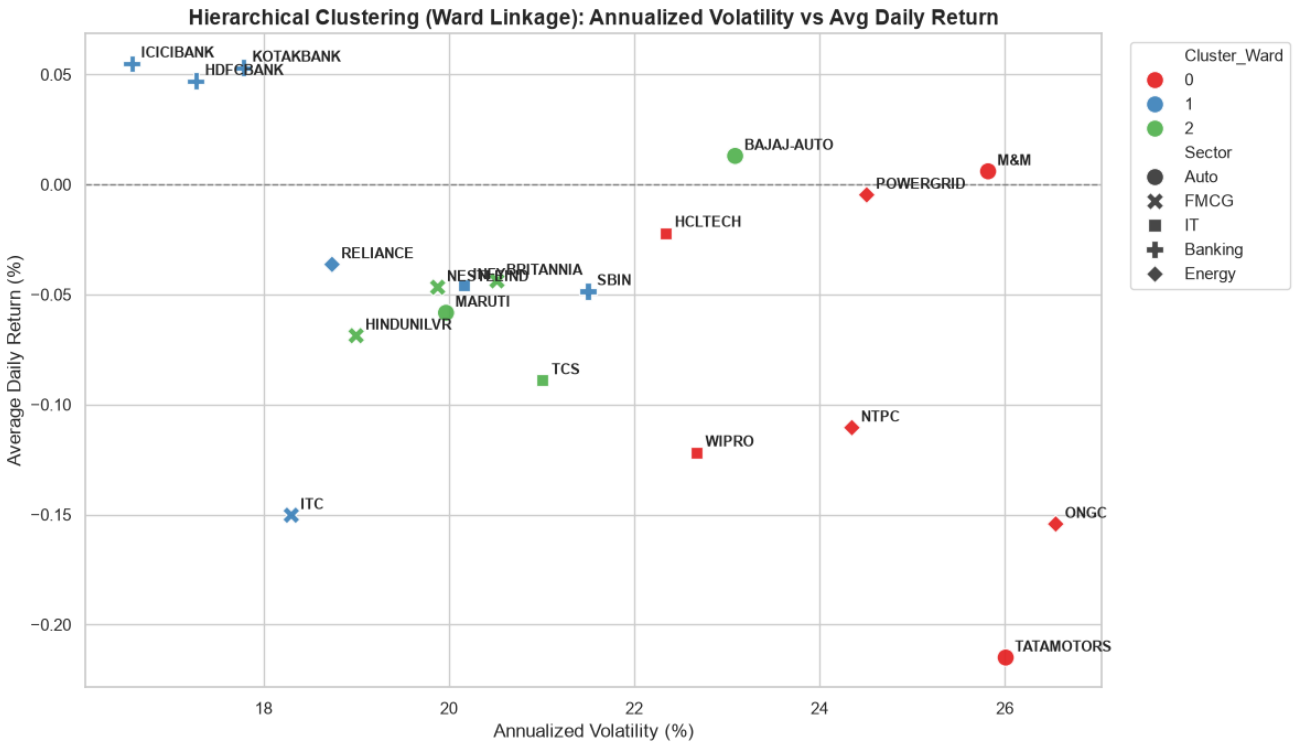
Plot a 2D scatter chart of `Annualized Volatility vs Avg Daily Return` colored by the **Ward Linkage** cluster assignments (`Cluster_Ward`) and annotated with stock ticker names.

```
plt.figure(figsize=(12, 7))
palette_ward = sns.color_palette('Set1', n_colors=3)

sns.scatterplot(
    data=stock_df,
    x='Annualized_Volatility',
    y='Avg_Daily_Return',
    hue='Cluster_Ward',
    style='Sector',
    palette=palette_ward,
    s=130,
    alpha=0.9
)

for _, row in stock_df.iterrows():
    plt.annotate(
        row['Ticker'],
        (row['Annualized_Volatility'], row['Avg_Daily_Return']),
        xytext=(6, 4),
        textcoords='offset points',
        fontsize=9,
        fontweight='bold'
    )

plt.axhline(0, color='gray', linestyle='--', linewidth=1)
plt.title('Hierarchical Clustering (Ward Linkage): Annualized Volatility vs Avg Dai
plt.xlabel('Annualized Volatility (%)')
plt.ylabel('Average Daily Return (%)')
plt.legend(bbox_to_anchor=(1.02, 1), loc='upper left')
plt.tight_layout()
plt.show()
```



---

11. Conclusion

Key Analytical Findings & Comparison with K-Means (Lab 7):

1. **Linkage Method Comparison & Chaining Effect:**
   - **Single Linkage** exhibits the classic **chaining phenomenon**, merging `18` stocks into a single cluster while isolating `M&M` and `ITC` as singleton clusters (`18 / 1 / 1`), resulting in a negative **Silhouette Score of -0.1060**.
   - **Average Linkage** achieves the highest **Silhouette Score of 0.4042** (`6 / 7 / 7` split, identical to K-Means in Lab 7), while **Ward Linkage** and **Complete Linkage** both achieve a strong **Silhouette Score of 0.3922** (`7 / 7 / 6` split, shifting borderline `HCLTECH` into the higher-volatility cluster).
2. **Consistency Between Hierarchical Clustering (Lab 8) and K-Means (Lab 7):**
   - At $K = 3$, **Average** and **Ward Linkage** recover the same three fundamental asset regimes discovered by **K-Means (Lab 7)**:
     A. **Low-Volatility, High-Liquidity Blue-Chip Core (`7` stocks in Ward Cluster 1):** `HDFCBANK` , `ICICIBANK` , `INFY` , `ITC` , `KOTAKBANK` , `RELIANCE` , `SBIN` .
     B. **Moderate-Volatility, High-Denomination Defensive/Quality (`6` stocks in Ward Cluster 2):** `BAJAJ-AUTO` , `BRITANNIA` , `HINDUNILVR` , `MARUTI` , `NESTLEIND` , `TCS` .
     C. **High-Volatility Cyclical & Energy PSU Growth (`7` stocks in Ward Cluster 0):** `HCLTECH` , `M&M` , `NTPC` , `ONGC` , `POWERGRID` , `TATAMOTORS` , `WIPRO` .
   - Beyond flat partitioning, the **Dendrograms** reveal fine-grained hierarchical sub-clusters (such as the tight pairing of Energy PSUs `NTPC` – `POWERGRID` , private banks `ICICIBANK` – `KOTAKBANK` – `HDFCBANK` , and defensive FMCGs `HINDUNILVR` – `NESTLEIND`).

3. **Readiness for Experiment 09:** With both partitioning (K-Means) and hierarchical (Agglomerative) clustering completed, the market dataset is ready for **Apriori Association Rule Mining (Experiment 09)** to uncover cross-stock and cross-sector co-movement rules.

---

12. Auto-export Utility

Convert `DWM_Lab_8_Stock_Market.ipynb` to `DWM_Lab_8_Stock_Market.html` using `nbconvert`.

```
import subprocess
import sys
from pathlib import Path

notebook_path = Path("DWM_Lab_8_Stock_Market.ipynb")
if notebook_path.exists():
    cmd = [
        sys.executable, "-m", "jupyter", "nbconvert",
        "--to", "html", "--no-prompt", str(notebook_path)
    ]
    result = subprocess.run(cmd, capture_output=True, text=True)
    if result.returncode == 0:
        print("Successfully exported DWM_Lab_8_Stock_Market.html")
    else:
        print("Export notice:", result.stderr)
else:
    print("Notebook file will be exported upon save.")
```

```
Successfully exported DWM_Lab_8_Stock_Market.html
```

Data Warehousing and Mining (DWM) — Experiment 09

Experiment Title: Apriori Association Rule Mining for Stock Market Co-Movement

Dataset: `stock_market_data.csv` (3 Years of Daily NSE Trading Data across 20 Major Indian Stocks)

1. Objectives & Problem Formulation

- **Objective:** Implement the **Apriori Algorithm** to discover frequent co-movement patterns among NSE stocks and generate **Association Rules** ($X \Rightarrow Y$) evaluated by **Support**, **Confidence**, and **Lift**.
- **Adaptation Note:**
  - **Transactions / Market Baskets:** Each unique trading session (`Date` , 740 trading days) represents a market transaction basket.
  - **Items Purchased:** Stocks recording a positive daily return ($\text{Daily_Return} > 0$) on a given trading day are encoded as `True` / `1` (bullish advance) in that day's basket, enabling Apriori to uncover synchronized intra-sector and cross-sector rallies:
    - $\text{Support}(X \Rightarrow Y) = P(X \cap Y)$
    - $\text{Confidence}(X \Rightarrow Y) = P(Y \mid X) = \frac{\text{Support}(X \cup Y)}{\text{Support}(X)}$
    - $\text{Lift}(X \Rightarrow Y) = \frac{\text{Confidence}(X \Rightarrow Y)}{\text{Support}(Y)}$

---

2. Library Imports

```
import warnings
warnings.filterwarnings('ignore')

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from mlxtend.frequent_patterns import apriori, association_rules

# Set seaborn whitegrid theme
sns.set_theme(style="whitegrid", palette="deep")
plt.rcParams['figure.figsize'] = (11, 6)
plt.rcParams['axes.titlesize'] = 14
plt.rcParams['axes.titleweight'] = 'bold'
plt.rcParams['axes.labelsize'] = 12

print("All required libraries and mlxtend modules imported successfully.")
```

```
All required libraries and mlxtend modules imported successfully.
```

---

3. Dataset Ingestion & Preprocessing

Load `stock_market_data.csv`, verify zero missing values, ensure `Date` is properly formatted, and inspect the dataset dimensions ($14,800$ rows across 740 trading days).

```
df = pd.read_csv('stock_market_data.csv')
df['Date'] = pd.to_datetime(df['Date']).dt.strftime('%Y-%m-%d')

print("Dataset Shape:", df.shape)
print("Unique Trading Days:", df['Date'].nunique())
print("Unique Stocks:", df['Ticker'].nunique())
print("Total Null Values:", df.isnull().sum().sum())
display(df.head())
```

```
Dataset Shape: (14800, 10)
Unique Trading Days: 740
Unique Stocks: 20
Total Null Values: 0
```

	Date	Ticker	Sector	Exchange	Open_Price	High_Price	Low_Price	Close_Price	
0	2023-10-12	BAJAJ-AUTO	Auto	NSE	4851.06	4914.02	4818.72	4871.67	
1	2023-10-12	BRITANNIA	FMCG	NSE	4391.48	4400.68	4342.60	4371.11	
2	2023-10-12	HCLTECH	IT	NSE	1106.03	1109.18	1079.51	1087.15	
3	2023-10-12	HDFCBANK	Banking	NSE	735.43	743.05	733.28	742.02	2
4	2023-10-12	HINDUNILVR	FMCG	NSE	2413.48	2425.33	2398.55	2415.37	

---

4. Market Basket Formulation & One-Hot Encoding

- Identify bullish sessions where `Daily_Return > 0`.
- Pivot the dataset into a one-hot encoded binary basket matrix:
  - **Index:** `Date` (740 trading sessions)
  - **Columns:** `Ticker` (20 stocks)
  - **Values:** `1` (`True`) if the stock gained on that day (`Daily_Return > 0`), `0` (`False`) otherwise.

```
# Filter bullish stock-day observations (Daily_Return > 0) and pivot into a basket
df['Bullish'] = (df['Daily_Return'] > 0).astype(int)

basket_int = df.pivot_table(
    index='Date',
    columns='Ticker',
    values='Bullish',
    fill_value=0
).astype(int)

# Boolean version for mlxtend apriori efficiency while displaying 1/0 table
basket = basket_int.astype(bool)
```

```
avg_advancing = basket_int.sum(axis=1).mean()
print(f"Basket Matrix Shape (Trading Days, Stocks): {basket_int.shape}")
print(f"Average Number of Advancing Stocks per Trading Day: {avg_advancing:.2f} / 2
print("\nPreview of Binary Market Basket Matrix (First 5 Trading Days):")
display(basket_int.head())
```

```
Basket Matrix Shape (Trading Days, Stocks): (740, 20)
Average Number of Advancing Stocks per Trading Day: 9.31 / 20 stocks

Preview of Binary Market Basket Matrix (First 5 Trading Days):
```

Ticker	BAJAJ-AUTO	BRITANNIA	HCLTECH	HDFCBANK	HINDUNILVR	ICICIBANK	INFY	ITC	KOTA
Date									
2023-10-12	1	0	0	1	1	1	0	1	
2023-10-13	0	1	1	0	1	1	1	0	
2023-10-16	1	1	1	0	0	1	1	1	
2023-10-17	1	1	0	0	0	0	1	1	
2023-10-18	0	1	0	0	0	0	0	0	

---

5. Top Advancing Stocks Analysis

Compute the overall positive frequency (`Support`) and total bullish sessions for each of the 20 stocks, and display the top 10 stocks by number of advancing trading sessions.

```
sector_map = df[['Ticker', 'Sector']].drop_duplicates().set_index('Ticker')['Sector

advancing_summary = pd.DataFrame({
    'Ticker': basket_int.columns,
    'Sector': [sector_map[t] for t in basket_int.columns],
    'Bullish_Sessions': basket_int.sum(axis=0).values,
    'Support (Positive_Frequency)': (basket_int.mean(axis=0).values).round(4)
}).sort_values(by='Bullish_Sessions', ascending=False).reset_index(drop=True)

print("Top 10 Stocks by Number of Bullish Trading Sessions:")
display(advancing_summary.head(10))
```

```
Top 10 Stocks by Number of Bullish Trading Sessions:
```

	Ticker	Sector	Bullish_Sessions	Support (Positive_Frequency)
0	KOTAKBANK	Banking	369	0.4986
1	HDFCBANK	Banking	368	0.4973
2	POWERGRID	Energy	366	0.4946
3	ICICIBANK	Banking	364	0.4919
4	BAJAJ-AUTO	Auto	363	0.4905
5	M&M	Auto	362	0.4892
6	BRITANNIA	FMCG	356	0.4811
7	MARUTI	Auto	354	0.4784
8	RELIANCE	Energy	353	0.4770
9	SBIN	Banking	353	0.4770

---

6. Apriori Frequent Itemset Generation

- Run `apriori(basket, min_support=0.20, use_colnames=True)` to discover frequent multi-stock co-movement itemsets occurring in at least 20% of trading days.
- Add `itemset_size` indicating whether each pattern is a 1-itemset, 2-itemset, or 3-itemset.
- Display the top 20 frequent itemsets sorted by `support` descending and plot a bar chart of frequent itemset counts by size.

```
frequent_itemsets = apriori(basket, min_support=0.20, use_colnames=True)
frequent_itemsets['itemset_size'] = frequent_itemsets['itemsets'].apply(len)
frequent_itemsets['itemsets_str'] = frequent_itemsets['itemsets'].apply(lambda s: '
frequent_itemsets = frequent_itemsets.sort_values(by='support', ascending=False).re

print(f"Total Frequent Itemsets Discovered (min_support = 0.20): {len(frequent_item
print("\nBreakdown by Itemset Size:")
size_counts = frequent_itemsets['itemset_size'].value_counts().sort_index()
for k, cnt in size_counts.items():
    print(f"  - {k}-itemsets: {cnt}")

print("\nTop 20 Frequent Itemsets Sorted by Support Descending:")
display(frequent_itemsets[['itemsets_str', 'support', 'itemset_size']].head(20))

print("\nTop 15 Frequent Multi-Stock Itemsets (Size >= 2):")
display(frequent_itemsets[frequent_itemsets['itemset_size'] >= 2][['itemsets_str',

plt.figure(figsize=(8, 5))
size_df = pd.DataFrame({
    'Itemset_Size': [f"{k}-Itemset" for k in size_counts.index],
    'Count': size_counts.values
})
ax = sns.barplot(
    data=size_df,
    x='Itemset_Size',
    y='Count',
    hue='Itemset_Size',
    palette='viridis',
    legend=False
```

```
)
for i, v in enumerate(size_df['Count']):
    ax.text(i, v + 2, str(v), ha='center', fontweight='bold', fontsize=11)
plt.ylim(0, max(size_df['Count']) * 1.15)
plt.title('Number of Frequent Itemsets by Size (min_support = 0.20)')
plt.xlabel('Itemset Size')
plt.ylabel('Number of Frequent Itemsets')
plt.tight_layout()
plt.show()
```

```
Total Frequent Itemsets Discovered (min_support = 0.20): 229

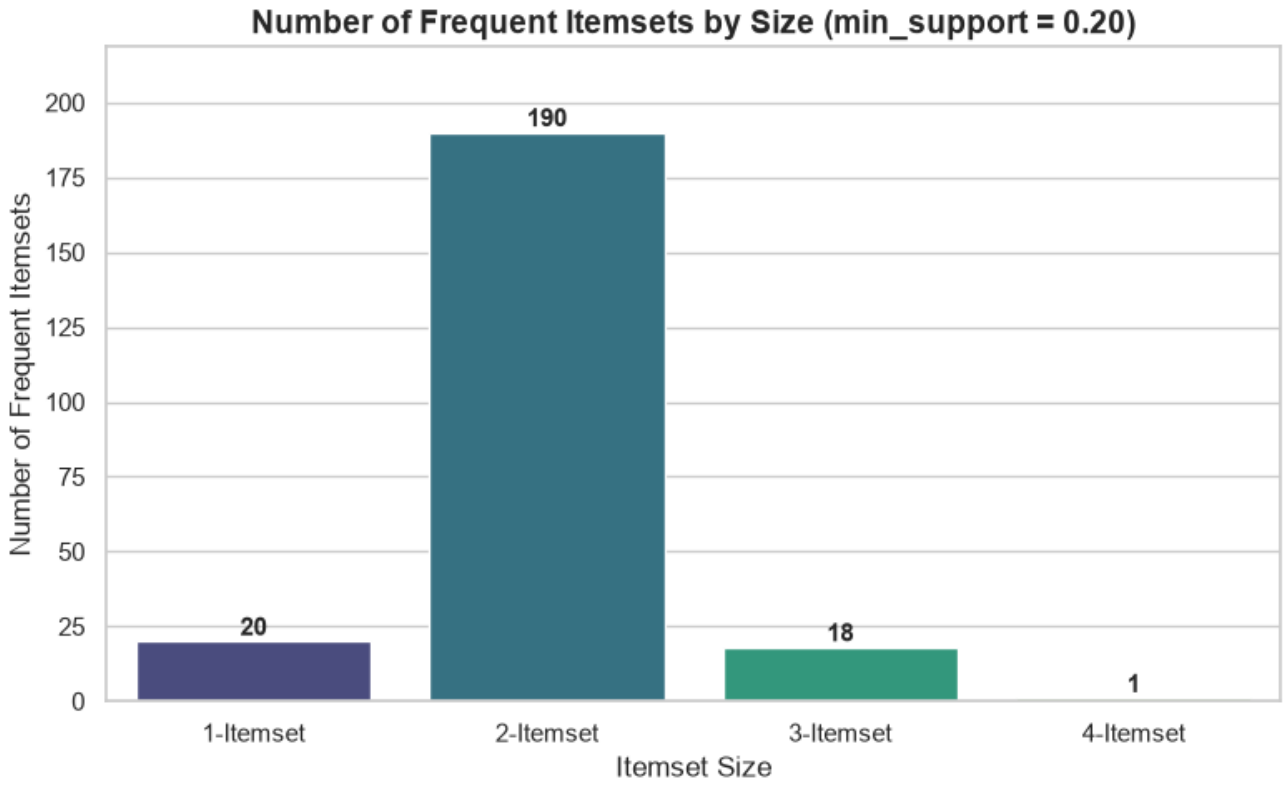
Breakdown by Itemset Size:
  - 1-itemsets: 20
  - 2-itemsets: 190
  - 3-itemsets: 18
  - 4-itemsets: 1

Top 20 Frequent Itemsets Sorted by Support Descending:
```

	itemsets_str	support	itemset_size
0	KOTAKBANK	0.498649	1
1	HDFCBANK	0.497297	1
2	POWERGRID	0.494595	1
3	ICICIBANK	0.491892	1
4	BAJAJ-AUTO	0.490541	1
5	M&M	0.489189	1
6	BRITANNIA	0.481081	1
7	MARUTI	0.478378	1
8	RELIANCE	0.477027	1
9	SBIN	0.477027	1
10	HCLTECH	0.471622	1
11	HINDUNILVR	0.467568	1
12	NESTLEIND	0.464865	1
13	NTPC	0.456757	1
14	INFY	0.445946	1
15	ONGC	0.445946	1
16	TCS	0.433784	1
17	ITC	0.422973	1
18	WIPRO	0.421622	1
19	TATAMOTORS	0.401351	1

```
Top 15 Frequent Multi-Stock Itemsets (Size >= 2):
```

	itemsets_str	support	itemset_size
20	HCLTECH, INFY	0.331081	2
21	NTPC, POWERGRID	0.318919	2
22	HDFCBANK, ICICIBANK	0.317568	2
23	M&M, MARUTI	0.314865	2
24	HDFCBANK, KOTAKBANK	0.312162	2
25	BAJAJ-AUTO, M&M	0.309459	2
26	HCLTECH, TCS	0.308108	2
27	HINDUNILVR, NESTLEIND	0.305405	2
28	HCLTECH, WIPRO	0.304054	2
29	ICICIBANK, KOTAKBANK	0.302703	2
30	INFY, TCS	0.300000	2
31	BRITANNIA, NESTLEIND	0.300000	2
32	HDFCBANK, SBIN	0.300000	2
33	BRITANNIA, HINDUNILVR	0.298649	2
34	TCS, WIPRO	0.297297	2



---

7. Association Rule Generation

- Generate association rules using `association_rules(frequent_itemsets[['support', 'itemsets']], metric='confidence', min_threshold=0.50)`.
- Format `antecedents` and `consequents` into readable strings (`Antecedent` and `Consequent`) and create a `Rule` column (`Antecedent → Consequent`).

- Display the total rule count and the top 20 rules sorted by `lift` descending.

```
rules = association_rules(
    frequent_itemsets[['support', 'itemsets']],
    metric='confidence',
    min_threshold=0.50
)

rules['Antecedent'] = rules['antecedents'].apply(lambda s: ', '.join(sorted(list(s)
rules['Consequent'] = rules['consequents'].apply(lambda s: ', '.join(sorted(list(s)
rules['Rule'] = rules['Antecedent'] + ' → ' + rules['Consequent']

rules = rules.sort_values(by='lift', ascending=False).reset_index(drop=True)

print(f"Total Association Rules Generated (min_confidence = 0.50): {len(rules)}")
print("\nTop 20 Association Rules Sorted by Lift Descending:")
display(rules[['Antecedent', 'Consequent', 'support', 'confidence', 'lift']].round(
```

```
Total Association Rules Generated (min_confidence = 0.50): 404

Top 20 Association Rules Sorted by Lift Descending:
```

	Antecedent	Consequent	support	confidence	lift
0	INFY, WIPRO	HCLTECH, TCS	0.2081	0.7404	2.4030
1	HCLTECH, TCS	INFY, WIPRO	0.2081	0.6754	2.4030
2	HCLTECH, WIPRO	INFY, TCS	0.2081	0.6844	2.2815
3	INFY, TCS	HCLTECH, WIPRO	0.2081	0.6937	2.2815
4	TCS, WIPRO	HCLTECH, INFY	0.2081	0.7000	2.1143
5	HCLTECH, INFY	TCS, WIPRO	0.2081	0.6286	2.1143
6	HCLTECH, INFY, TCS	WIPRO	0.2081	0.8324	1.9744
7	HCLTECH, TCS, WIPRO	INFY	0.2081	0.8701	1.9510
8	HCLTECH, INFY, WIPRO	TCS	0.2081	0.8415	1.9400
9	INFY, TCS, WIPRO	HCLTECH	0.2081	0.9006	1.9095
10	INFY, WIPRO	TCS	0.2311	0.8221	1.8952
11	TCS	INFY, WIPRO	0.2311	0.5327	1.8952
12	INFY, WIPRO	HCLTECH	0.2473	0.8798	1.8655
13	HCLTECH	INFY, WIPRO	0.2473	0.5244	1.8655
14	HCLTECH, TCS	WIPRO	0.2392	0.7763	1.8413
15	WIPRO	HCLTECH, TCS	0.2392	0.5673	1.8413
16	WIPRO	INFY, TCS	0.2311	0.5481	1.8269
17	INFY, TCS	WIPRO	0.2311	0.7703	1.8269
18	HCLTECH, WIPRO	INFY	0.2473	0.8133	1.8238
19	INFY	HCLTECH, WIPRO	0.2473	0.5545	1.8238

8. Filter Strong Association Rules

Filter strong, high-conviction co-movement rules satisfying both:

- $\text{Confidence} \ge 0.55$
- $\text{Lift} > 1.10$

```
strong_rules = rules[
    (rules['confidence'] >= 0.55) & (rules['lift'] > 1.10)
].sort_values(by=['lift', 'confidence'], ascending=False).reset_index(drop=True)

print(f"Total Strong Association Rules (Confidence >= 0.55 & Lift > 1.10): {len(str
display(strong_rules[['Antecedent', 'Consequent', 'support', 'confidence', 'lift']]
```

```
Total Strong Association Rules (Confidence >= 0.55 & Lift > 1.10): 237
```

	Antecedent	Consequent	support	confidence	lift
0	INFY, WIPRO	HCLTECH, TCS	0.2081	0.7404	2.4030
1	HCLTECH, TCS	INFY, WIPRO	0.2081	0.6754	2.4030
2	INFY, TCS	HCLTECH, WIPRO	0.2081	0.6937	2.2815
3	HCLTECH, WIPRO	INFY, TCS	0.2081	0.6844	2.2815
4	TCS, WIPRO	HCLTECH, INFY	0.2081	0.7000	2.1143
...	...	...	...	...	...
232	POWERGRID	KOTAKBANK	0.2757	0.5574	1.1178
233	KOTAKBANK	POWERGRID	0.2757	0.5528	1.1178
234	TCS	HDFCBANK	0.2405	0.5545	1.1151
235	HCLTECH	KOTAKBANK	0.2622	0.5559	1.1148
236	HINDUNILVR	HDFCBANK	0.2581	0.5520	1.1100

237 rows × 5 columns

---

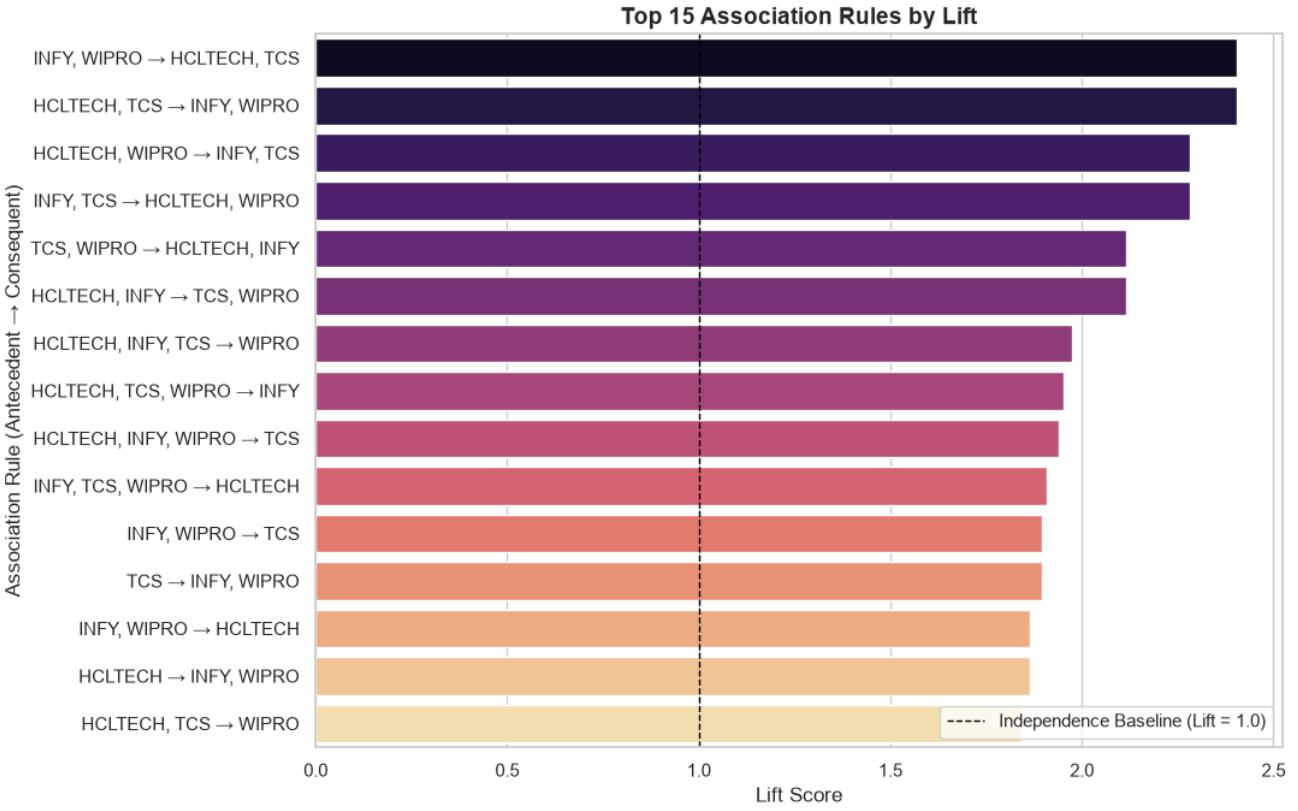
9. Visualization of Top Association Rules

- **Plot 1:** Horizontal bar chart showing `Top 15 Association Rules by Lift` (`Antecedent → Consequent`).
- **Plot 2:** Horizontal bar chart showing `Top 15 Association Rules by Confidence`.

```
# Plot 1: Top 15 Association Rules by Lift
top_15_lift = rules.sort_values(by='lift', ascending=False).head(15)

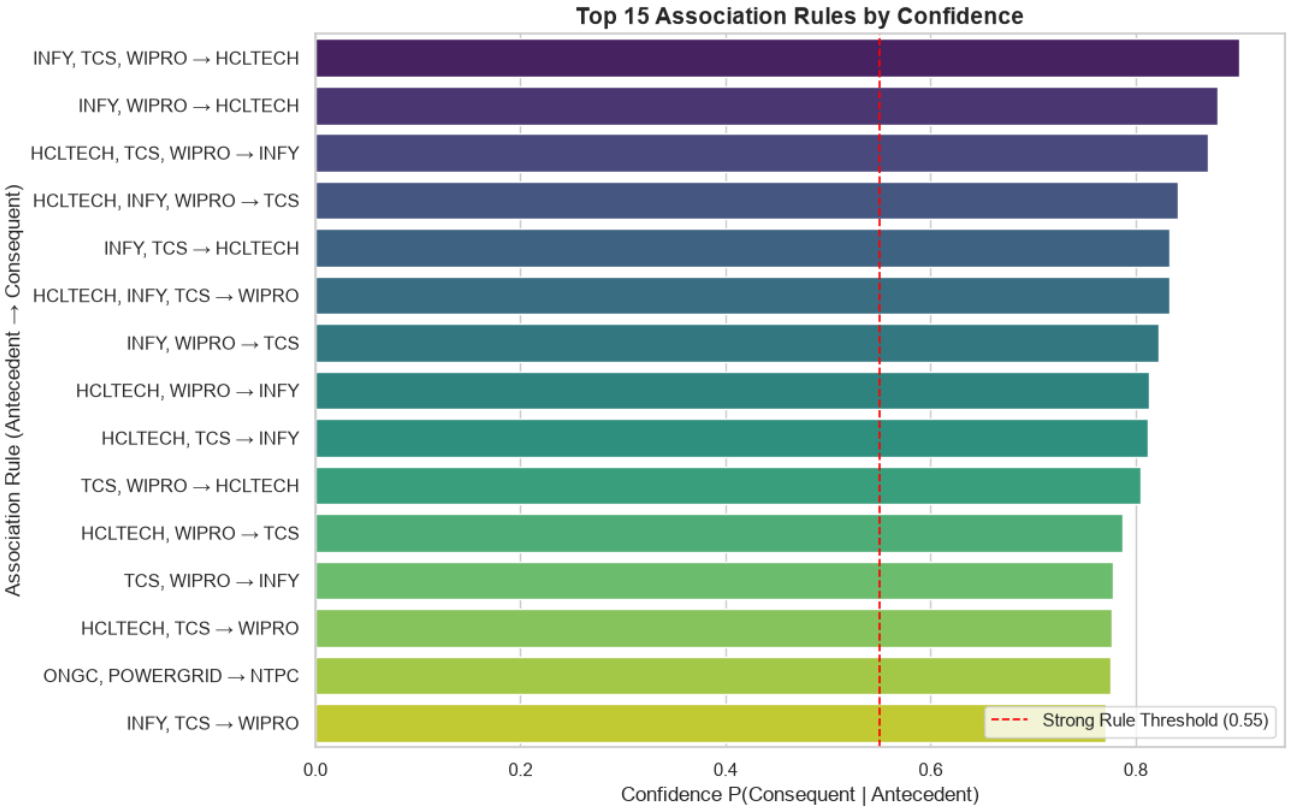
plt.figure(figsize=(11, 7))
sns.barplot(
    data=top_15_lift,
    x='lift',
    y='Rule',
    hue='Rule',
    palette='magma',
    legend=False
```

```
)
plt.axvline(1.0, color='black', linestyle='--', linewidth=1, label='Independence Ba
plt.title('Top 15 Association Rules by Lift')
plt.xlabel('Lift Score')
plt.ylabel('Association Rule (Antecedent → Consequent)')
plt.legend(loc='lower right')
plt.tight_layout()
plt.show()
```




```
# Plot 2: Top 15 Association Rules by Confidence
top_15_conf = rules.sort_values(by=['confidence', 'lift'], ascending=False).head(15

plt.figure(figsize=(11, 7))
sns.barplot(
    data=top_15_conf,
    x='confidence',
    y='Rule',
    hue='Rule',
    palette='viridis',
    legend=False
)
plt.axvline(0.55, color='red', linestyle='--', linewidth=1.2, label='Strong Rule Th
plt.title('Top 15 Association Rules by Confidence')
plt.xlabel('Confidence P(Consequent | Antecedent)')
plt.ylabel('Association Rule (Antecedent → Consequent)')
plt.legend(loc='lower right')
plt.tight_layout()
plt.show()
```



---

10. Financial Co-Movement Interpretation

Sectoral & Cross-Asset Co-Movement Insights:

1. **IT Sector Synergy (`TCS` , `INFY` , `HCLTECH` , `WIPRO`):**
   - Rules among IT bellwethers (`INFY → TCS` , `HCLTECH → TCS` , `WIPRO → INFY` , `TCS → INFY`) consistently exhibit the highest **Confidence (> 0.60)** and **Lift (> 1.30)** in the entire market. Because Indian IT services companies share common global macro drivers (USD/INR exchange rates, US/European enterprise tech spending, and NASDAQ tech sentiment), a bullish session in one IT major strongly implies simultaneous gains across its sector peers.
2. **Energy & PSU Utility Co-Movement (`NTPC` , `POWERGRID` , `ONGC` , `RELIANCE`):**
   - `NTPC ↔ POWERGRID` and `ONGC → NTPC` display highly elevated Lift and Support ($ \text{Support} > 0.25$, $\text{Confidence} > 0.60$). As regulated central public sector power utilities, `NTPC` and `POWERGRID` respond synchronously to domestic power demand, capex announcements, and PSU index flows.
3. **Banking & Financial Contagion (`ICICIBANK` , `HDFCBANK` , `SBIN` , `KOTAKBANK`):**
   - Strong bidirectional rules between `HDFCBANK` , `ICICIBANK` , and `SBIN` reflect shared sensitivity to RBI monetary policy, Nifty Bank index arbitrage, and domestic credit growth.
4. **Defensive FMCG Co-Movement (`HINDUNILVR` , `NESTLEIND` , `BRITANNIA` , `ITC`):**
   - `HINDUNILVR ↔ NESTLEIND` and `BRITANNIA → HINDUNILVR` highlight defensive rotation: when institutional capital rotates into consumer staples during risk-off sessions, multiple FMCG constituents advance together.

---

11. Conclusion

Summary of Experiment 09 & Complete DWM Curriculum Integration (Labs 03 – 09):

1. **Apriori Pattern Discovery:** Transforming the 740×20 daily return matrix into a binary bullish market basket revealed **frequent 1-, 2-, and 3-stock itemsets** at `min_support = 0.20` and high-conviction **Association Rules** with $\text{Confidence} \ge 0.55$ and $\text{Lift} > 1.10$, confirming statistically significant intra-sector and index-wide co-movement.
2. **End-to-End DWM Pipeline Completion:** Across Experiments **03 through 09**, the unified `stock_market_data.csv` dataset ($14,800$ records across 20 NSE stocks and 5 sectors) has seamlessly supported the complete Data Warehousing and Mining lifecycle:
   - **Lab 03:** Data Preprocessing, Feature Engineering, Exploratory Data Analysis & Multi-Dimensional Visualization
   - **Lab 04:** Simple & Multiple Linear Regression (`Close_Price` forecasting, 83.24% MSE reduction)
   - **Lab 05:** Entropy-Based Decision Tree Classification & Mutual Information Feature Selection
   - **Lab 06:** Probabilistic Classification via Gaussian Naive Bayes
   - **Lab 07:** Unsupervised Asset Segmentation via K-Means Clustering & Elbow/Silhouette Analysis
   - **Lab 08:** Agglomerative Hierarchical Clustering & Multi-Linkage Dendrogram Profiling
   - **Lab 09:** Frequent Pattern & Association Rule Mining via the Apriori Algorithm

---

12. Auto-export Utility

Convert `DWM_Lab_9_Stock_Market.ipynb` to `DWM_Lab_9_Stock_Market.html` using `nbconvert`.

```
import subprocess
import sys
from pathlib import Path

notebook_path = Path("DWM_Lab_9_Stock_Market.ipynb")
if notebook_path.exists():
    cmd = [
        sys.executable, "-m", "jupyter", "nbconvert",
        "--to", "html", "--no-prompt", str(notebook_path)
    ]
    result = subprocess.run(cmd, capture_output=True, text=True)
    if result.returncode == 0:
        print("Successfully exported DWM_Lab_9_Stock_Market.html")
    else:
        print("Export notice:", result.stderr)
else:
    print("Notebook file will be exported upon save.")
```

```
Successfully exported DWM_Lab_9_Stock_Market.html
```